

Module: Paper-2 | Theory of Computation

Notes

Coal India Management Trainee (CS) Recruitment 2026

Contains 31 PDF lessons

Generated: 3 September 2026

KnowledgeGate

Module

Paper-2 | Theory of Computation

Topic

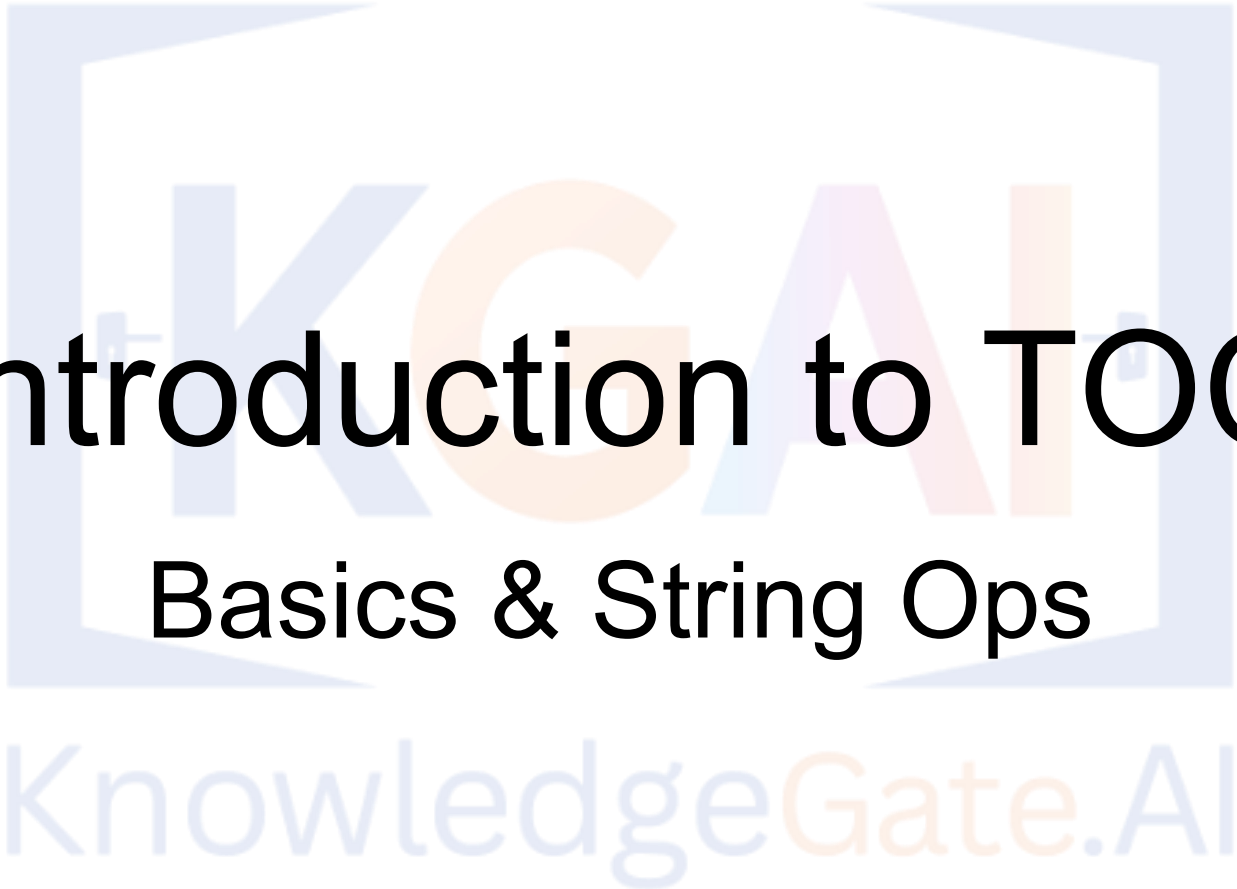
Introduction to TOC

Subtopic

Basics & String Ops

Lesson

Basics & String Ops

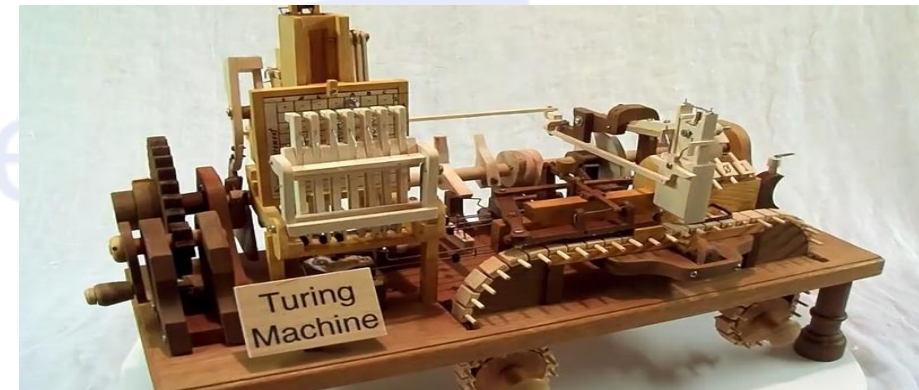
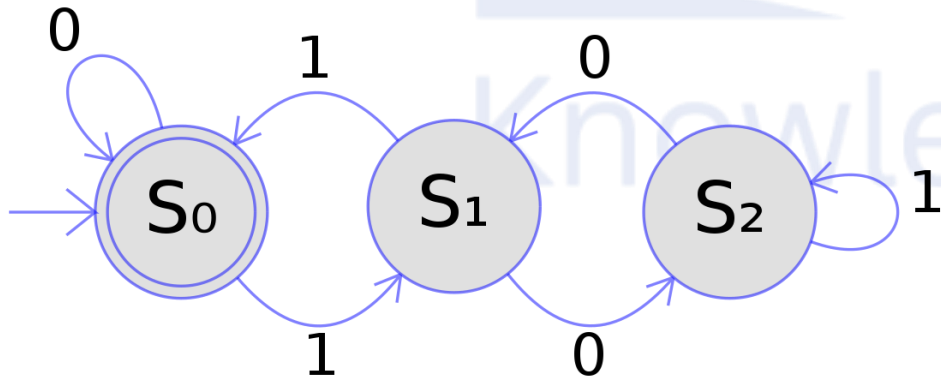
The background features a large, faint watermark of the KnowledgeGate.AI logo. It consists of a blue square frame containing the letters 'KGA' in a stylized font. The 'K' is blue, the 'G' is orange, and the 'A' is pink. Below the frame, the text 'KnowledgeGate.AI' is written in a light blue font, with 'Gate' in orange.

Introduction to TOC

Basics & String Ops

INTRODUCTION TO THEORY OF COMPUTATIONS

- **Theory of Computation (TOC)** is a branch of theoretical computer science that addresses:
- **What can be computed, How efficiently can it be computed, What limitations exist in computation?**
- It involves the study of abstract machines, also known as **automata**, and mathematical models that define what problems can be solved using these machines.
- TOC focuses on different computational models, their capabilities, and the complexity of solving problems. The main areas it deals with include:
 - **Automata Theory**: Studies different types of automata (e.g., finite automata, pushdown automata) and their abilities to solve problems.
 - **Formal Languages**: Explores the classification of languages and grammars that machines can recognize.
 - **Computability Theory**: Investigates which problems can or cannot be solved by any computational model.
 - **Complexity Theory**: Considers the resources (time, space) needed to solve problems and categorizes them based on difficulty (e.g., P, NP).



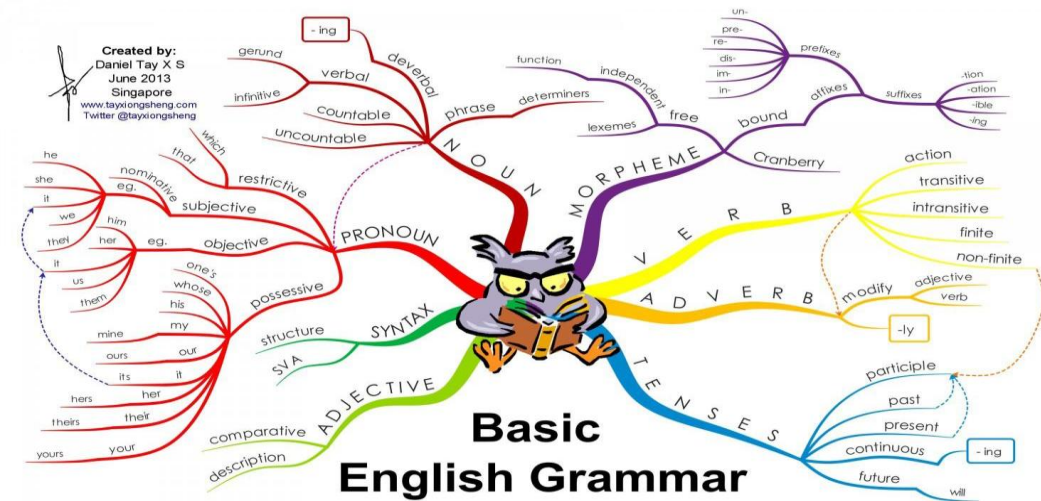
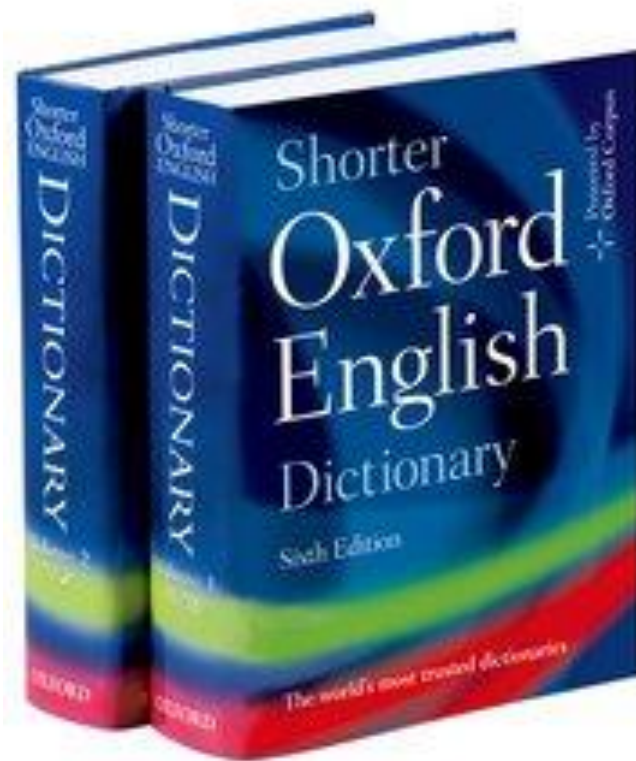
PROBLEM

- **Machine Communication:**
 - Machines play a crucial role in human development, and we need a language to communicate with them. However, this language need not be as complex as natural languages.
- **Formal vs. Informal Languages:**
 - Machines use **formal languages**, which are structured and concise, unlike **informal languages** used in daily human interaction.
- **Language Definition:**
 - A language is "a system suitable for expressing ideas, facts, or concepts" using a set of symbols.



METHODS TO DEFINE LANGUAGE

- In natural language, we can list a finite and predefined set of words in a dictionary, but the sentences formed using these words are infinite. To determine which sentences are valid or invalid, we use **grammar** or rules as a mechanism.



MATHEMATICAL DEFINATION OF LANGUAGE

- **SYMBOL**- Symbols are the basic building blocks, which can be any character/token. (cow, sheep, 😊, white flag, 🚫, ⚙️, 🏠 etc.) (in English we called them as letters).
- **ALPHABET**- An alphabet is a finite non empty set of symbols, (every language has its own alphabet). here in toc, we use symbol Σ for depicting alphabet. e.g. $\Sigma = \{0,1\}$. for English $\Sigma = \{a, b, c, \dots, z\}$ (in English also alphabet is a set of letters, thought in general we called them as alphabet).
- **STRING** - It is a finite sequence of symbols (which are the member of set alphabet). E.g. $\Sigma = \{a, b\}$ String- aabb, aa, b, so on. (in English we called them as words).
- **LANGUAGE** - A language is defined as a set of strings. In the next level we consider programs as a string and programming constructs/tokens like int, floats as letters/symbols.

- Similarly, in our system we have finite number of symbols/letters but using those letters we can generate infinite strings/words.
- So, we may have languages that have infinite number of words, so it is not possible for us to list them, we have to use some framework, which can somehow represent the same language. There are mainly two methods to represent a language
 - by a grammar that generates a language [RG generate RL]
 - by a machine that accepts a language [FA accept RL language]

Machine

Grammar

SOME BASIC OPERATIONS ON STRINGS

- So, before we proceed further let's do a little home work on string, which will help us throughout the subject.
- **Length of a string** - It is defined as number symbol in the string. Denoted like $|W|$, e.g. length of string $|00110| = 5$.
- $|aaba| =$
- $|010| =$

KnowledgeGate.AI

Concatenation of string- Let x and y be two strings, then concatenation is defined as the string formed by making a copy of string x followed by a copy of string y . (NOTE- It's not commutative)

E.g. $w = ab$, $x = ba$

$wx =$

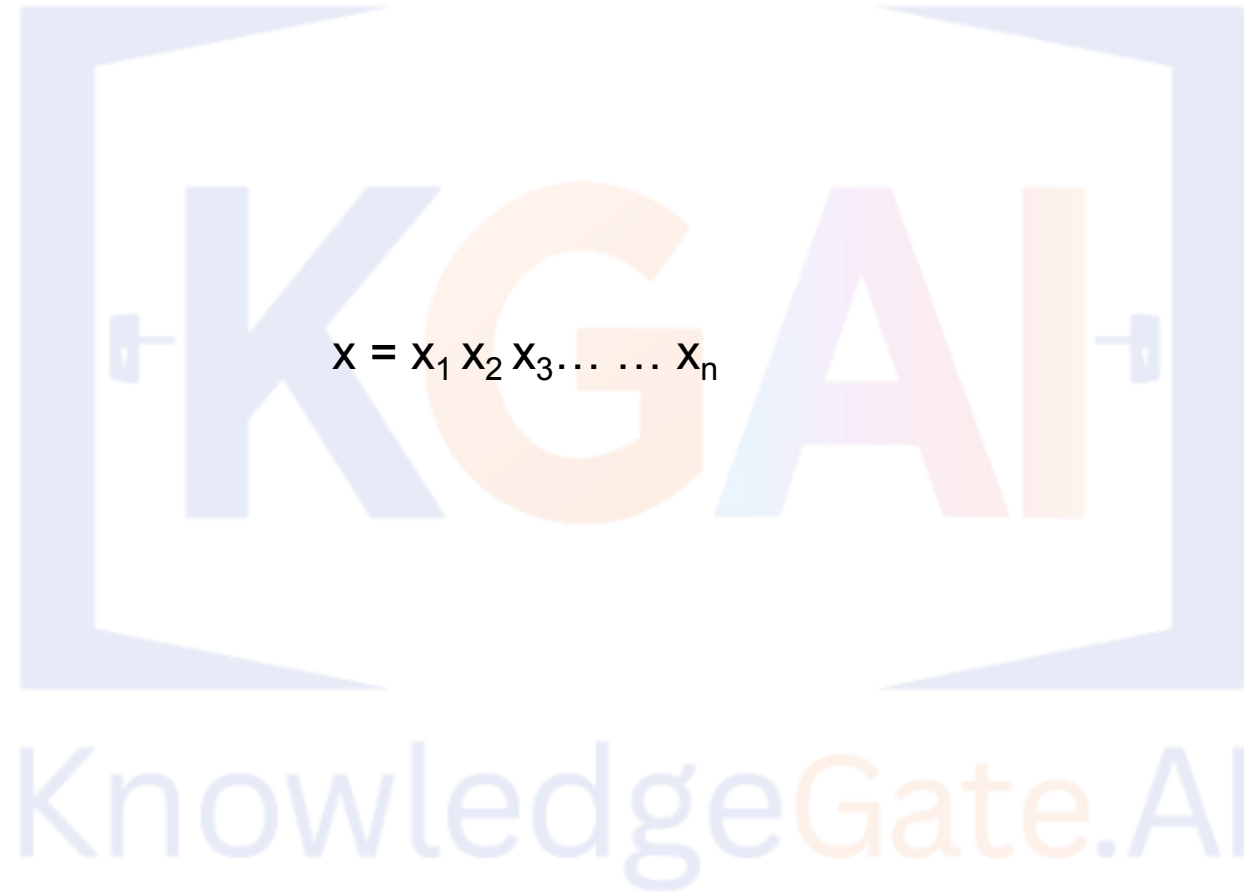
$xw =$

$w = w_1 w_2 w_3 \dots \dots w_m$

$wx =$

$|wx| = |w| + |x|$

$x = x_1 x_2 x_3 \dots \dots x_n$



Reverse of a string – If there is a string w then reverse of a string a denoted by w^r it is just the same string but written in reverse order.

$$W = w_1 w_2 w_3 \dots \dots w_n$$

$$w^r =$$

$$|w| \boxed{} |w^r|$$



Empty/Null String- The string with zero occurrence of symbols. It is denoted by ϵ , $|\epsilon|=0$. If there is a string w , then w^n stands for the strings obtained by repeating w , n times.

$$w^3 = www$$

$$w^2 = ww$$

$$w^1 = w$$

$$w^0 =$$

$$w\epsilon = \epsilon w =$$



Substring- Any string of consecutive symbols in some string 'w' can be collectively said as a substring. E.g. w= abab its substrings can be ab, a, ba...etc.

S u b s t r i n g

- 1. utg
- 2. sbr
- 3. rts
- 4. str
- 5. sub
- 6. ϵ



Q Consider a sting 'GATE' find the total number of substring possible?

	Substring of length 0	Substring of length 1	Substring of length 2	Substring of length 3	Substring of length 4
Number of sting possible	1	4	3	2	1
SUBSTRINGS	€	G, A, T, E	GA, AT, TE	GAT, ATE	GATE

Q Consider a sting 'GGGE' find the total number of substring possible?

	Substring of length 0	Substring of length 1	Substring of length 2	Substring of length 3	Substring of length 4
Number of sting possible					
SUBSTRINGS					

If a string has 'n' distinct symbols then total number of different sub string will be $[n(n+1)/2] + 1$. If w is any string than empty string € and the string w itself is called a trivial substring and the remaining of the other are the non-trivial sub string $[[n(n+1)/2]-1]$

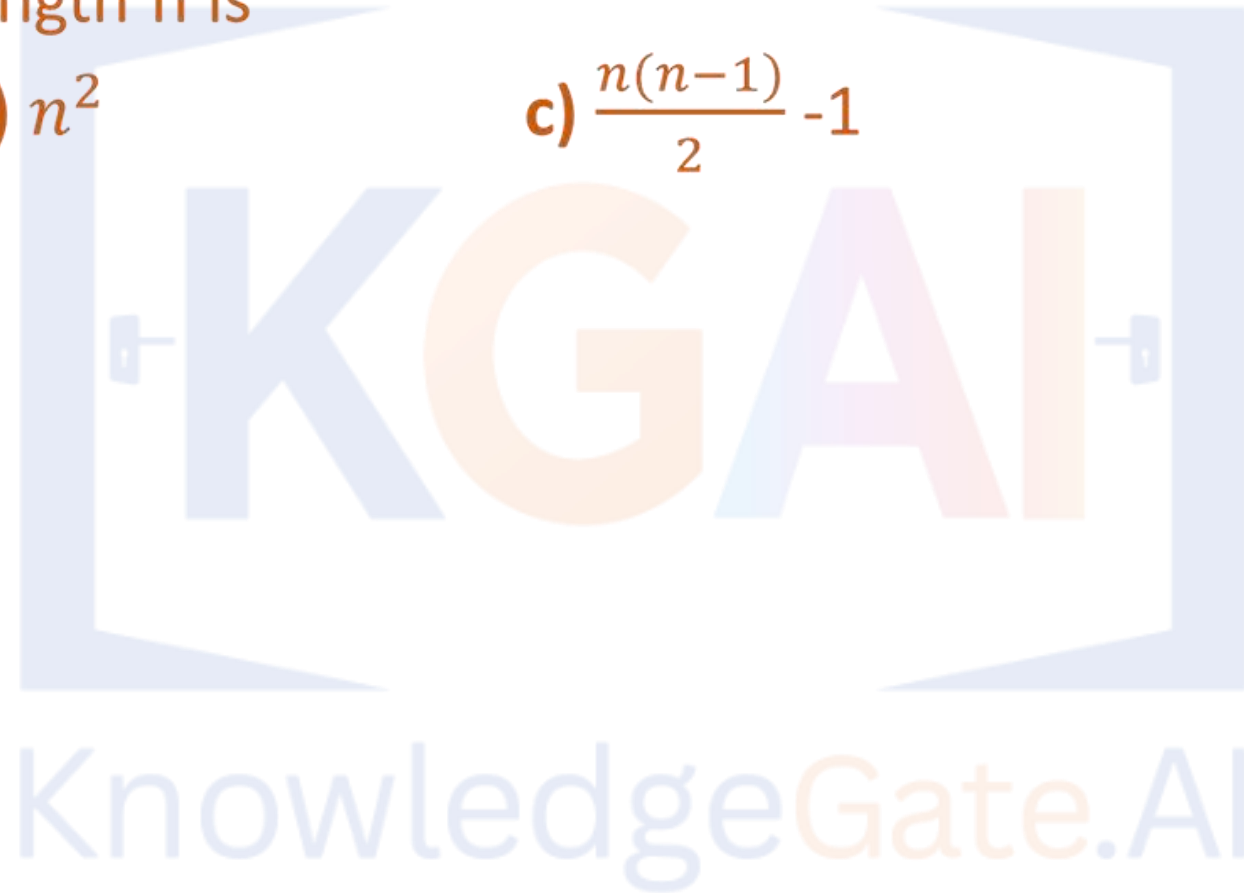
Q The number of substrings (of all lengths inclusive) that can be formed from a character string of length n is

a) $n!$

b) n^2

c) $\frac{n(n-1)}{2} - 1$

d) $\frac{n(n+1)}{2} + 1$

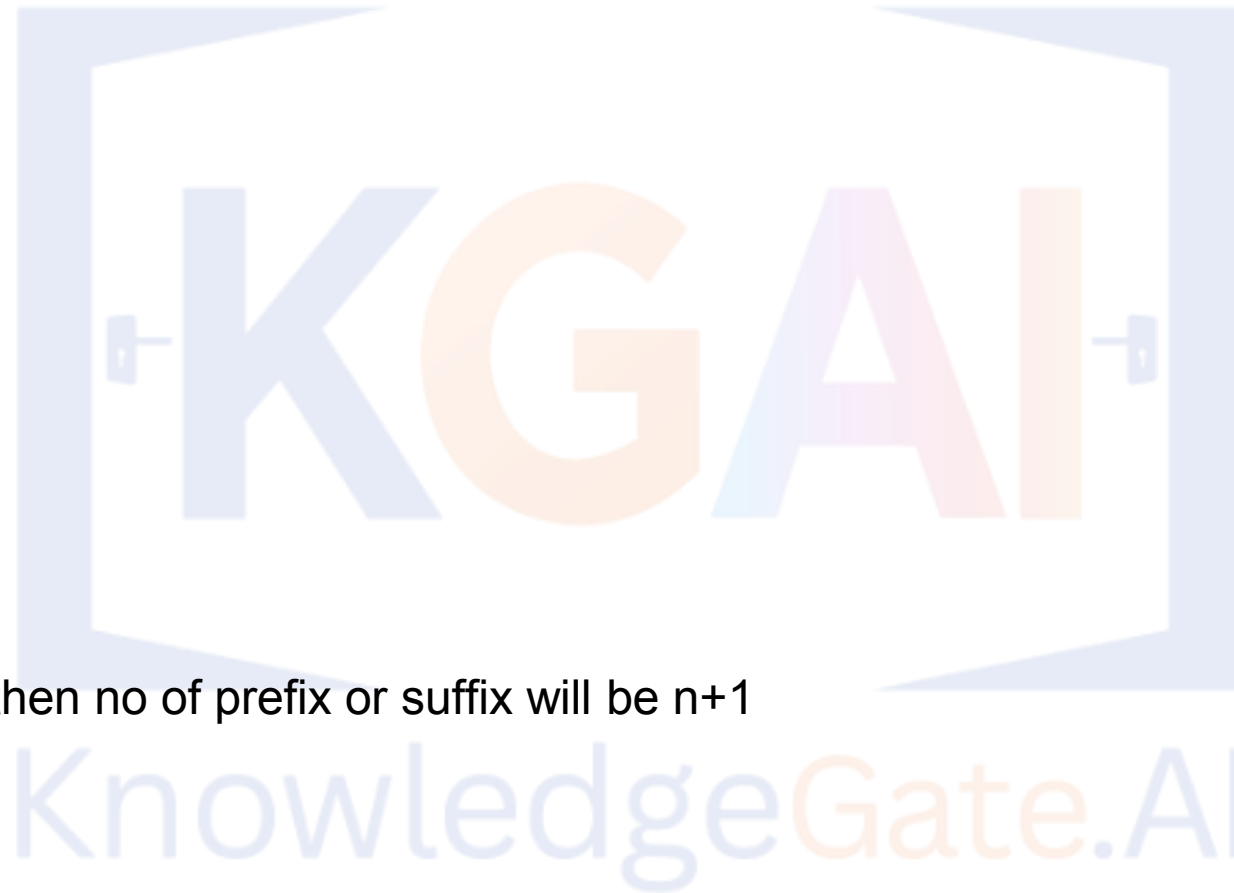


Q Consider a sting 'GATE' find the total number of prefix and suffix possible?

Prefix(GATE) =

Suffix(GATE) =

If there is a string of length n then no of prefix or suffix will be $n+1$



Q In a string of length n , how many proper prefixes can be generated

- a)** 2^n **b)** n **c)** $n(n + 1)/2$ **d)** $n - 1$



Q if $\Sigma = \{a, b\}$ then, find the following?

$$\Sigma^0 =$$

$$\Sigma^1 =$$

$$\Sigma^2 =$$

$$\Sigma^3 =$$

- Σ^K is the set of all the strings from the alphabet Σ of length exactly K.
- $\Sigma^K = \{W \mid |W| = K\}$ (using the symbols from the alphabet Σ)

Kleene closure- If Σ is a set of symbols, then we use Σ^* to denote the set of strings obtained by concatenating zero or more symbols from Σ of any length, in general any string of any length which can have only symbols specified in Σ .

- $\Sigma^* = \bigcup_{i=0}^{\infty} \{w \mid |w| = i\}$ (using the symbols from the alphabet Σ)

Positive closure – If Σ is a set of symbols, then we use Σ^+ to denote the set of strings obtained by concatenating one or more symbols from Σ of any length, in general any string of any length which can have only symbols specified in Σ (except ϵ).

- $\Sigma^+ = \bigcup_{i=1}^{\infty} \{w \mid |w| = i\}$ (using the symbols from the alphabet Σ)

KnowledgeGate.AI

Q Let x_n denote the number of binary strings of length n that contain no consecutive 0 's, which of the following recurrences does x_n satisfy?

a) $x_n = 2x_{n-1}$

b) $x_n = x_{\left\lfloor \frac{n}{2} \right\rfloor} + 1$

c) $x_n = x_{\left\lfloor \frac{n}{2} \right\rfloor} + n$

d) $x_n = x_{n-1} + x_{n-2}$

Q The value of x_5 is?

a) 5

b) 7

c) 13

d) 16

Module

Paper-2 | Theory of Computation

Topic

Introduction to TOC

Subtopic

Language Ops & Sets

Lesson

Language Ops & Sets

Introduction to TOC

Language Ops & Sets

KnowledgeGate.AI

LANGUAGES

- Since languages are sets, the union, intersection and difference of two languages are immediately defined.
- The complement of a language is defined with respect of Σ^*
 - $L^c =$
- The reverse of a language is the set of all the strings after reversal
 - $L^r =$
- The concatenation of two languages L_1 and L_2 is the set of all the strings obtained by concatenating any elements of L_1 and L_2 .
 - $L_1 L_2 =$
- Let L^n is defined as L is concatenated with itself n times.
 - $L^n =$
 - $L^* =$

Q Suppose $L_1 = \{10, 1\}$ and $L_2 = \{011, 11\}$. How many elements are there in $L = L_1 L_2$

a) 4

b) 3

c) 2

d) None of these



Module

Paper-2 | Theory of Computation

Topic

Deterministic FA (DFA)

Subtopic

DFA Basics & Definitions

Lesson

DFA Basics & Definitions

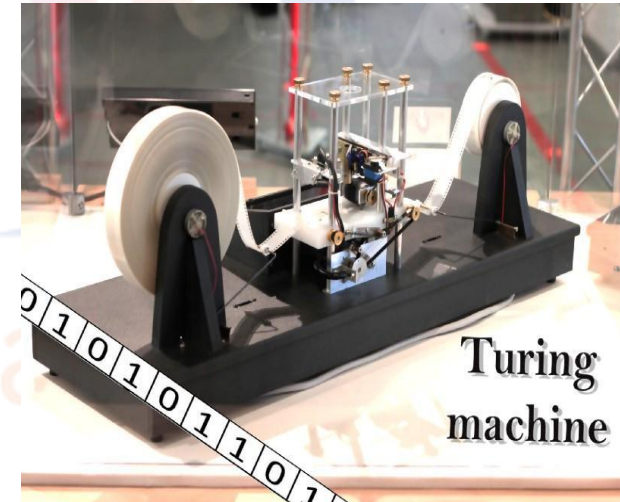
Deterministic FA (DFA)

DFA Basics & Definitions

KnowledgeGate.AI

Automation

- An automaton is a self-operating system that transforms, transmits, and utilizes energy, materials, and information to perform functions without human involvement. The term often refers to a machine that follows predefined rules to execute complex tasks autonomously. Examples include automatic machine tools, packing machines, and photo printing devices.



FINITE AUTOMATA

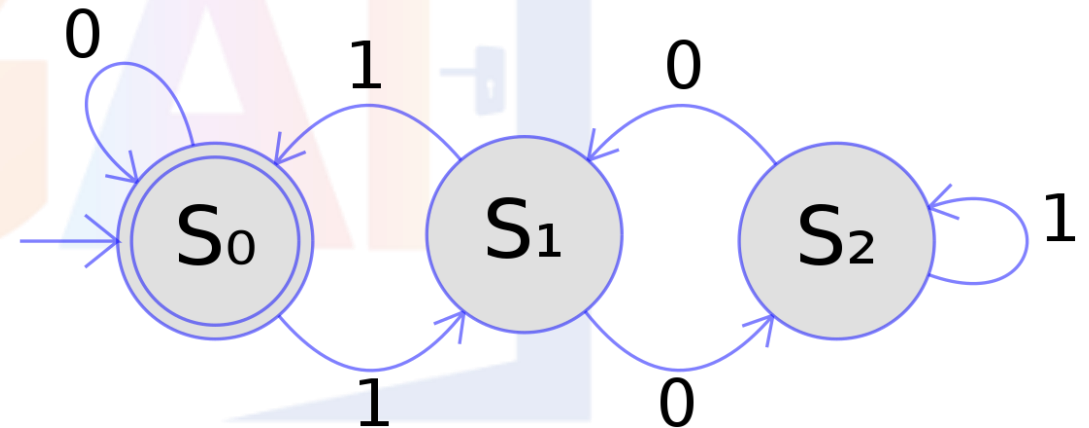
- A finite automaton is a computational model with a limited set of states, where control shifts between states based on external inputs. It is commonly used as an abstract machine in computer science for recognizing patterns in symbol strings. Finite automata have applications in pattern matching, lexical analysis, parsing, and more. Finite automata can be broadly classified into two types-

1. Finite automata without output

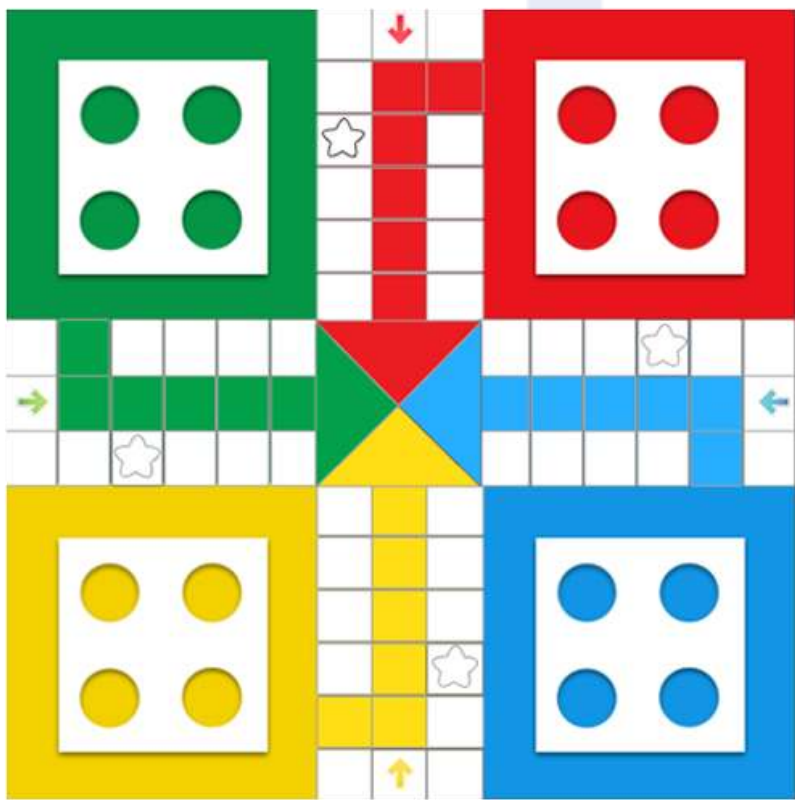
1. Deterministic finite automata.
2. Non deterministic finite automata.
3. Non deterministic finite automata with ϵ

2. Finite automata with output

1. Moore machine
2. Mealy machine



- A finite automaton is limited in its ability to store information during computation because it lacks temporary storage. The machine can only store a finite amount of information by switching between a limited number of states. As a result, it can only handle scenarios where the amount of information to be stored is strictly bounded.



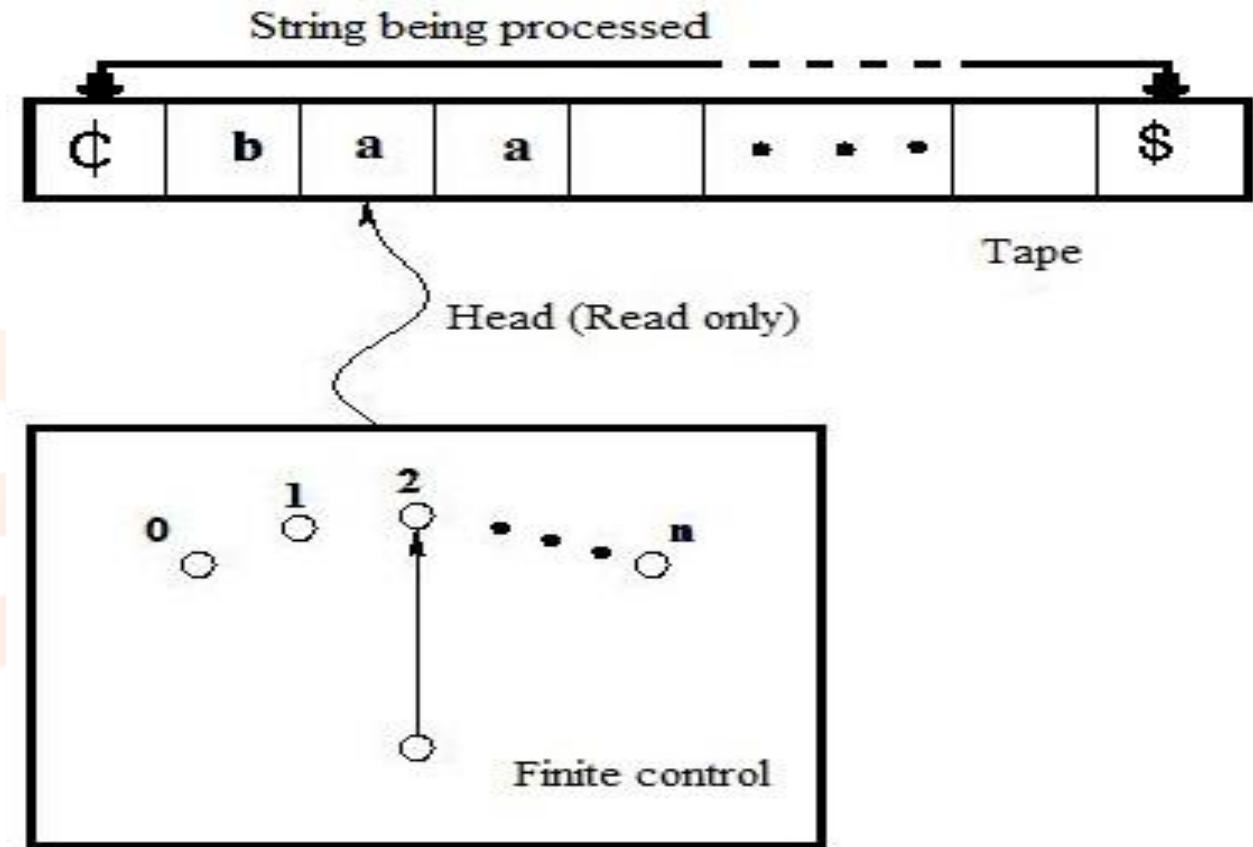
DETERMINISTIC FINITE AUTOMATA

A **deterministic finite automaton (DFA)** is defined by 5-tuple $(Q, \Sigma, \delta, S, F)$ where:

- Q is a finite and non-empty set of states
- Σ is a finite non-empty set of finite input alphabet
- δ is a transition function, ($\delta: Q \times \Sigma \rightarrow Q$)
- S is **initial state** (always one) ($S \in Q$)
- F is a set of final states ($F \subseteq Q$) ($0 \leq |F| \leq N$, where n is the number of states)

A deterministic finite automaton (DFA) operates through various components:

- **Input Tape:** Divided into squares, each containing a symbol from the input alphabet. It has end markers (ϵ at the left and $\$$ at the right) or can be infinite. The input string lies between these markers.
- **Reading Head:** This head reads one square at a time and moves to the right.
- **Finite Control:** Acts as the inference engine and manages transitions between states.
- A DFA ensures a unique computation for each input string, making it deterministic. DFAs are commonly used in tasks like lexical analysis (spell checking) in compiler design.



Representation

TRANSITION STATE DIAGRAM- A graphical representation where each circle represents a state, and directed edges show transitions between states. The initial state has an arrow pointing to it, and the final state is represented by two concentric circles.

TRANSITION TABLE- A two-dimensional table where rows represent states, and columns represent input symbols. The table shows how the machine transitions between states based on inputs.

TRANSITION ID- $\delta \{q_i, a\} = q_j$, This notation indicates that when the machine is in state q_i and reads input symbol a , it transitions to state q_j .

δ		Σ	
		a	b
Q	q_0		
	q_1		

ACCEPTANCE BY DFA

- Let 'w' be any string designed from the alphabet Σ , corresponding to w, if there exist a transition for which it starts at the initial state and ends in any One of the final states, then the string 'w' is said to be accepted by the finite automata. $\delta^*(q_0, w) = q_f$ for some $q_f \in F$.
- Mathematically, it can be represented as: - $L(M) = \{w \in \Sigma^* \mid \delta^*(S, w) \in F\}$

KnowledgeGate.AI

Q. Consider the following table of an FA
If the final state is q4, to which of the following string is accepted?

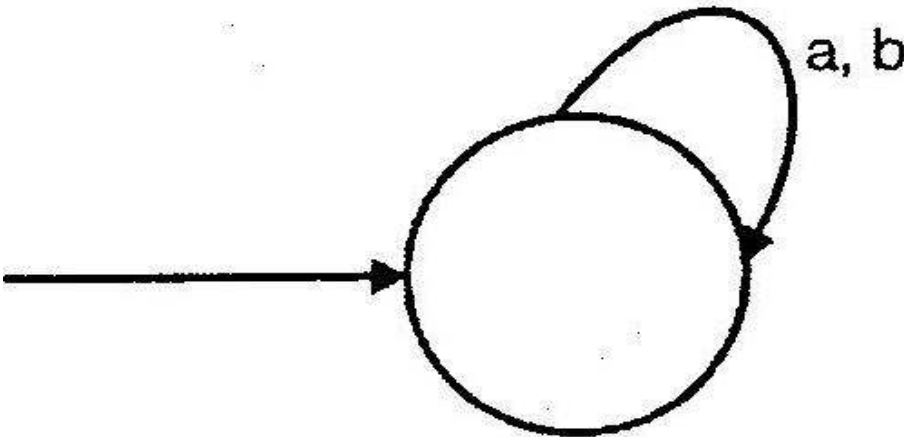
- 1. aaaaa
 - 2. aabbaabbbb
 - 3. bbabababb
- a) 1 & 2
b) 2 & 3
c) 3 & 1
d) all of the above

δ	a	b
start	q ₁	q ₀
q ₀	q ₁	q ₀
q ₁	q ₂	q ₁
q ₂	q ₃	q ₂
q ₃	q ₄	q ₃
q ₄	q ₄	q ₄

Q The FSM over an alphabet {a, b} shown in the figure accepts

- a)** all strings
- c)** ϵ - alone

- b)** no strings
- d)** None of these



Module

Paper-2 | Theory of Computation

Topic

Deterministic FA (DFA)

Subtopic

DFA Construction & Design

Lesson

DFA Construction & Design

Deterministic FA (DFA)

DFA Construction & Design

KnowledgeGate.AI

Q Design a minimal DFA that accepts a language 'L', where $L=\{a\}$ over the alphabet $\Sigma=\{a\}$.



Q design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, where every accepted string 'w' starts with substring s

i) s = b

ii) s = ab

iii) s = abb



Conclusion

- if $w = sx$, $|s| = m$, then no of states in the DFA is $m+2$
- Dead state is required
- Loop on final state



Q design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$. where every accepted string 'w' ends with substring 's'.

i) $s = ab$

ii) $s = aa$

iii) $s = bab$



Conclusion

- if $w = xs$, $|s| = m$, then no of states in the DFA is $m+1$
- No need of dead state
- Can not stay of final state for ever



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$. where every accepted string 'w' contains sub string s.

i) abb

ii) aba



Conclusion

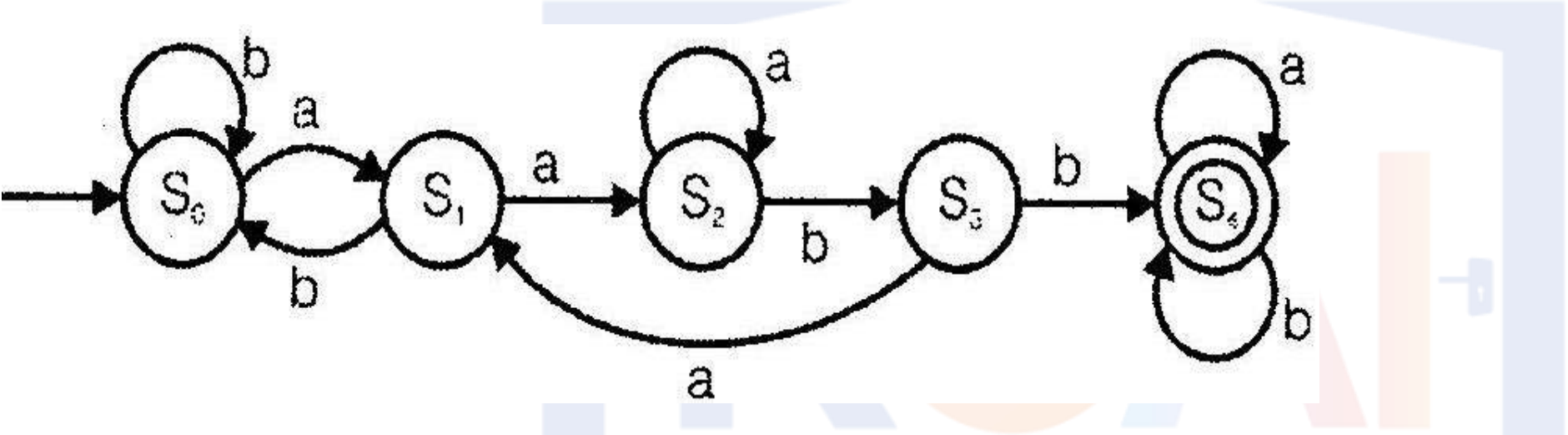
- if $w = xsx$, $|s| = m$, then no of states in the DFA is $m+1$
- No need of dead state
- Can loop on final state



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with a.



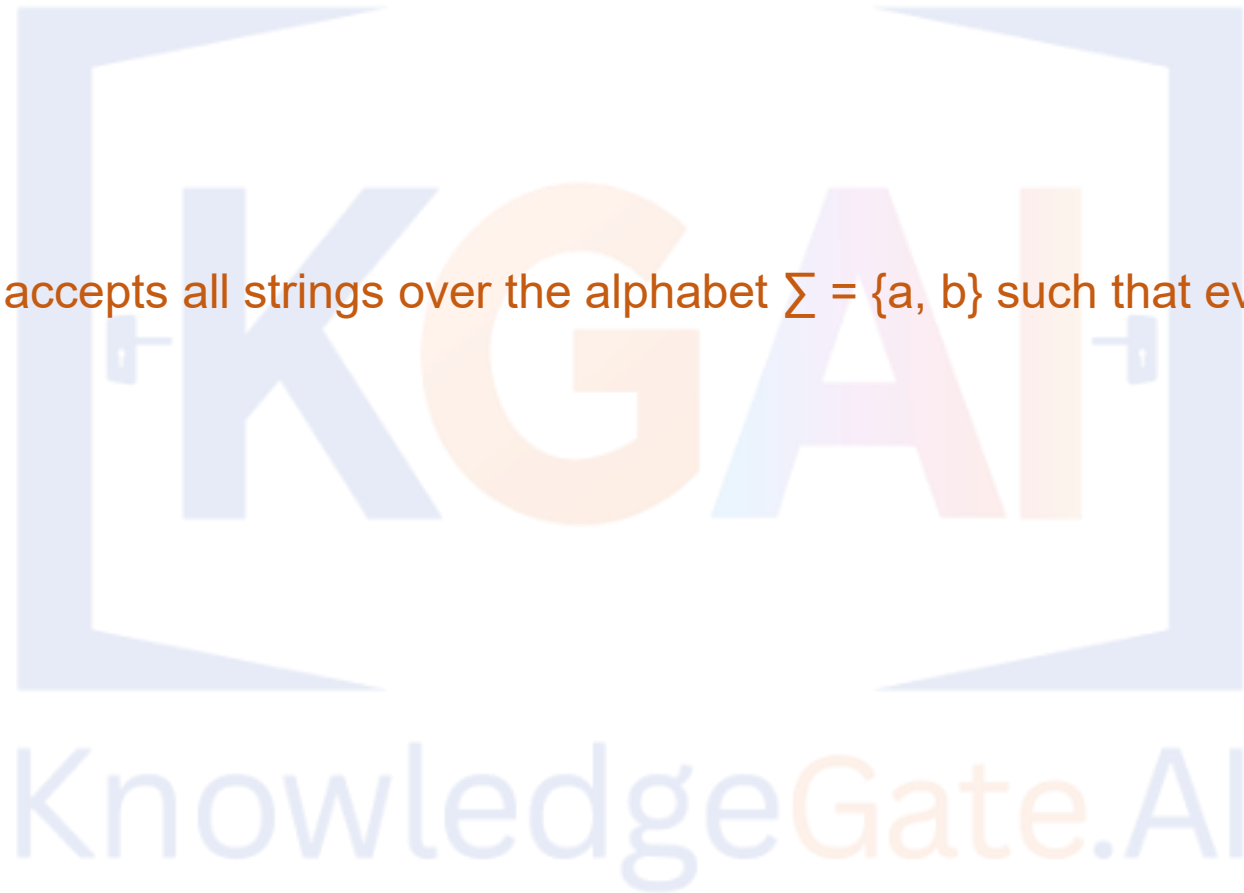
Q Consider the following machine M
What is the language $L(M)$ accepted by this machine?



- a) $L(M) = \{\text{Set of all words starting with aabb}\}$
- b) $L(M) = \{\text{Set of all words having aabb as a sub word}\}$
- c) $L(M) = \{\text{Set of all words ending with aabb}\}$
- d) $L(M) = \{\text{Set of all words with exactly one occurrence of aabb}\}$

Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with same symbol.

Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with different symbol.



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=SX$.

i) $s= aa/bb$

ii) $s=aaa/bbb$



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=XS$.

i) $s = aa/bb$

ii) $s = aaa/bbb$



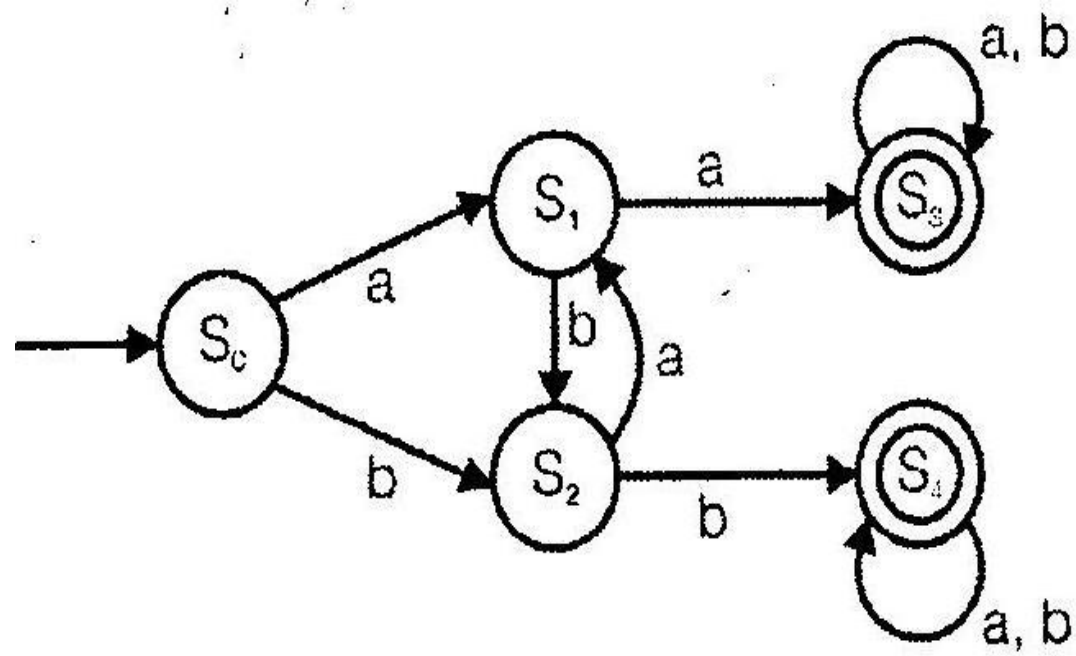
Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=XSX$.

i) $s = aa/bb$

ii) $s = aaa/bbb$



Q Consider the machine M shown below L(M)?



- a) $L(M) = \{\text{words starting with aa or bb}\}$
- b) $L(M) = \{\text{words ending with aa or bb}\}$
- c) $L(M) = \{\text{words containing aa or bb as a sub word}\}$
- d) None of these

Q1. Which should be the final state(s) of the DFA if it should accept strings starting with 'a' and ending with 'b'?

a) q_0 b) q_1 c) q_0, q_1 d) q_3

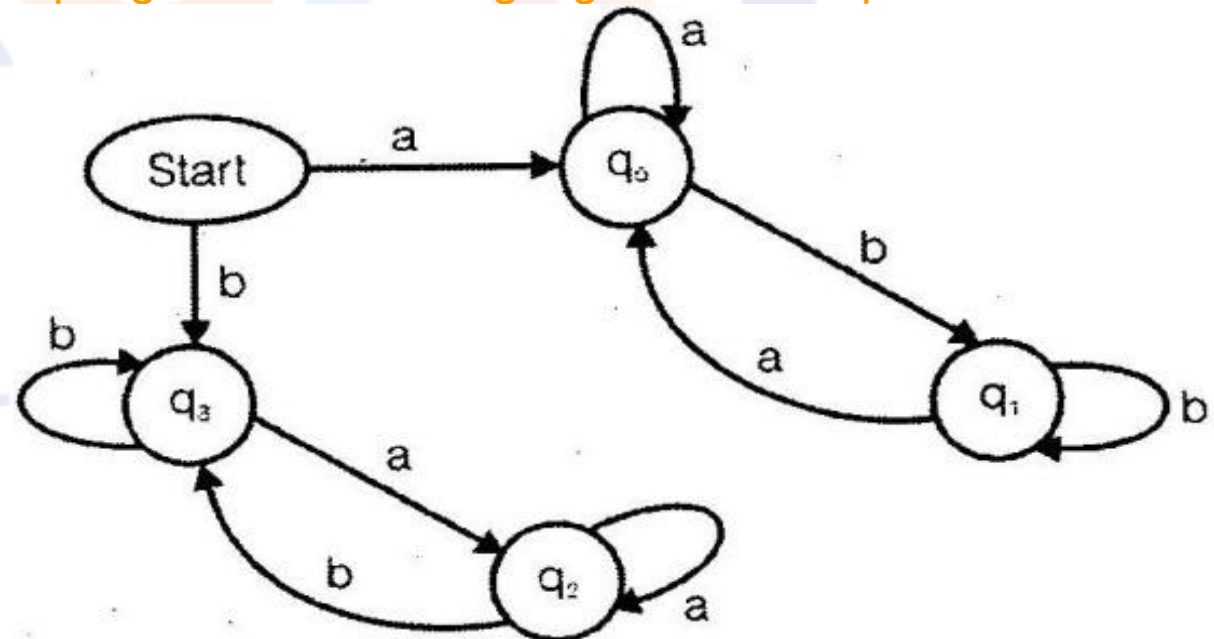
Q2. Which of the following strings will be accepted if q_0 and q_1 are accepting states?

1. abbab 2. Babaaa 3. aaaba

a) 1,2 b) 2,3 c) 1,3 d) None of these

Q3. Which of the following represents the set of accepting states if the language to be accepted contains strings having the same starting and ending symbols?

a) $\{q_0\}$ b) $\{q_0, q_3\}$ c) $\{q_3\}$ d) $\{q_3, q_1\}$



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' accepted must be like

i) $|w| = 3$

ii) $|w| \leq 3$

iii) $|w| \geq 3$



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain exactly two a 's, $|w|_a = 2$.

Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain at least two a 's,
 $|w|_a \geq 2$

Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain at most two a 's,
 $|w|_a \leq 2$

Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' accepted must be like

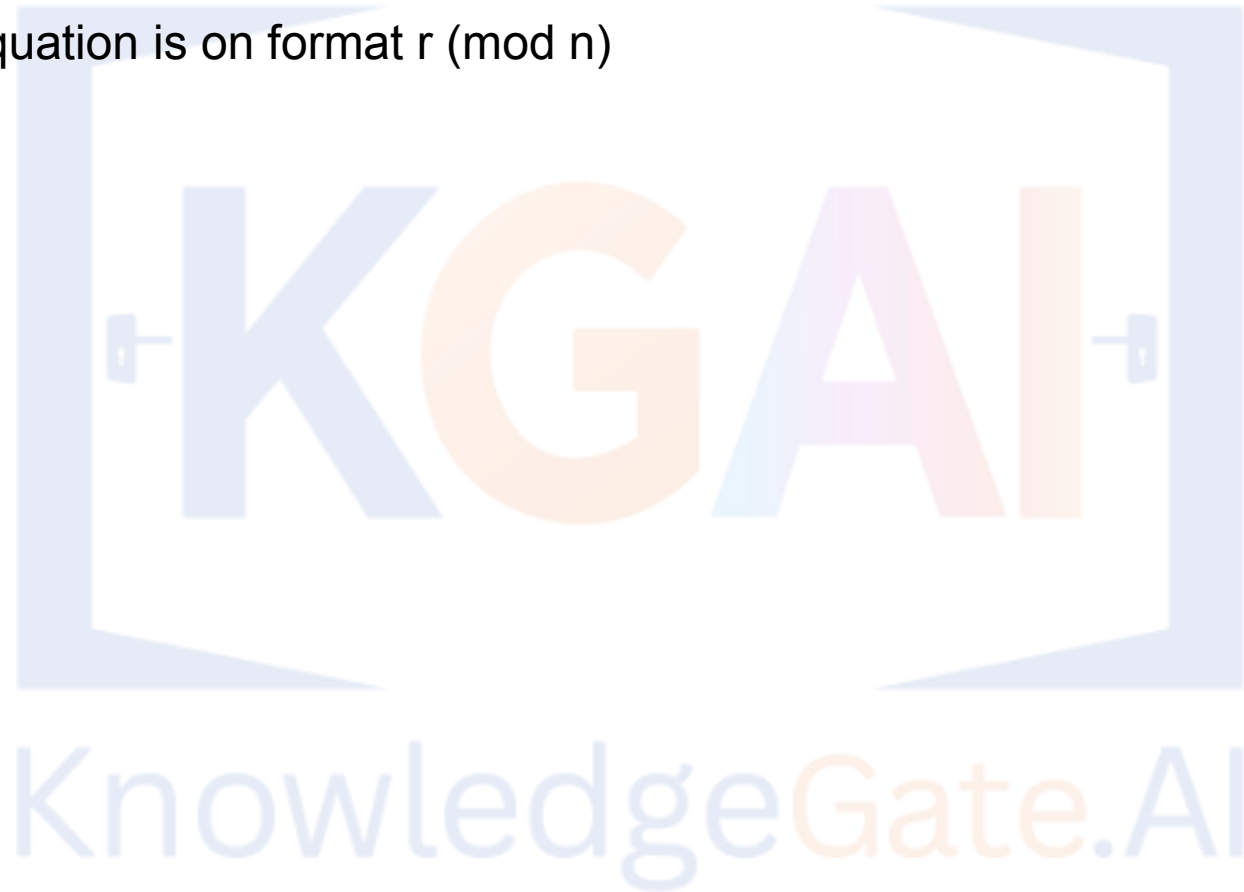
i) $|w| = 0(\text{mod } 3)$

ii) $|w| = 1(\text{mod } 4)$



Conclusion

No of states will be n , if the equation is on format $r \pmod n$



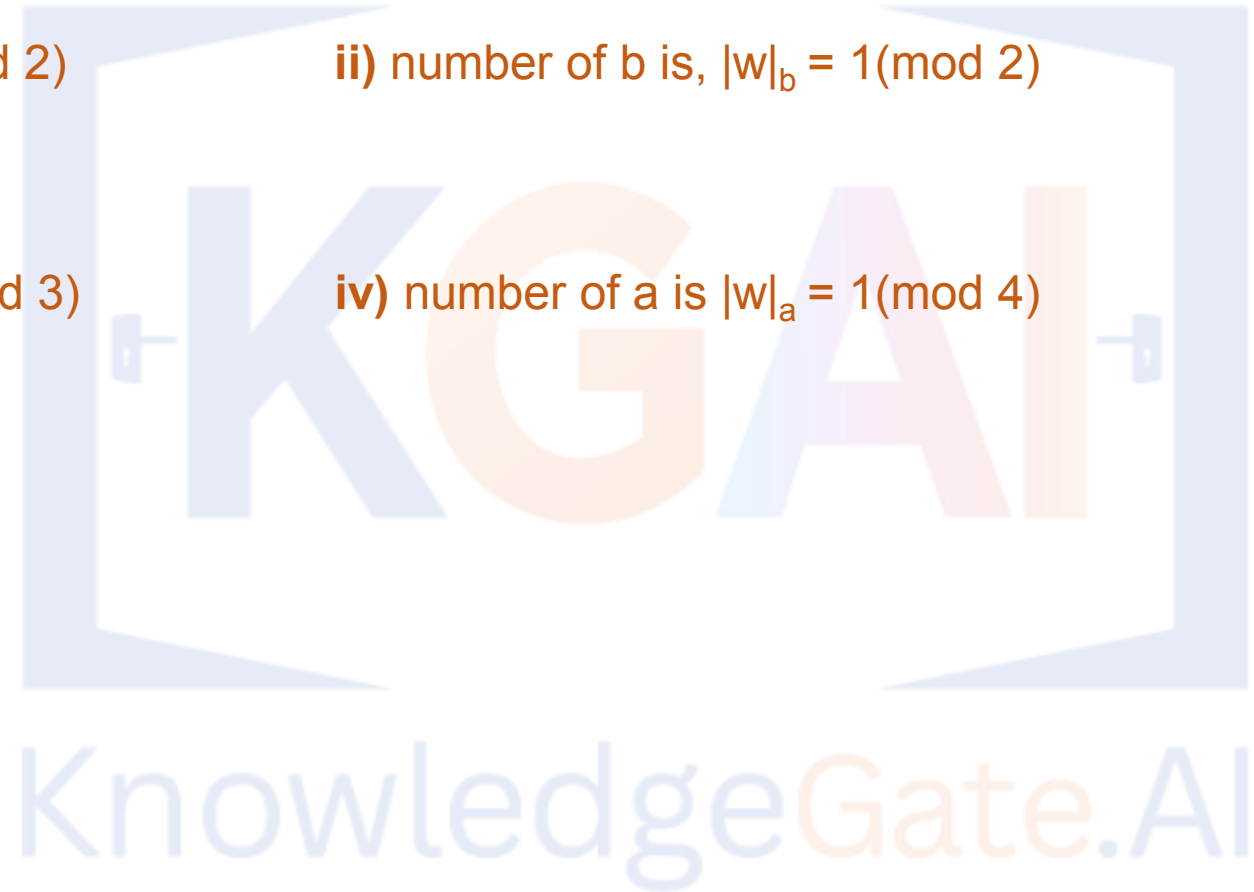
Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain

i) number of a is, $|w|_a = 0(\text{mod } 2)$

ii) number of b is, $|w|_b = 1(\text{mod } 2)$

iii) number of a is $|w|_a = 2(\text{mod } 3)$

iv) number of a is $|w|_a = 1(\text{mod } 4)$



Q Let $\Sigma = \{0, 1\}$, then an automaton A accepting only those words from Σ having an odd number of 1's requires _____ states including the start state

a) 2

b) 3

c) 4

d) 5



Q The smallest finite automaton which accepts the language $\{x \mid \text{length of } x \text{ is divisible by } 500\}$ has

a) 200 states

b) 300 states

c) 400 states

d) 500 states



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain odd occurrence of the substring 'ab'.

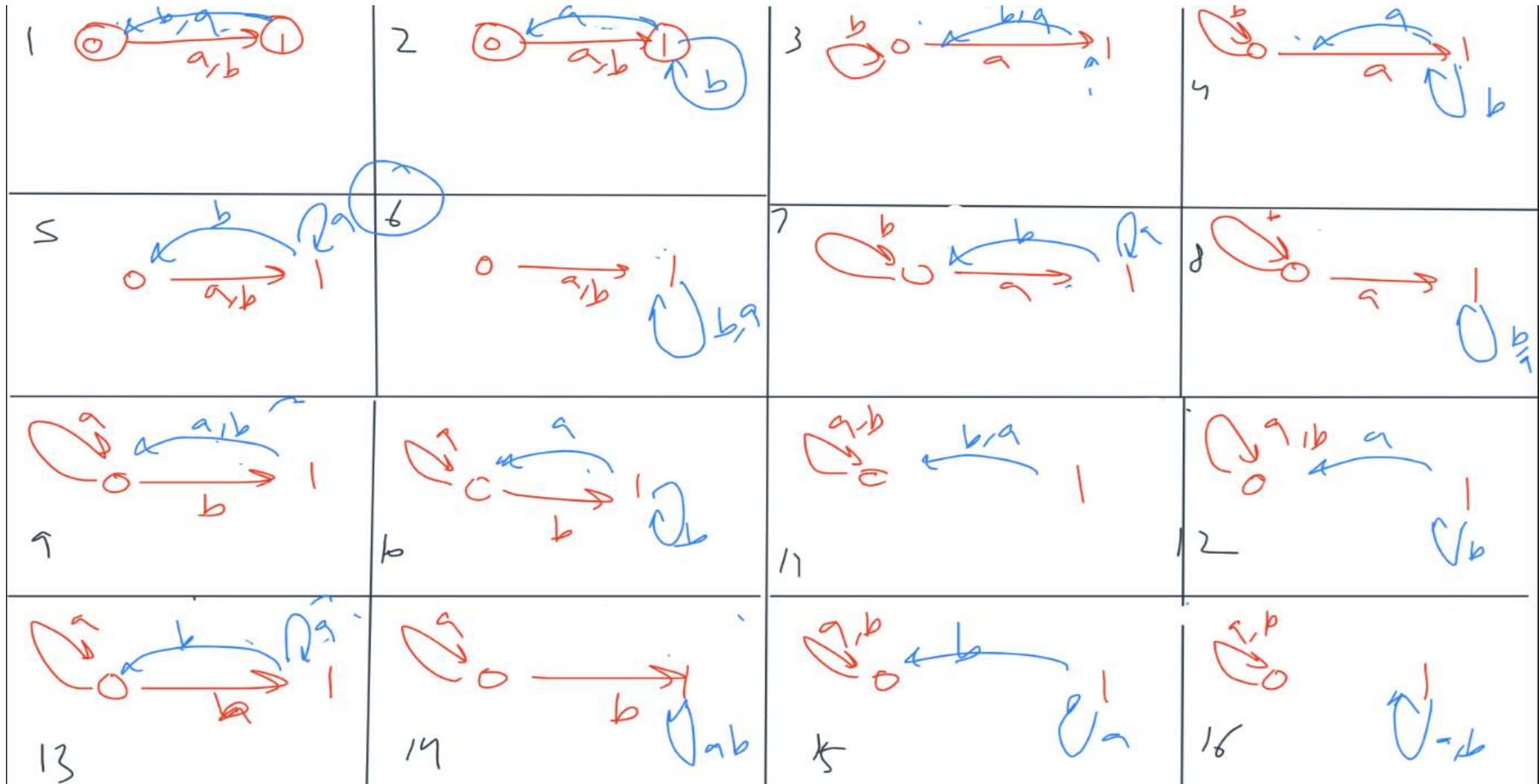


Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain even occurrence of the substring 'baa'.



Q how many DFA can be designed, with a fixed initial state, over an alphabet $\Sigma = \{a, b\}$, and the number of states be two.





- If no of states is $|Q| = n$, and no of input alphabet $|\Sigma| = m$
- Then no of states will be n^{m*2^n}

Q how many DFA can be constructed over the alphabet, such that no of states $|Q| = 2$ and size of input alphabet is $|\Sigma| = 2$, that accepts empty language(Φ)?



Q how many DFA can be constructed over the alphabet, such that no of states $|Q| = 2$ and size of input alphabet is $|\Sigma| = 2$, that accepts universal language (Σ^*)?



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that for every accepted string 2nd from left end is always b.



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that for every accepted string 2nd from right end is always b.



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' which is accepted starts with 'a' & length is divisible by 3, i.e. $0 \pmod{3}$?



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' which is accepted starts with 'ab' & length is divisible by 2(mod 4)?



- No of states will be n, $n = \text{first condition} + \text{second condition} - 1$

Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain be like,
no of a = $0 \pmod{2}$ || no of b = $0 \pmod{2}$?



cross product method(one more approach , where concentrate on even and odd logic)

Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain be like, no of a = $0 \pmod{2}$ && no of b = $0 \pmod{2}$?



Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b, c\}$, such that every string accepted must contain be like, $a = 0(\text{mod } 2)$ && no of $b = 0(\text{mod } 3)$ && no of $c = 0(\text{mod } 5)$?



- if, $n(\text{mod } m)$ && $p(\text{mod } r)$, then the no of states will be $m*r$, if m & r are relatively prime to each other and working on different symbol

www.knowledgegate.ai

Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{0,1\}$, such that every string 'w' which is accepted has a decimal equivalent

- i)** $0 \pmod{3}$
- ii)** $2 \pmod{4}$
- iii)** $0 \pmod{5}$
- iv)** $2 \pmod{6}$
- v)** $3 \pmod{8}$



on a format of $m \pmod{n}$

- 1)** if n is odd, number of states will be n
- 2)** if n is even and $n=2^m$, then number of states will be $m+1$
- 3)** if n is even and $n \neq 2^m$, then for number of states no direct formula

Q construct the DFA for the following languages:

i) $L = \{a^m b^n \mid m, n \geq 0\}$

ii) $L = \{a^m b^n c^p \mid m, n, p \geq 0\}$



- If type is $L = \{a^m b^n \mid m \geq 0, n \geq 0\}$, then no of states is no of alphabet +1

www.knowledgegate.ai

Q construct the DFA for the following languages:

i) $L = \{a^m b^n \mid m \geq 0, n \geq 1\}$

ii) $L = \{a^m b^n \mid m \geq 0, n \geq 2\}$



- If type is $L = \{a^m b^n \mid m \geq 0, n \geq j\}$, then no of states is $j+2$

Q construct the DFA for the following languages:

i) $L = \{a^m b^n \mid m \geq 1, n \geq 0\}$

ii) $L = \{a^m b^n \mid m \geq 2, n \geq 0\}$



- If type is $L = \{a^m b^n \mid m \geq i, n \geq 0\}$, then no of states is $i+3$

www.knowledgegate.ai

Q construct the DFA for the following languages:

$L = \{a^m b^n \mid m \geq 1, n \geq 2\}$



- If type is $L = \{a^m b^n \mid m \geq i, n \geq j\}$, then no of states is $i+j+2$

www.knowledgegate.ai

Module

Paper-2 | Theory of Computation

Topic

Deterministic FA (DFA)

Subtopic

Complement & Minimization

Lesson

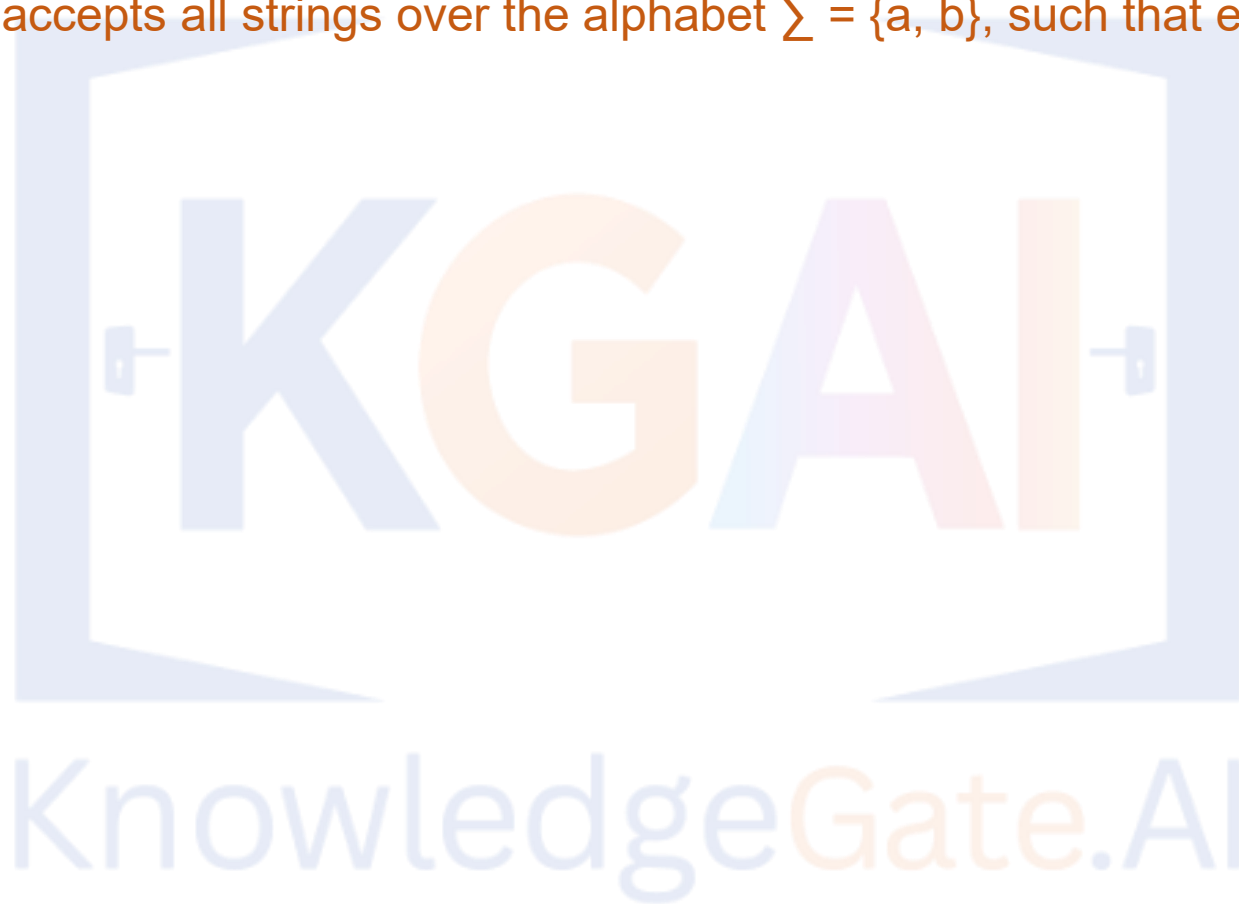
Complement & Minimization

Deterministic FA (DFA)

Complement & Minimization

COMPLEMENT OF DFA

Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must not contain a substring aaa?



Note

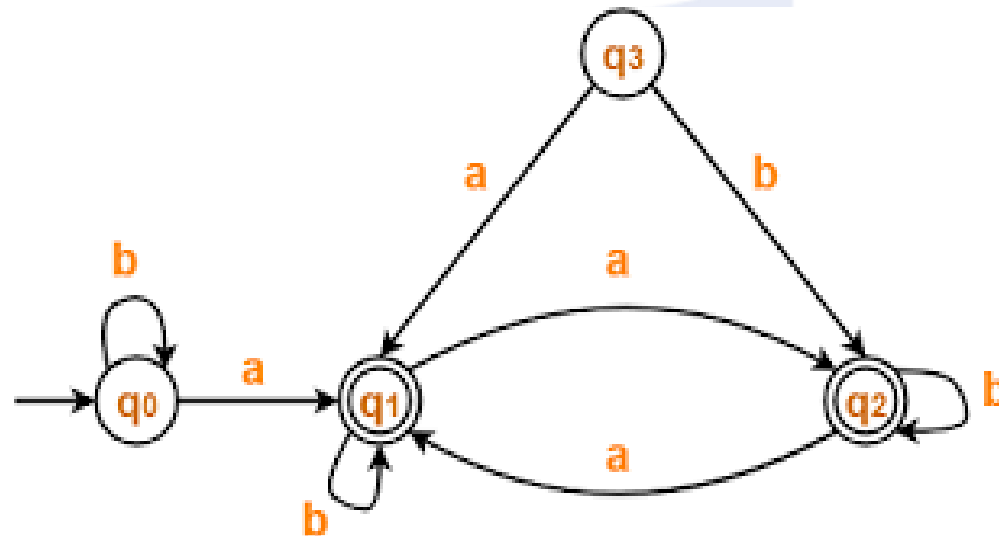
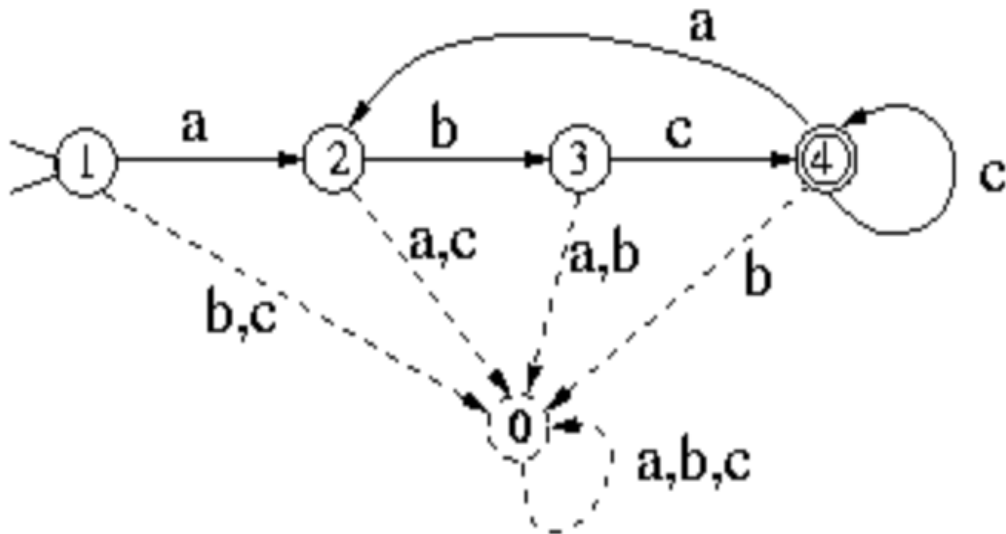
- The finite automata which is obtained by interchanging final & non-final states is known as complement of the finite automata.
- $L(FA) \cup L(FA^c) = \Sigma^*$ $L(FA) \cap L(FA^c) = \Phi$
- $L(FA)$ subset of Σ^* and $L(FA)^c$ subset of Σ^*
- No of states in FA = no of states in complement of FA
- No of final states in a FA and it's Complement FA may have different final states
- Complement always exist for DFA (as it is a complete system)

KnowledgeGate.AI

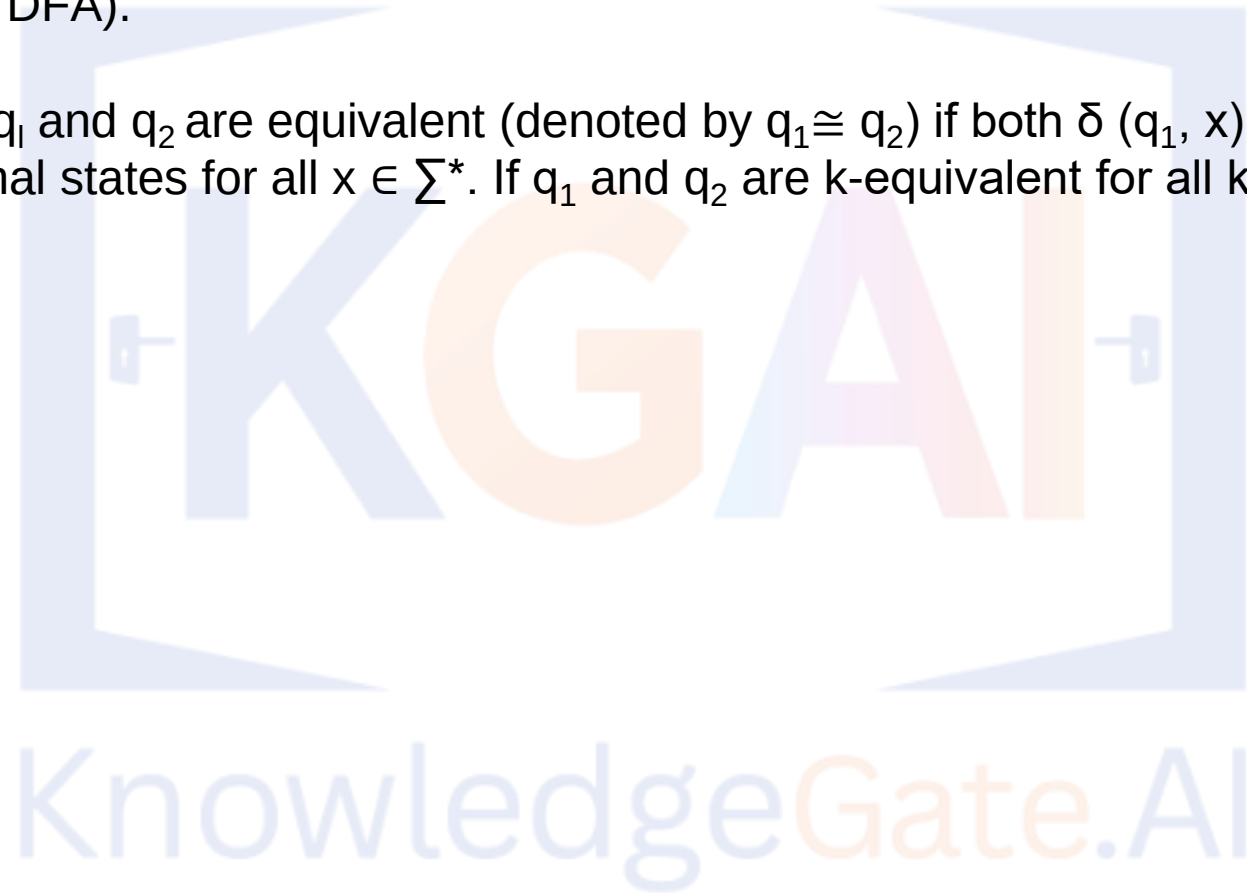
MINIMIZATION OF FINITE AUTOMATA

- The process of removing states that do not affect the language-accepting capability of a deterministic finite automaton (DFA). The result is a minimal deterministic finite automaton (MFA), which is unique for a given language.
- Designing a minimal DFA directly can be difficult, so the recommended approach is to first design a DFA and then minimize it.
- States in DFA can be classified as:
 - **Productive States:** These states contribute to the machine's ability to accept a language. Their presence affects the language-accepting power.
 - **Non-Productive States:** These states do not add any value to the language-accepting capability of the machine and can be removed during minimization.
- **NOTE- MFA is always unique for a language.**

- **NON- PRODUCTIVE STATES**- These states don't add anything to the language accepting power to the machine. They can further be divided into three types, Dead State, Unreachable State, Equal State
- **Dead State**- It is basically created to make the system complete, can be defined as a state from which there is no transition possible to the final state. In a DFA there can be more than one dead state but logically always one dead state is sufficient to complete the functionality.
- **Unreachable Sate**- It is that state which cannot be reached starting from initial state by parsing any input string.

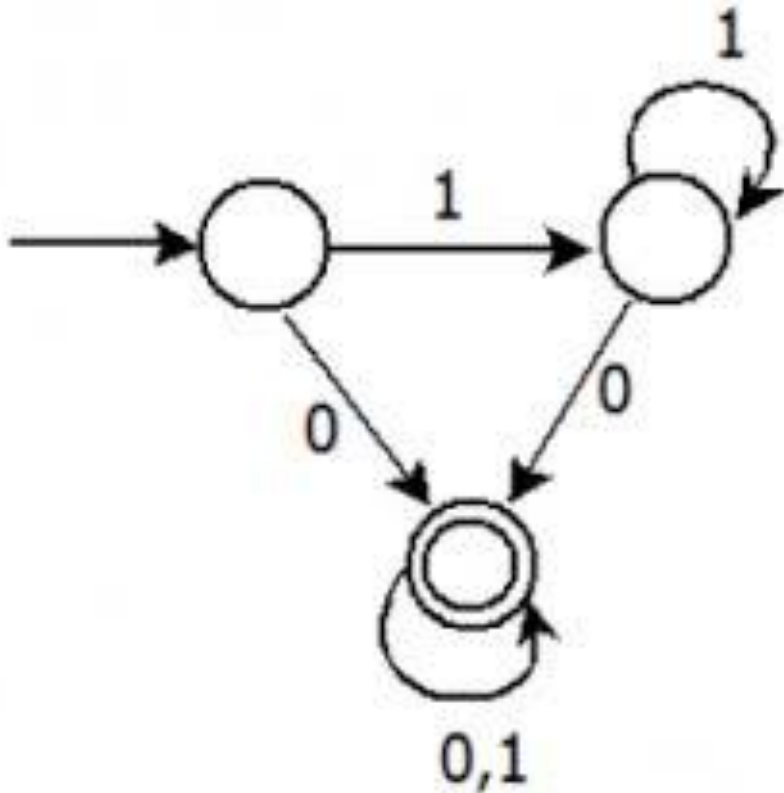


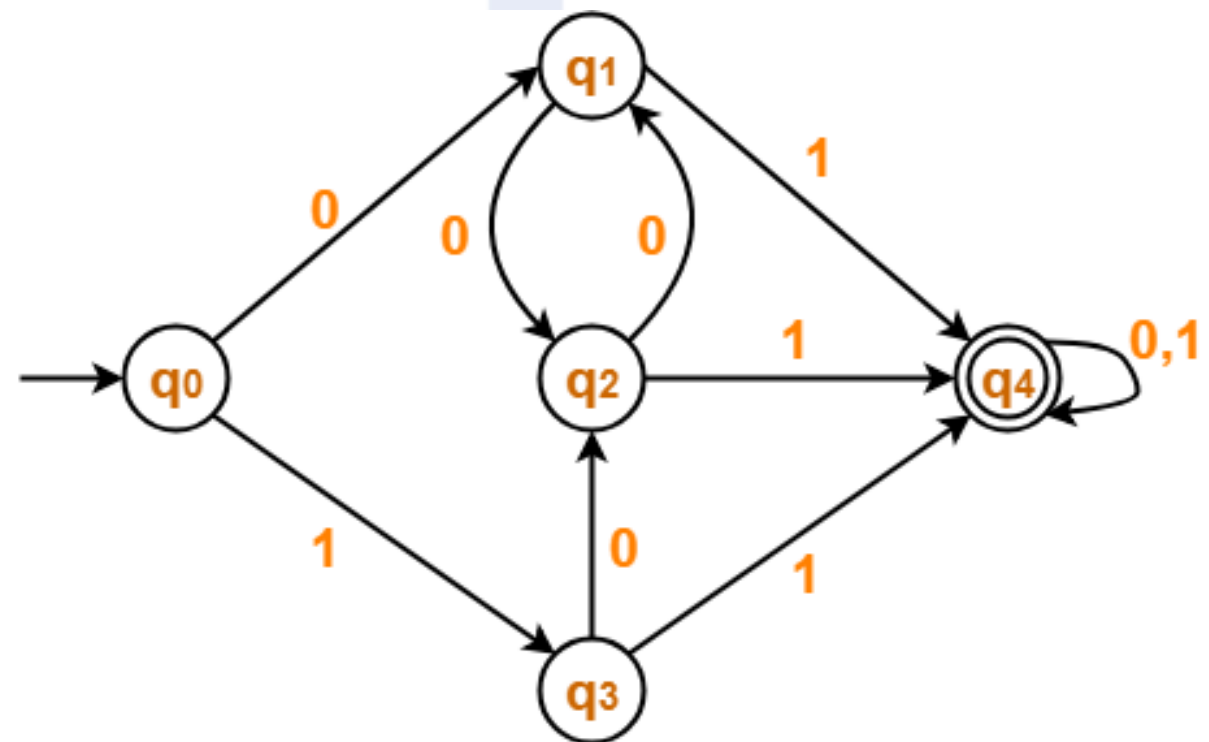
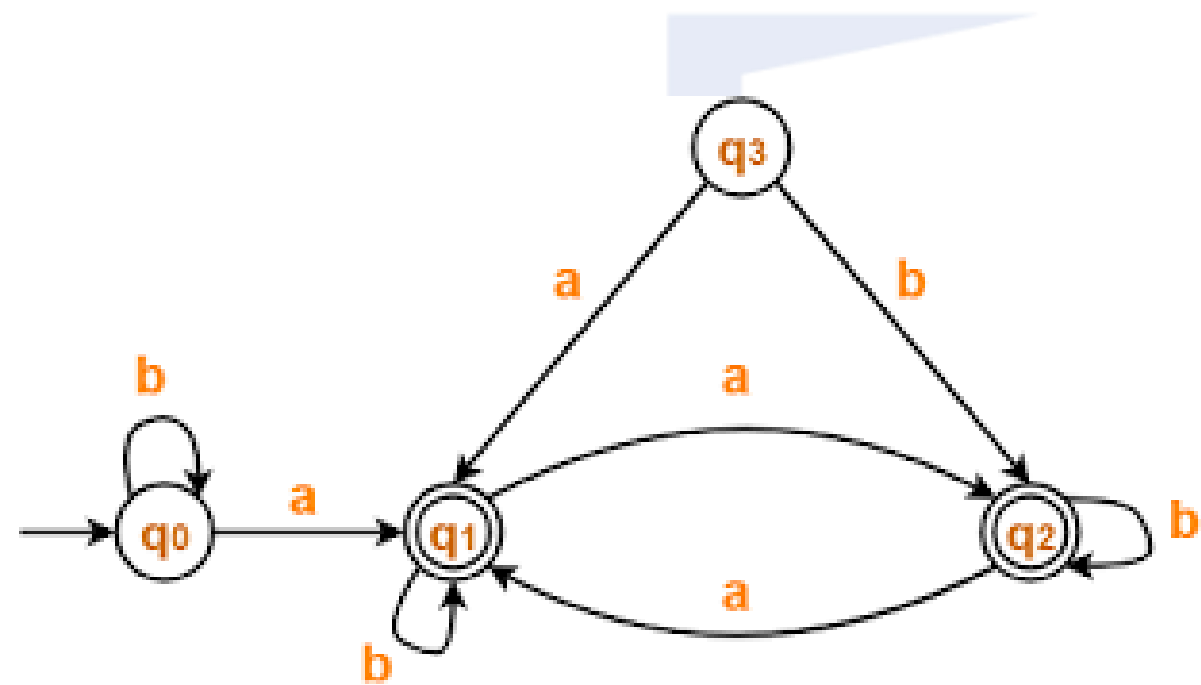
- **Equal State**-These are those states that behave in same manner on each and every input string. That is for any string w where $w \in \Sigma^*$ either both of the states will go to final state or both will go to non-final state. (remember the example of an equal state DFA).
- More formally, two states q_1 and q_2 are equivalent (denoted by $q_1 \cong q_2$) if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both of them are non-final states for all $x \in \Sigma^*$. If q_1 and q_2 are k -equivalent for all $k \geq 0$, then they are k -equivalent.

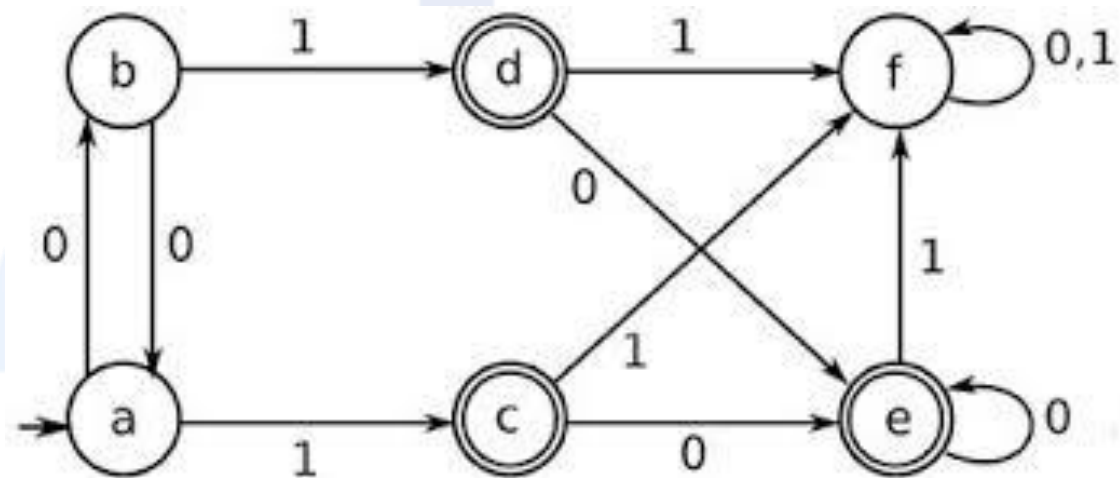
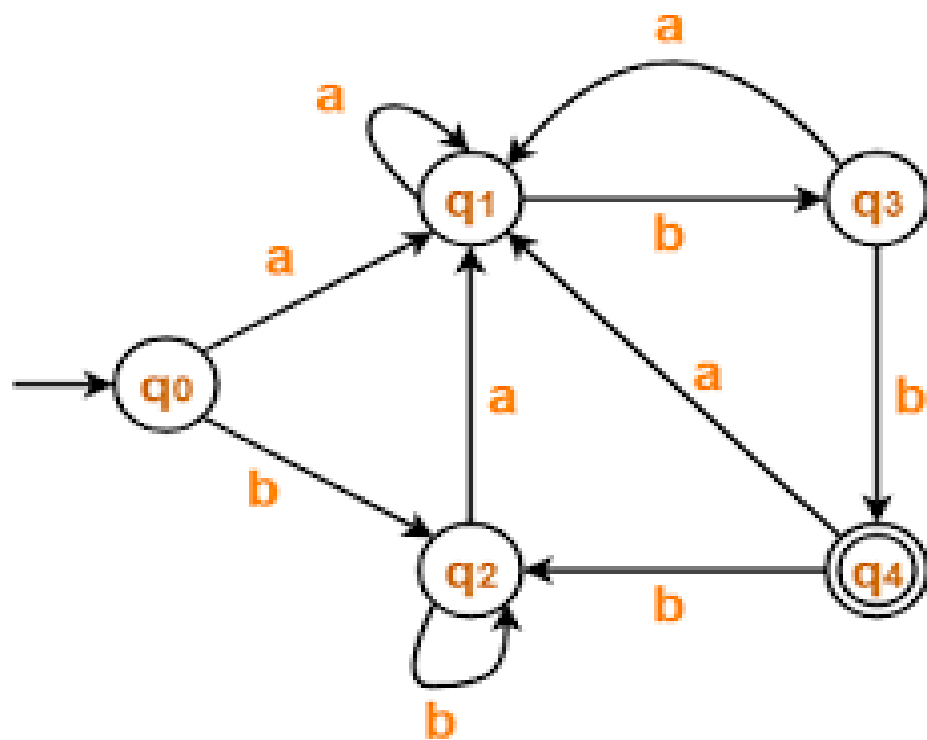


Procedure of Minimization

- For this first of all, group all the non-final states in one set and all final states in another set.
- Now, on both the sets, individually check, whether any of the underlying elements (states) of that particular set are behaving in the same way, that is are they having same transition(to same set) on each input alphabet.
- if the answer is yes, then these two states are equal, otherwise not.







KnowledgeGate.AI

Module

Paper-2 | Theory of Computation

Topic

Non-Deterministic FA

Subtopic

NFA Basics & Design

Lesson

NFA Basics & Design

Non-Deterministic FA

NFA Basics & Design

KnowledgeGate.AI

NON DETERMINISTIC FINITE AUTOMATA

- Non-determinism in finite automata means that an automaton has multiple possible moves in a given state for a particular input. Unlike deterministic automata, where the next move is uniquely defined, non-deterministic automata allow for a set of possible transitions.
- Key points:
 - **Choice of Moves:** At any given step, the automaton can choose from multiple transitions for the same input.
 - **Theoretical Concept:** Non-deterministic machines are theoretical models, not meant to be physically implemented. They help simplify the design process but are not directly used in practice.
 - **Purpose:** NFAs are easier to design compared to deterministic automata (DFA), and every NFA can be converted into an equivalent DFA for practical applications.

KnowledgeGate.AI

FORMAL DESCRIPTION OF NDFA

- A Non-Deterministic finite automaton (NDFA) is a 5-tuple $(Q, \Sigma, \delta, S, F)$ where:
 - Q is a finite and non-empty set of states
 - Σ is a finite non-empty set of finite input alphabet
 - δ is a transition function $\delta: Q \times \Sigma \rightarrow 2^Q$
 - q_0 is **initial state** (always one) ($q_0 \in Q$)
 - F is a set of final states ($F \subseteq Q$) ($0 \leq |F| \leq N$), where n is the number of states

Some points to remember

- Every DFA is also an NFA. Every NFA can be translated to an equivalent DFA, so their language accepting capability is same Both only recognize **regular languages**. Accepting power of NDFA= Accepting power of DFA.
- NFA need not to be a complete system. There can be a state that doesn't have any transition on some input symbol. A null transition is also possible for NFA, such special NFA are called Null-NFA. We will discuss it later.
- NDFA is a theoretical engine and is not implementable, but it is very easy to design compare to DFA. No concept of dead state, therefore complementation of NFA is also not possible. NDFA will respond for only valid strings and no need to respond for invalid strings. (it is a Incomplete system)

KnowledgeGate.AI

ACCEPTANCE BY NDFA

- Let 'w' be any string defined over the alphabet Σ , corresponding to w, there can be multiple transitions for NFA starting from initial state, if there exist at least one transition for which we start at the initial state and ends in any One of the final state, then the string 'w' is said to be accepted by the non-deterministic finite automata, otherwise not.
- Mathematically, it can be represented as, $L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$

Q design a NDFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, where every accepted string 'w' starts with substring s, Where $s = aba$



conclusion $\square$ if $w = sx$, $|s| = m$, then no of states in the NDFA is $m+1$

www.knowledgegate.ai

Q Design a NDFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$. where every accepted string 'w' ends with substring 's', Where $s = bab$



conclusion □ if $w = sx$, $|s| = m$, then no of states in the NDFA is $m+1$

www.knowledgegate.ai

Q Design a minimal DFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$. where every accepted string 'w' contains sub string s, Where $s = aba$



conclusion $\square$ if $w = xsx$, $|s| = m$, then no of states in the DFA is $m+1$

www.knowledgegate.ai

Q Design a NDFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with same symbol.



Q Design a NDFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with different symbol.



Q Design a minimal NDFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w= SX$, Where $s = aaa/bbb$



Q Design a minimal NDFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=XS$, Where $s = aaa/bbb$



Q Design a minimal NDFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=SXS$, Where $s = aaa/bbb$



Q Design a NFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' accepted must be like
i) $|w| = 3$

ii) $|w| \leq 3$

iii) $|w| \geq 3$



Q Design a NFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain exactly two a's.



Q Design a NFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain at least two a's.



Q Design a NFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain at most two a 's.

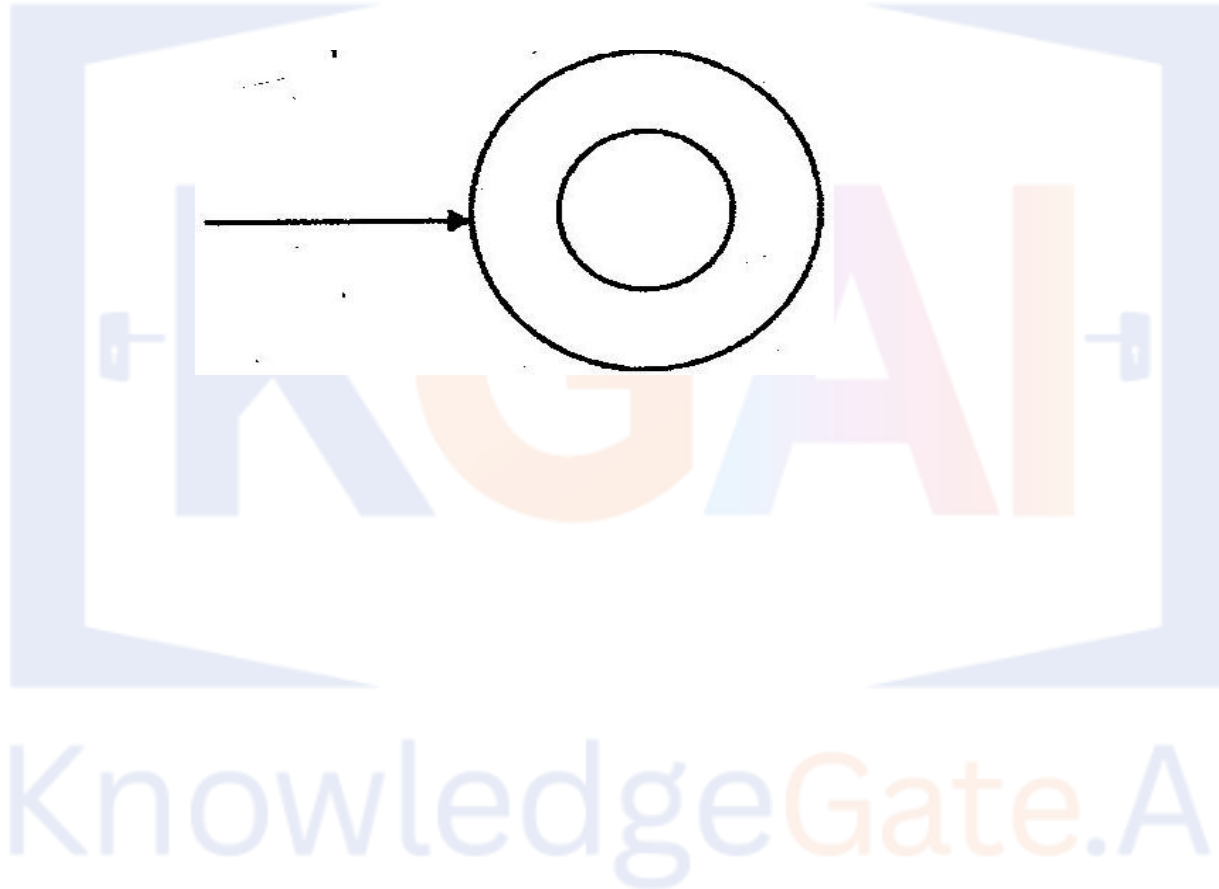


Q Design a NDFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that for every accepted string 3rd from right end is always a.



Q The FSM shown in the figure accepts

- a)** all strings
- b)** no strings
- c)** ϵ - alone
- d)** None of these



Module

Paper-2 | Theory of Computation

Topic

Non-Deterministic FA

Subtopic

NFA to DFA Conversion

Lesson

NFA to DFA Conversion

Non-Deterministic FA

NFA to DFA Conversion

KnowledgeGate.AI

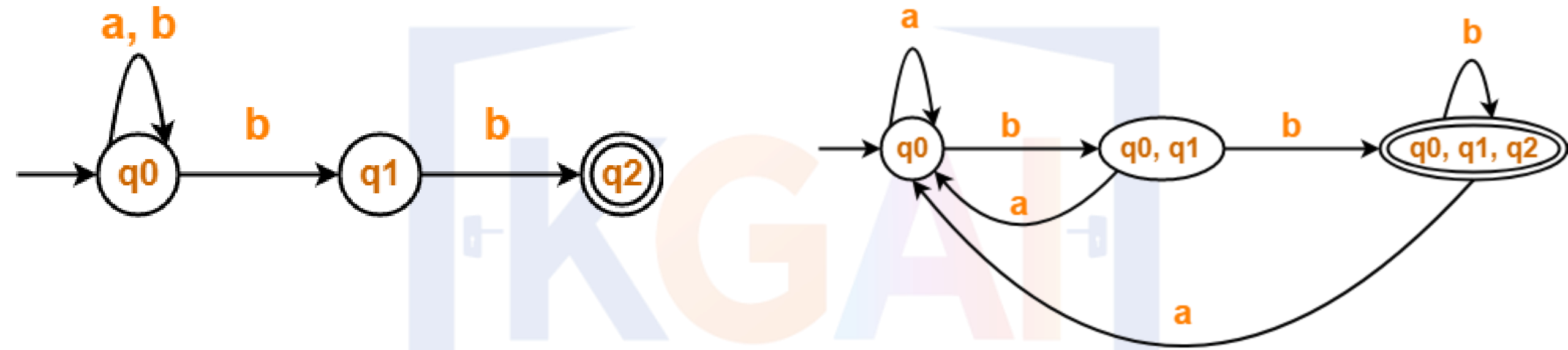
NFA and DFA Equivalence

- In this topic we will be learning about the equivalence of NFA and DFA and how an NFA can be converted to equivalent DFA. Let us take an example and understand the conversion. Since every NFA and DFA has equal power that means, for every language if a NFA is possible, then DFA is also possible. So, every NFA can be converted to DFA. The process of conversion of an NFA into a DFA is called Subset Construction.
- If NFA have 'n' states which is converted into DFA which 'm' states than the relationship between n and m will be $1 \leq m \leq 2^n$

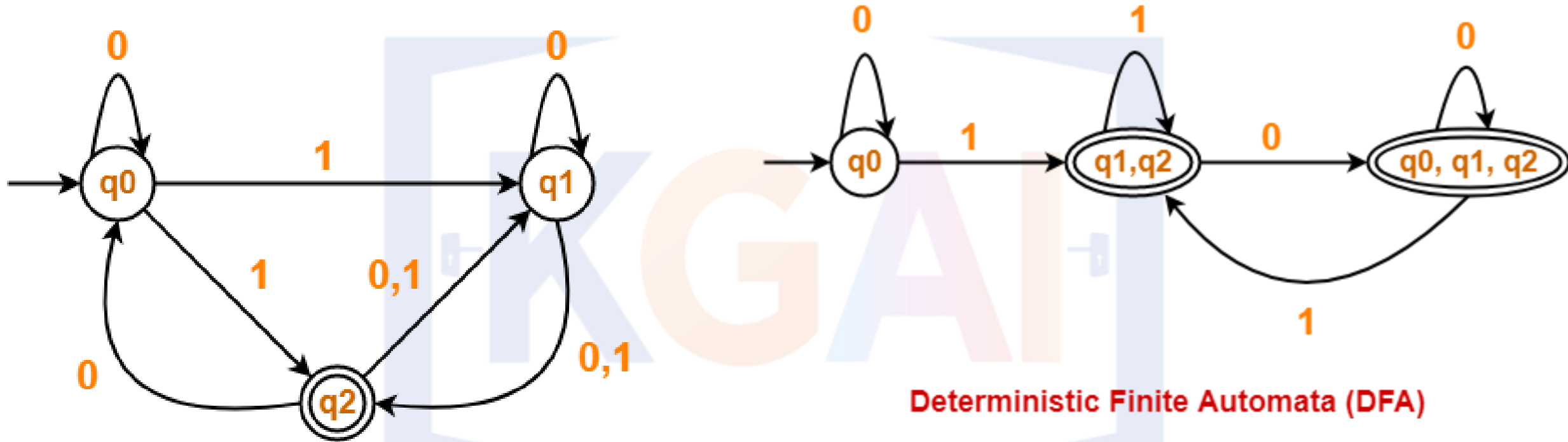
KnowledgeGate.AI

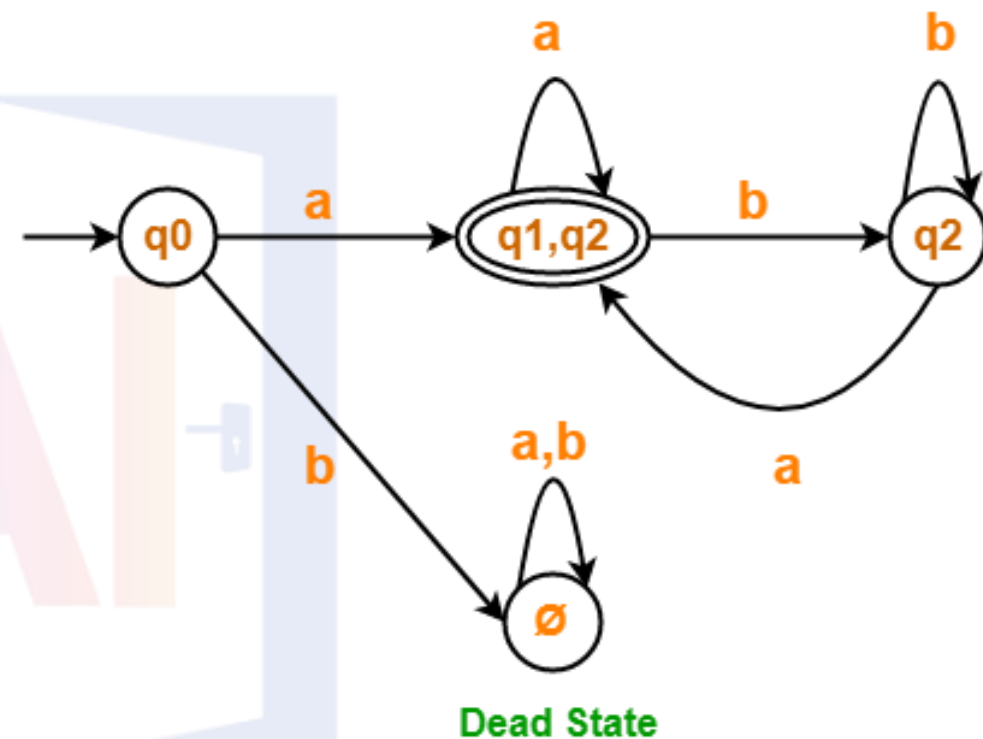
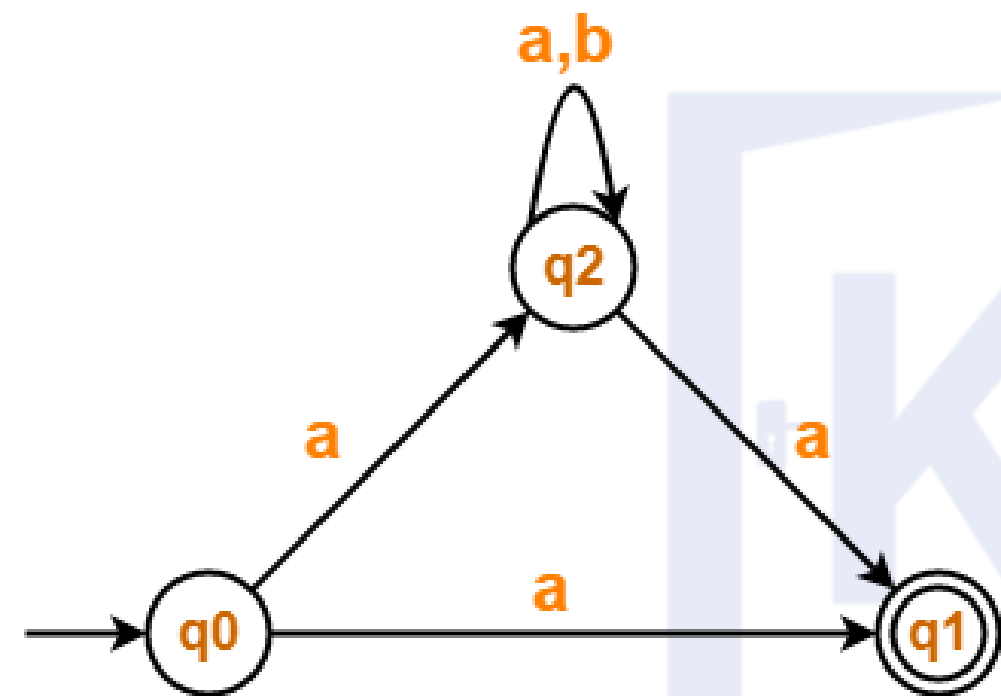
Procedure for Conversion

- There lies a fixed algorithm for the NFA and DFA conversion. Following things must be considered
 - Initial state will always remain same.
 - Start the construction of δ' with the initial state & continue for every new state that comes under the input column and terminate the process whenever no new state appears under the input column.
 - Every subset of states that contain the final state of the NFA is a final state in the resulting DFA.
 - $\delta'(q_0, q_1, q_2, q_3, \dots, q_{n-1}, a) = \bigcup_{i=0}^{n-1} \delta(q_i, a)$

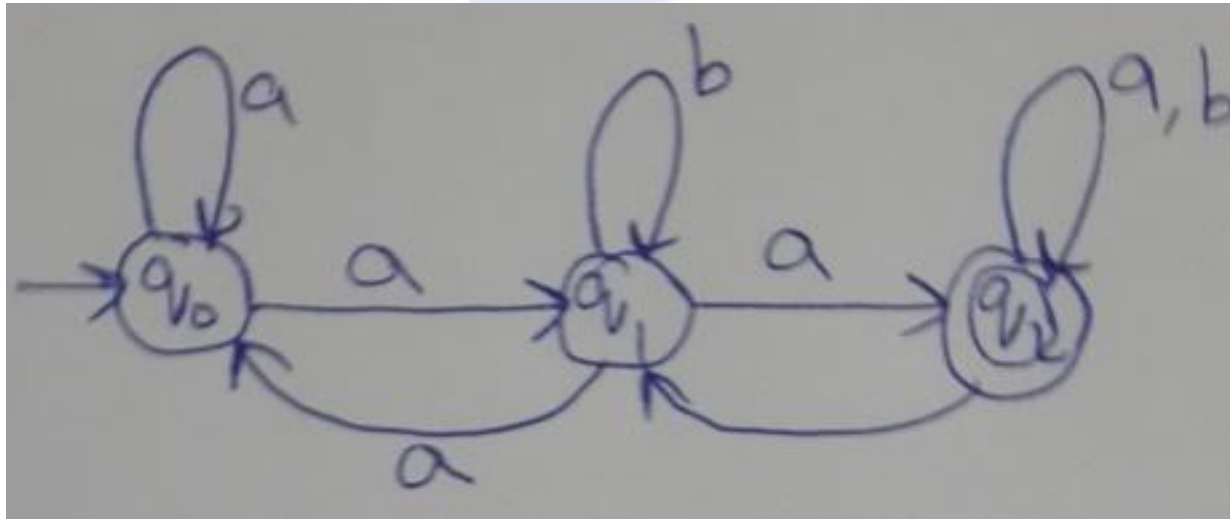


Deterministic Finite Automata (DFA)





Deterministic Finite Automata (DFA)



KnowledgeGate.AI

Module

Paper-2 | Theory of Computation

Topic

Non-Deterministic FA

Subtopic

Epsilon NFA & Conversion

Lesson

Epsilon NFA & Conversion

Non-Deterministic FA

Epsilon NFA & Conversion

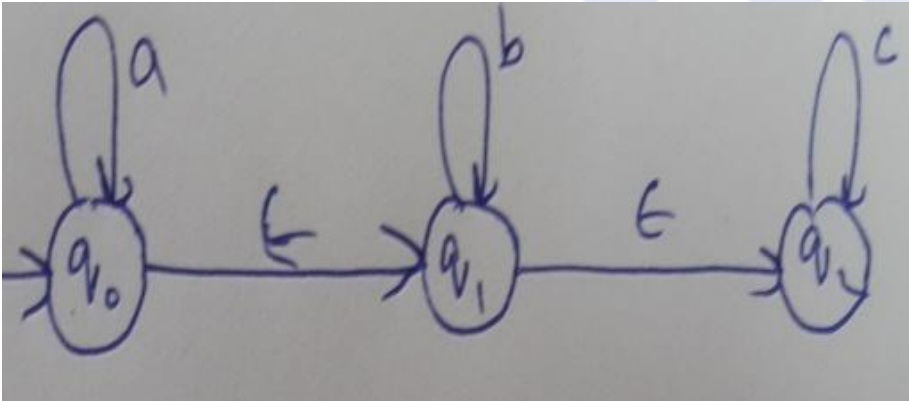
KnowledgeGate.AI

NFA WITH EPSILON MOVES (ϵ -NFA)

- An automaton that consist of null transitions is called a Null- NFA i.e. we allow a transition on null means empty string.
- ϵ -NFA is a 5-tuple $(Q, \Sigma, \delta, S, F)$ where:
 - Q is a finite and non-empty set of states
 - Σ is a finite non-empty set of finite input alphabet
 - δ is a transition function $\delta: (Q \times \{\Sigma \cup \epsilon\}) \rightarrow 2^Q$
 - S is **initial state** (always one) ($S \in Q$)
 - F is a set of final states ($F \subseteq Q$) ($0 \leq |F| \leq N$, where n is the number of states)

NULL-CLOSURE

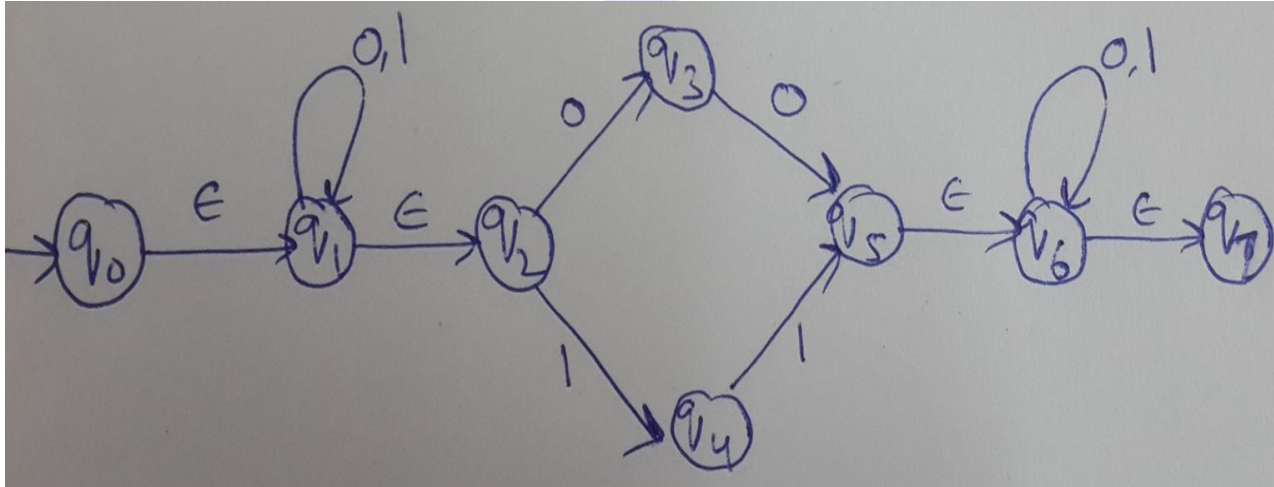
- Null closure of a set Q is defined as a set of all the states, which are at zero distance from the state Q . A set of all the states, that can be reached from the state and along a null- transition.
- ϵ -Closure(q_i)**- The set of all the states which are at zero distance from the state q_i is called ϵ -closure(q_i). Or the set of all the states that can be reached from the state q_i along ϵ labelled transition path, is known as ϵ -closure(q_i).



EQUIVALENCE BETWEEN NULL NFA TO NFA

- There will be no change in the initial state.
- No change in the total no. of states
- May be change in the number of final states.
- All the states will get the status of the final state in the resulting NFA, whose ϵ -closure contains at least one final state in the initial ϵ -NFA.

KnowledgeGate.AI



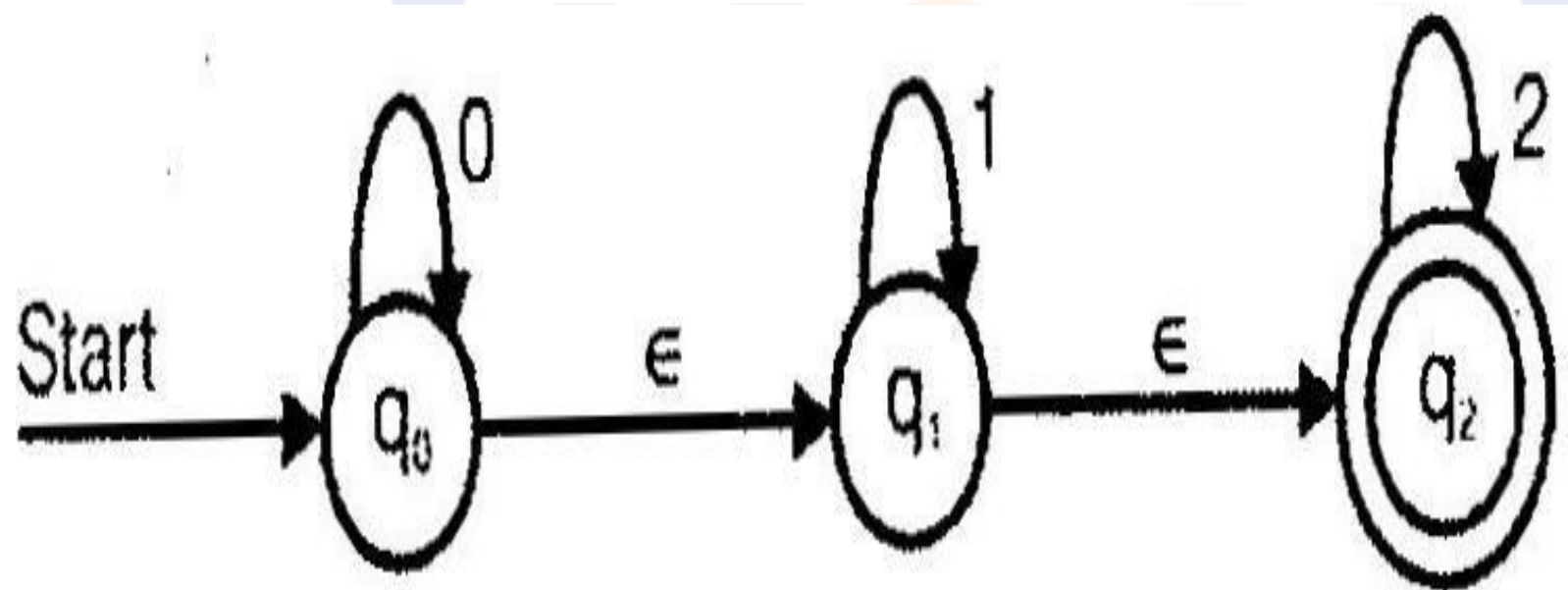
KnowledgeGate.AI

Q. If the given NFA is converted to NFA without ϵ moves , which of the following denotes the set of final states?

a) $\{q_2\}$ b) $\{q_1, q_2\}$ c) $\{q_0, q_1, q_2\}$ d) can't be determined

Q. Which of the following strings will not be accepted by the given NFA?

a) 00 11 22 b) 11 22 c) 21 d) 22



Q. Which of the following is False?

- a) The languages accepted by FAs are regular languages
- a) Every DFA is an NFA
- a) There are some NFAs for which no DFA can be constructed
- a) If L is accepted by an NFA with ϵ transitions then L is accepted by an NFA without ϵ transitions

KnowledgeGate.AI

Q. An FSM can be used to add two given integers. This remark is

- a) true
- a) false
- a) may be true
- a) none of the above



Module

Paper-2 | Theory of Computation

Topic

Non-Deterministic FA

Subtopic

Regularity & Identification

Lesson

Regularity & Identification

Non-Deterministic FA

Regularity & Identification

KnowledgeGate.AI

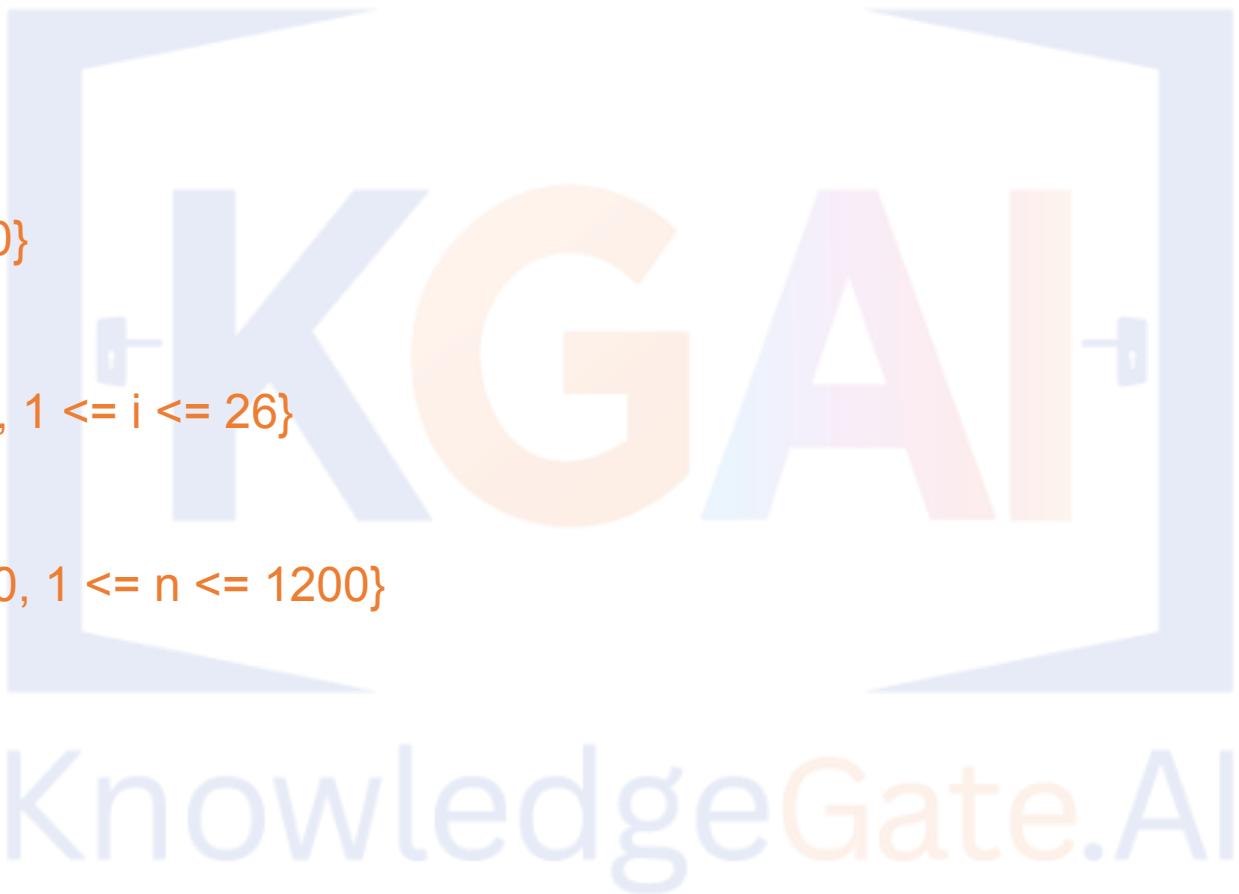
Q Consider the following languages and find which of them are regular?

1. $L = \{a^m b^n \mid m, n \geq 0\}$

1. $L = \{a^m b^n c^p \mid m, n, p \geq 0\}$

1. $L = \{a^{x_1} b^{x_2} \dots z^{x_{26}} \mid x_i \geq 0, 1 \leq i \leq 26\}$

1. $L = \{a^m b^n \mid 1 \leq m \leq 100, 1 \leq n \leq 1200\}$



1. $L = \{a^m b^n \mid m \cdot n = \text{finite}\}$

1. $L = \{a^n b^n \mid 1 \leq n \leq 2^{|GATE|}\}$

1. $L = \{a^n b^n \mid 1 \leq n \leq 2^{37\text{th prime}}\}$

1. $L = \{a^m b^n \mid m = n, 1 \leq n \leq 2^{2^{10}}\}$



1. $L = \{a^m b^n \mid m = n \mid m, n \geq 0\}$

1. $L = \{a^m b^n \mid m < n \mid m, n \geq 0\}$

1. $L = \{a^m b^n \mid m \neq n \mid m, n \geq 0\}$

1. $L = \{a^m b^n \mid m \text{ is divisible by } n\}$

1. $L = \{a^m b^n \mid m = n^p, p \geq 1\}$

1. $L = \{a^m b^n \mid \text{HCF}(m, n) = 1 \mid m, n \geq 1\}$

KnowledgeGate.AI

1. $L = \{a^m b^n \mid m + n = \text{even}\}$

1. $L = \{a^m b^n c^p \mid m = n = p\}$

1. $L = \{a^m b^n c^p \mid m + p = n\}$

1. $L = \{a^m b^n \mid m + n = \text{odd}\}$



1. $L = \{w c w^r \mid w \in \Sigma^*\}$

1. $L = \{w c w \mid w \in \Sigma^*\}$

1. $L = \{w c w^r \mid w \in \Sigma^+\}$

1. $L = \{w c w \mid w \in \Sigma^+\}$

1. $L = \{w w \mid w \in \Sigma^*\}$

1. $L = \{w w^r \mid w \in \Sigma^+\}$



1. $L = \{w c w^r \mid c, w \in \Sigma^*\}$

1. $L = \{c w w^r \mid c, w \in \Sigma^*\}$

1. $L = \{w w^r c \mid c, w \in \Sigma^*\}$

1. $L = \{w c w^r \mid c, w \in \Sigma^+\}$

1. $L = \{c w w^r \mid c, w \in \Sigma^+\}$

1. $L = \{w w^r c \mid c, w \in \Sigma^+\}$



1. $L = \{w c w \mid c, w \in \Sigma^*\}$

1. $L = \{c w w \mid c, w \in \Sigma^*\}$

1. $L = \{w w c \mid c, w \in \Sigma^*\}$

1. $L = \{w c w \mid c, w \in \Sigma^+\}$

1. $L = \{c w w \mid c, w \in \Sigma^+\}$

1. $L = \{w w c \mid c, w \in \Sigma^+\}$



Module

Paper-2 | Theory of Computation

Topic

Regular Expressions

Subtopic

Regex Basics & Definitions

Lesson

Regex Basics & Definitions

Regular Expressions

Regex Basics & Definitions

KnowledgeGate.AI

Regular Expressions

- One way of describing regular language is via the notation of regular expression. An expression of strings which represents regular language is called regular expression.
- The regular expressions are useful for representing certain sets of strings(Language) in an algebraic fashion.
- We give a formal recursive definition of regular expressions over Σ as follows:
 - Any terminal symbol (i.e. an element of Σ), ϵ and Φ are regular expressions (**Primitive regular expressions**).
 - A regular expression is valid iff it can be derived from a primitive regular expression by a finite number of applications of operators.

Regular Language:- Any set(language) represented by a regular expression is called a **Regular language**. If for example, $a, b \in \Sigma$, then

- $R = a$ denotes the $L = \{a\}$
- $R = a.b$ denotes $L = \{ab\}$ concatenation
- $R = a + b$ denotes $L = \{a, b\}$ Union
- $R = a^*$ denotes the set $\{\epsilon, a, aa, aaa, \dots\}$ known as Kleene closure.
- $R = a^+$ Positive closure $\{a, aa, aaa, \dots\}$
- $R = (a + b)^*$ denotes $\{a, b\}^*$

Operators

When we view a in Σ as a regular expression, we denote it by $\mathbf{a}$.

- If R is a regular expression, then $(\mathbf{R})$ is also a regular expression.
- The iteration (or closure) of a regular expression $\mathbf{R}$ written as $\mathbf{R}^*$, is also a regular expression.
- The iteration (or closure) of a regular expression $\mathbf{R}$ written as $\mathbf{R}^+$, is also a regular expression.
- The concatenation of two regular expressions $\mathbf{R}_1$ and $\mathbf{R}_2$, written as $\mathbf{R}_1 \mathbf{R}_2$, is also a regular expression.
- The union of two regular expressions $\mathbf{R}_1$ and $\mathbf{R}_2$, written as $\mathbf{R}_1 + \mathbf{R}_2$, is also a regular expression.
- The precedence order to solve is $()$ Bracket, $*$ (Kleene Closure), $+$ Positive Closure, Concatenation, Union.

IDENTITIES FOR Regular Expression

- Two regular expressions P and Q are equivalent (we write $P = Q$)
 - if P and Q represent the same set of strings.
- Every regular expression can generate only one regular language but, a regular language can be generated by more than one regular expression I.e. means two different regular expression can generate same language.
- Two regular expression are said to be equal if they generate same language.
- $r_1 = a^*$
- $r_2 = a^* + (aa)^*$



हमारी language बताओ

- $R=\{a\}$
- $R=\{a + b\}$
- $R=\{a + b + c\}$
- $R=\{a.b\}$
- $R=\{a.b + a\}b$



Module

Paper-2 | Theory of Computation

Topic

Regular Expressions

Subtopic

Regex Design & Algebra

Lesson

Regex Design & Algebra

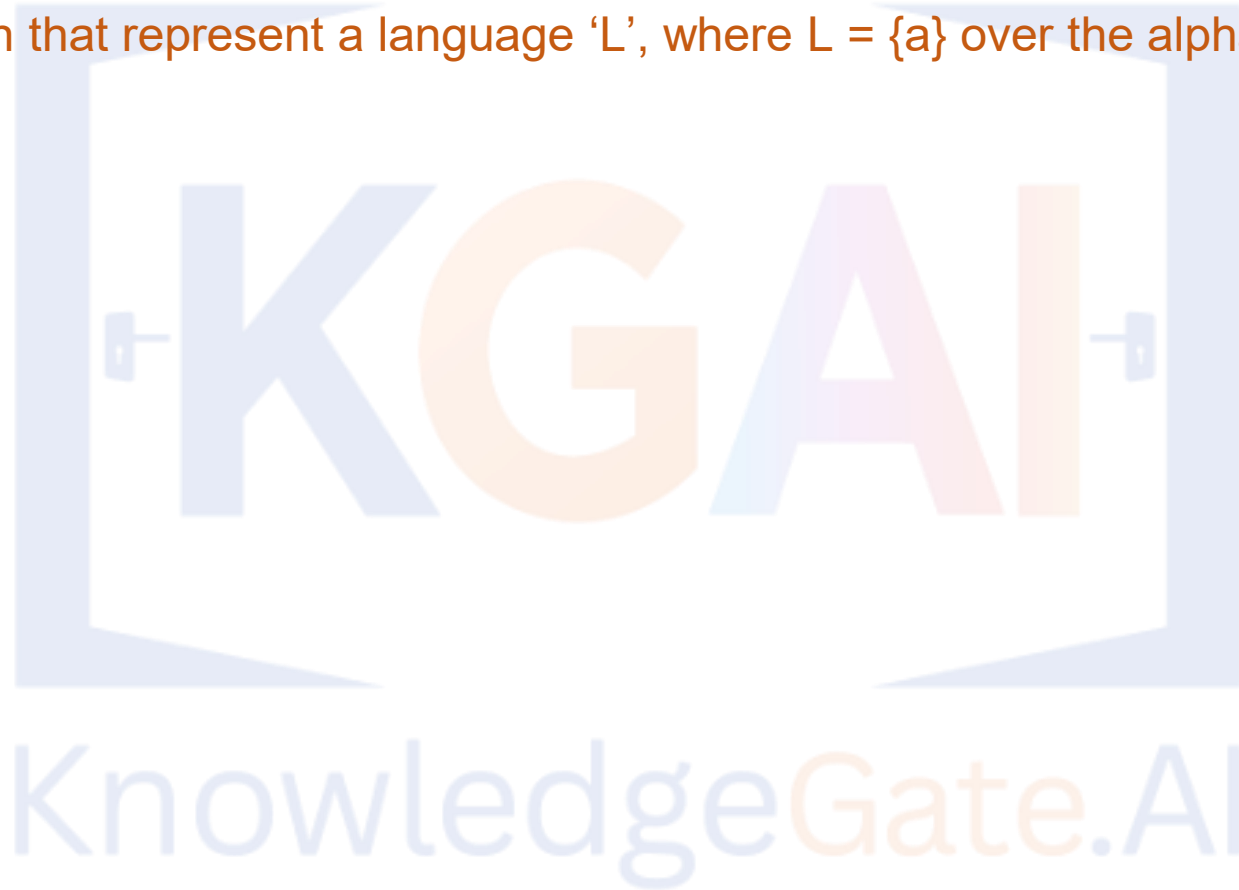
Regular Expressions

Regex Design & Algebra

KnowledgeGate.AI

FINITE AUTOMATA AND regular expression

Q Design a regular expression that represent a language 'L', where $L = \{a\}$ over the alphabet $\Sigma = \{a\}$.



Q design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$, where every accepted string 'w' starts with substring s

i) s = b

ii) s = ab

iii) s = abb



Q design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$. where every accepted string 'w' ends with substring 's'.

i) s = ab

ii) s = aa

iii) s = bab



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$. where every accepted string 'w' contains sub string s.

i) abb

ii) aba



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with a.



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with same symbol.



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with different symbol.



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=SX$.

i) $s = aa/bb$

ii) $s = aaa/bbb$



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=XS$.

i) $s = aa/bb$

ii) $s = aaa/bbb$



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=XSX$.

i) $s = aa/bb$

ii) $s = aaa/bbb$



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' accepted must be like

i) $|w| = 3$

ii) $|w| \leq 3$

iii) $|w| \geq 3$



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain exactly two a's.



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain at least two a's.



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain at most two a's.



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$ such that for every accepted string 2nd from left end is always b.



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$ such that for every accepted string 4th from right end is always a.



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' where $|W| = 0(\text{mod } 3)$?



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' where $|W| = 3(\text{mod } 4)$?



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' where $|w|_a \equiv 0 \pmod{3}$?



Q Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' where $|w|_a \equiv 2 \pmod{3}$?



$$Q L = \{a^m b^n \mid m, n \geq 0\}$$

$$Q L = \{a^m b^n \mid m \geq 1, n \geq 1\}$$

$$Q L = \{a^m b^n \mid m \geq 2, n \geq 3\}$$



$$Q L = \{a^m b^n c^p \mid m, n, p \geq 1\}$$

$$Q L = \{a^n b^n \mid n \geq 1\}$$



$Q L = \{a^m b^n \mid m + n = \text{even}\}$



www.knowledgegate.ai

Q Write a regular expression for the language, $L = \{w c w^r \mid w, c \in \{a, b\}^+\}$



Algebraic Properties of regular expression

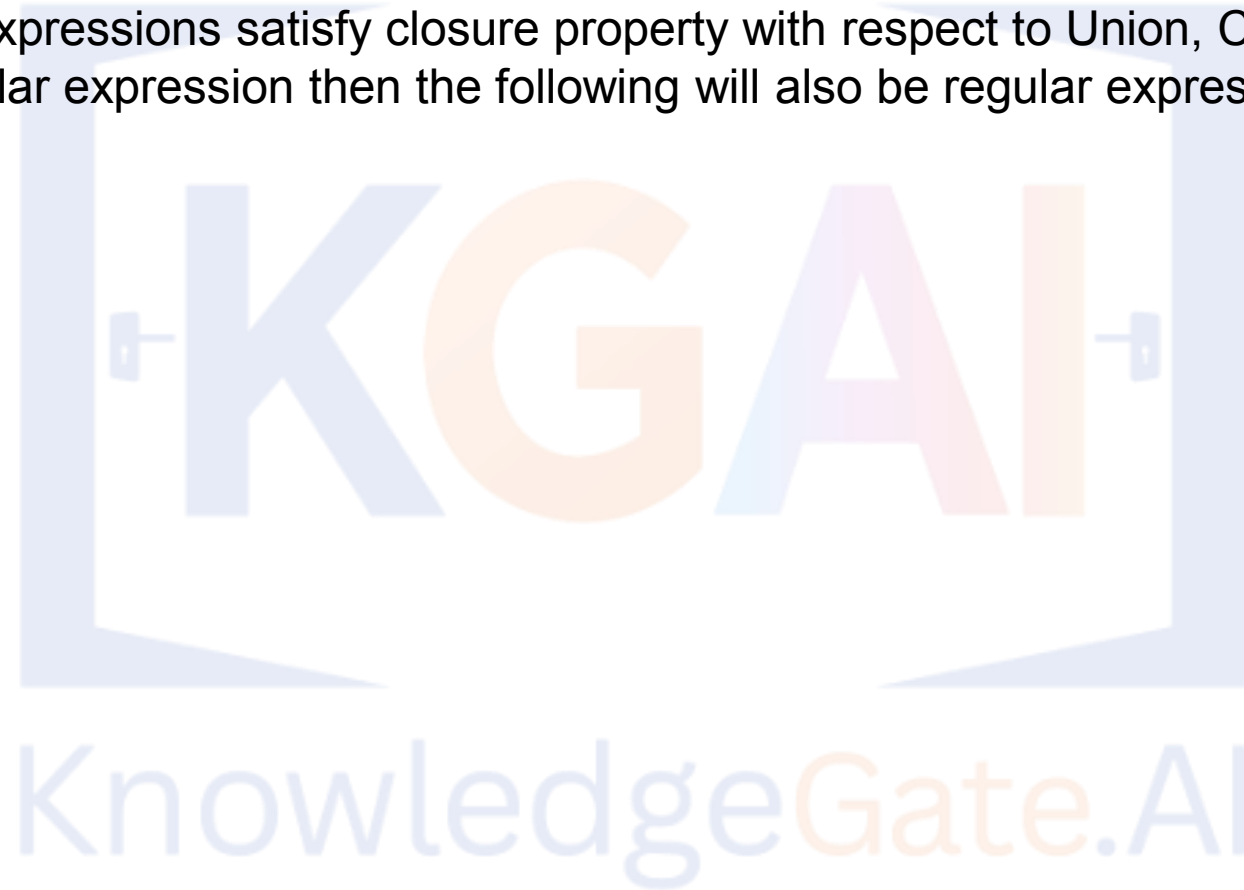
Closure Property - regular expressions satisfy closure property with respect to Union, Concatenation and kleene closure. If R_1 and R_2 are regular expression then the following will also be regular expression.

$$r_1 + r_2$$

$$r_1 \cdot r_2$$

$$r_1^*$$

$$r_1^+$$



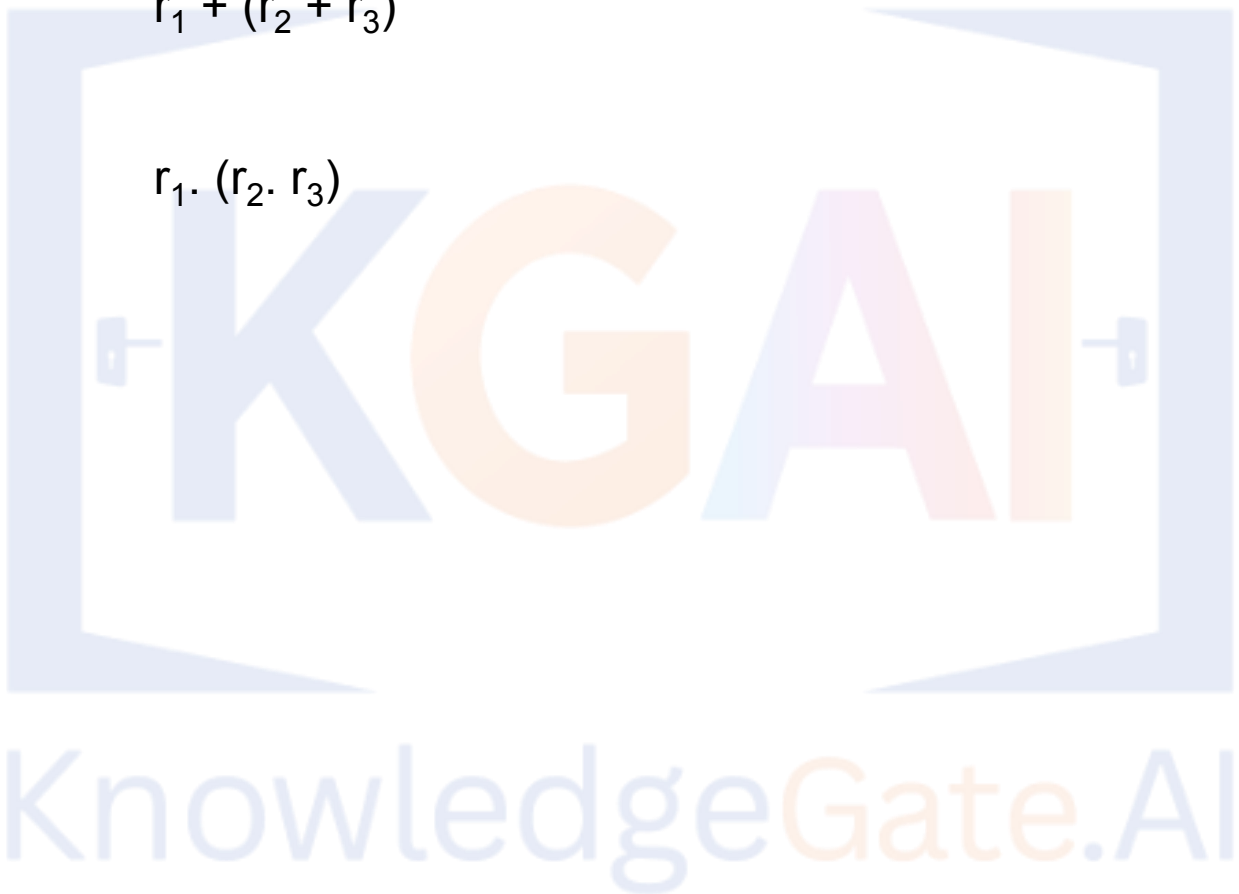
Associative Property- regular expression satisfy associative property with respect to union and intersection

$$(r_1 + r_2) + r_3$$

$$r_1 + (r_2 + r_3)$$

$$(r_1 \cdot r_2) \cdot r_3$$

$$r_1 \cdot (r_2 \cdot r_3)$$



Identity Property- The identity property is satisfied as follows-

$r \cdot \boxed{} = r$

$r + \boxed{} = r$



Inverse Property- The inverse property is not satisfied with respect to concatenation and union.

$r \cdot \boxed{}$

$= \epsilon$

$r + \boxed{}$

$= \phi$



Commutative Property-regular expressions are commutative with respect to union but not with respect to concatenation.

$r_1 + r_2$

$r_1 \cdot r_2$



Distributive Property-regular expression satisfy this property as follows-

$r_1(r_2+r_3)$

$r_1r_2+r_1r_3$

$(r_1+r_2) r_3$

$r_1r_3+r_2r_3$

$r_1+(r_2 \cdot r_3)$

$(r_1+r_2)(r_1+r_3)$

$(r_1 \cdot r_2) + r_3$

$(r_1 + r_3) \cdot (r_2 + r_3)$



Idempotent Property-regular expressions satisfies idempotent property with respect to union but not with respect to concatenation.

$$r_1 + r_1$$


$$r_1 \cdot r_1$$


r_1

r_1



Identities for regular expressions

1. $\varphi + r =$

1. $\varphi \cdot r = r \cdot \varphi =$

1. $\epsilon \cdot r = r \cdot \epsilon =$

1. $\epsilon^* =$

1. $\varphi^* =$



1. $r^*r^* =$

1. $r. r^* =$

1. $r^+. r^* =$

1. $r^+ r^+ =$



1. $r^+ \cup r^* =$

1. $r^+ \cap r^* =$



1. $(r^*)^* =$

1. $(r^+)^* =$

1. $((r^+)^*)^* =$

1. $((r^*)^*)r^+ =$



1) $(r_1+r_2)^*$

2) $(r_1+r_2)^*$

3) $(r_1+r_2)^*$

4) $(r_1+r_2)^*$

$(r_1^* + r_2^*)^*$

$(r_1 + r_2^*)^*$

$(r_1^* + r_2)^*$

$(r_1^* + r_2^*)$



KnowledgeGate.AI

1. $(r_1+r_2)^*$ $(r_1^* \cdot r_2^*)^*$

1. $(r_1+r_2)^*$ $(r_1 \cdot r_2^*)^*$

1. $(r_1+r_2)^*$ $(r_1^* \cdot r_2)^*$

1. $(r_1+r_2)^*$ $(r_1 \cdot r_2)^*$

1. $(r_1+r_2)^*$ $(r_1^* \cdot r_2^*)$

$$r_1(r_2 \cdot r_1)^*$$



$$(r_1 \cdot r_2)^* r_1$$



KnowledgeGate.AI

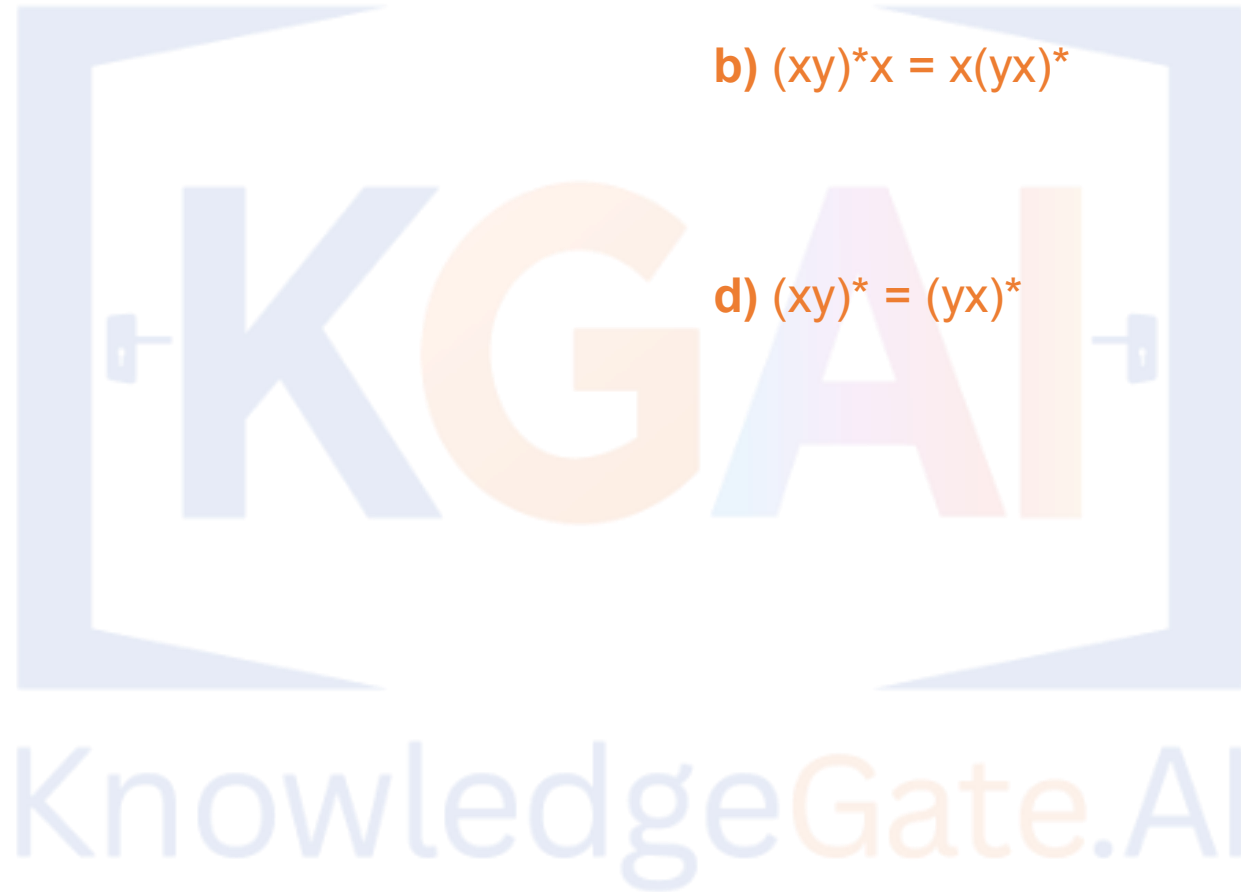
Q Which of the following is correct

a) $(xx)^*y = x(xy)^*$

b) $(xy)^*x = x(yx)^*$

c) $x(xy)^* = (xx)^*y$

d) $(xy)^* = (yx)^*$



Q find the no of strings of length ≤ 3 generated by the $(a + ab)^+$



Q which of the following statements are correct

a) r^* , r^+ always represent finite language.

b) r^* , r^+ always represent infinite language.

c) $r^* = r^+$ if and only if $r = \epsilon$

d) $r^* = r^+ = r$ if $r = \Phi$



Module

Paper-2 | Theory of Computation

Topic

Regular Expressions

Subtopic

FA to Regex Conversion

Lesson

FA to Regex Conversion

Regular Expressions

FA to Regex Conversion

KnowledgeGate.AI

EQUIVALENCE BETWEEN Regular Expression AND FINITE AUTOMATA

- **ARDEN'S THEOREM** is the mechanism for the construction of a regular expression from a finite automaton.

Q Consider a DFA and convert it into regular expression using Arden's theorem?

$$\delta(A, a) = A$$

$$\delta(A, b) = B$$

$$\delta(B, a) = B$$

$$\delta(B, b) = B$$

A is the initial state and B is the final state

Q Consider a DFA and convert it into regular expression using Arden's theorem?

$$\delta(A, b) = A$$

$$\delta(A, b) = B$$

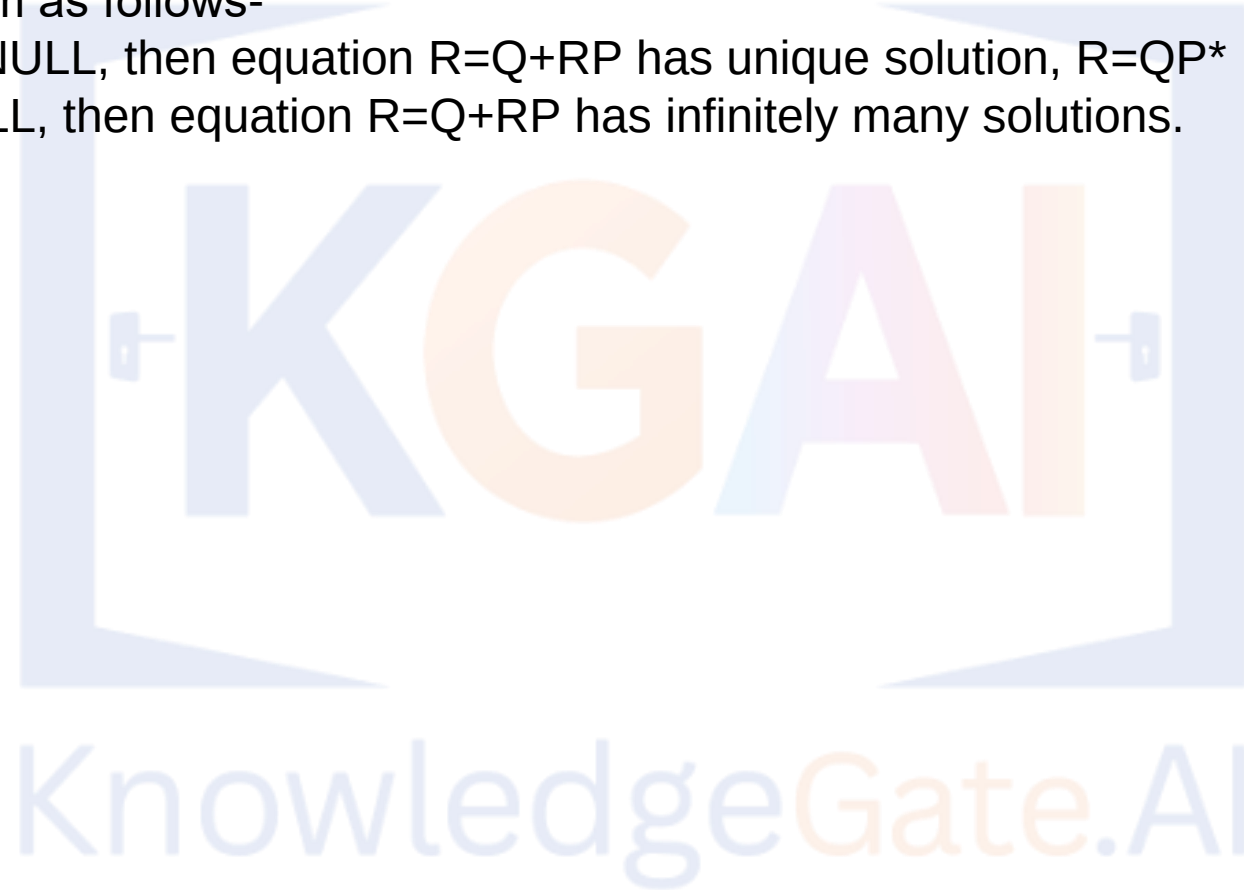
$$\delta(B, a) = C$$

$$\delta(C, b) = C$$

A is the initial state and B,C Is the final state

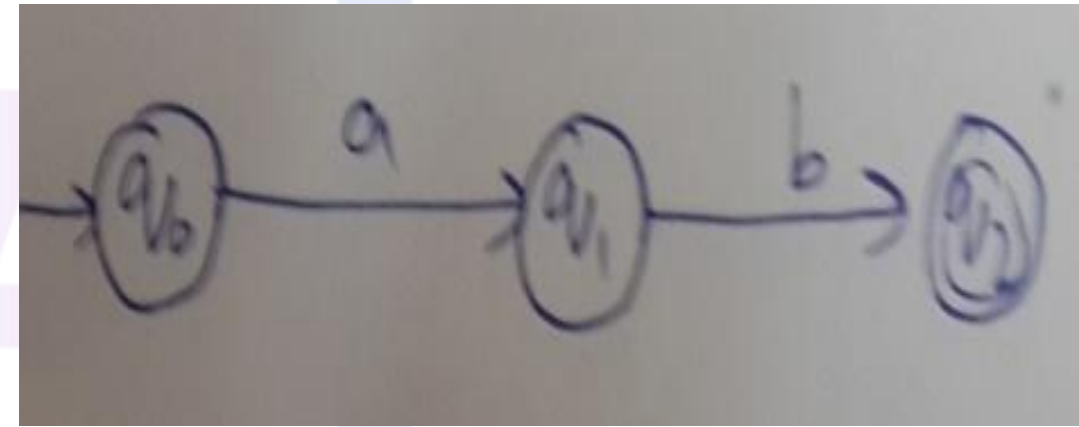
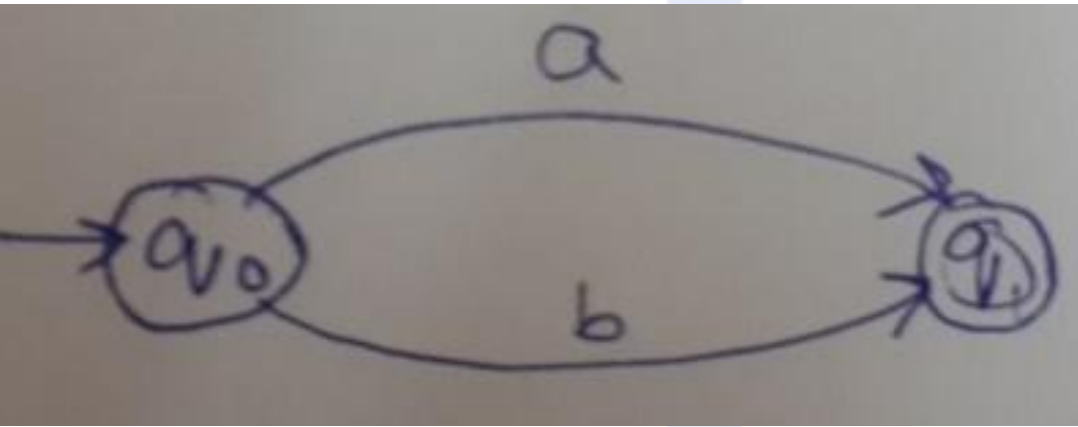


- Steps used-
 - For, every individual state of the DFA, write an expression for every incoming and outgoing input alphabet.
 - Apply Arden's theorem as follows-
 - If P is free from NULL, then equation $R=Q+RP$ has unique solution, $R=QP^*$
 - If P contains NULL, then equation $R=Q+RP$ has infinitely many solutions.

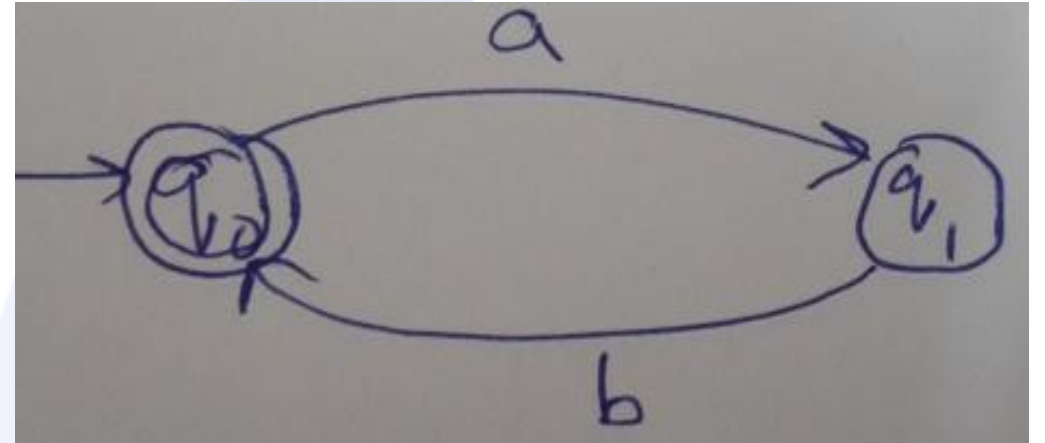
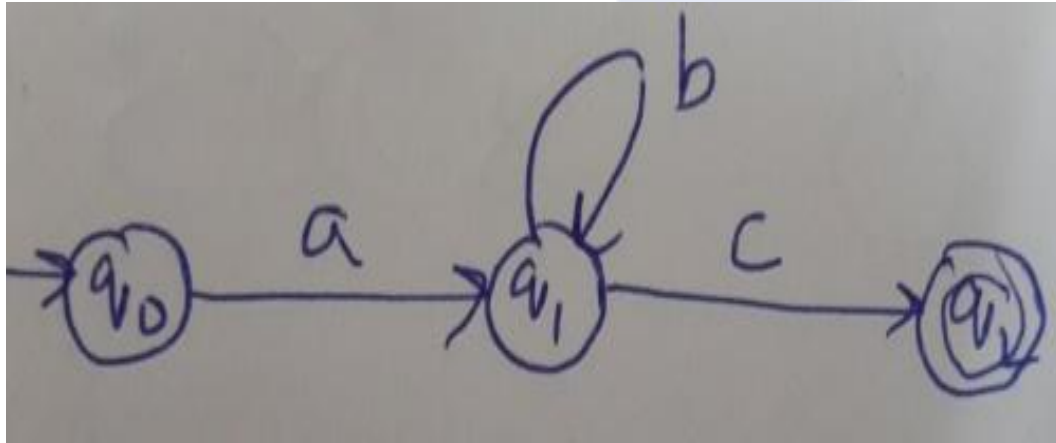


Conversion from Finite Automata to regular expression

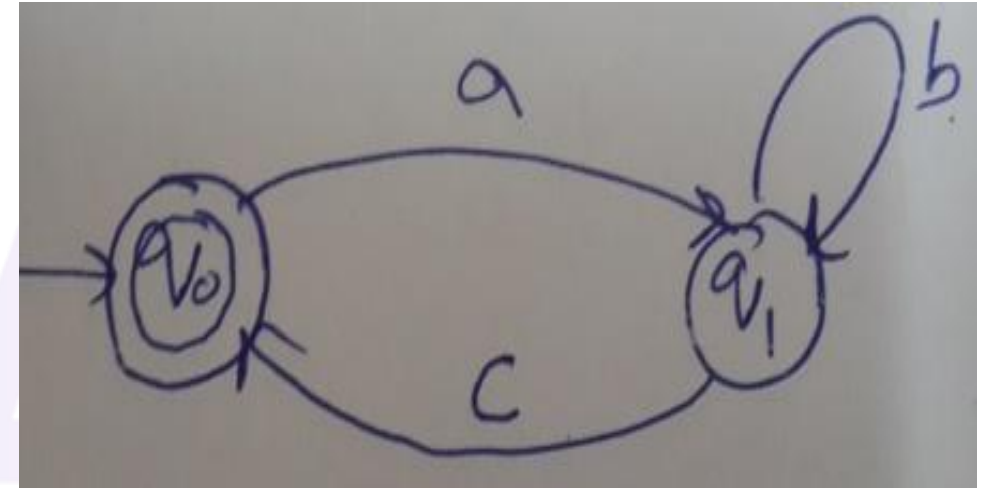
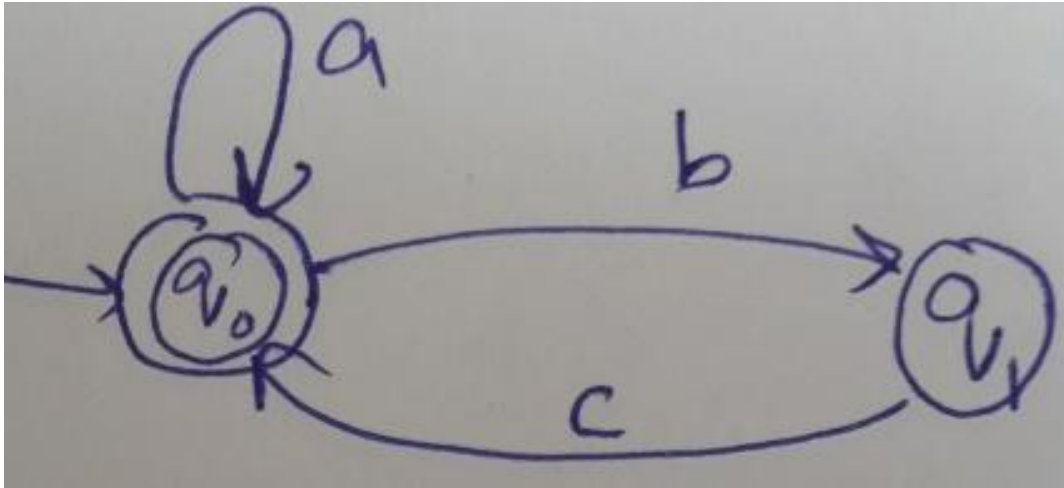
Q Write regular expressions for the following machines



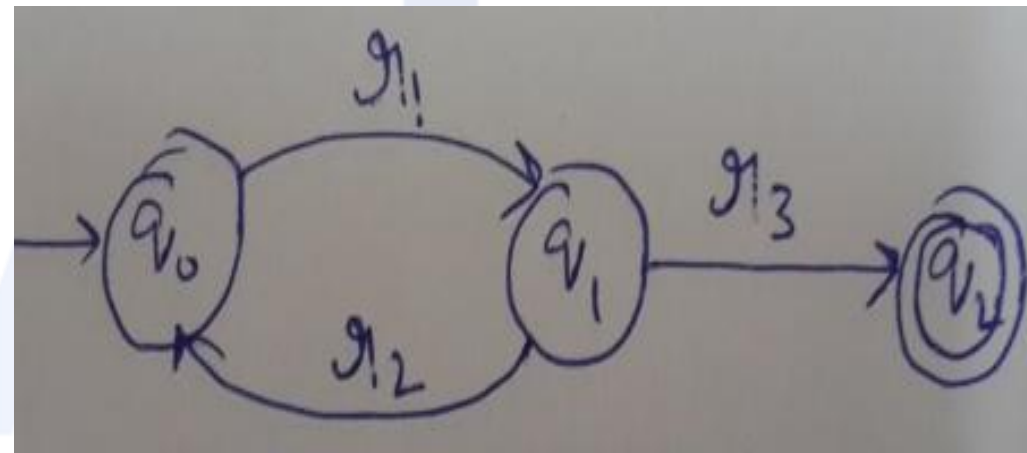
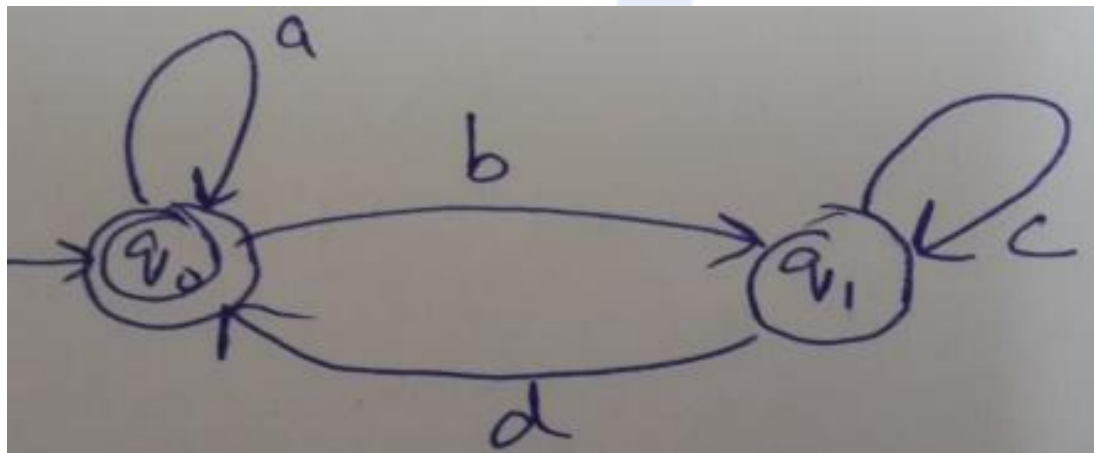
KnowledgeGate.AI



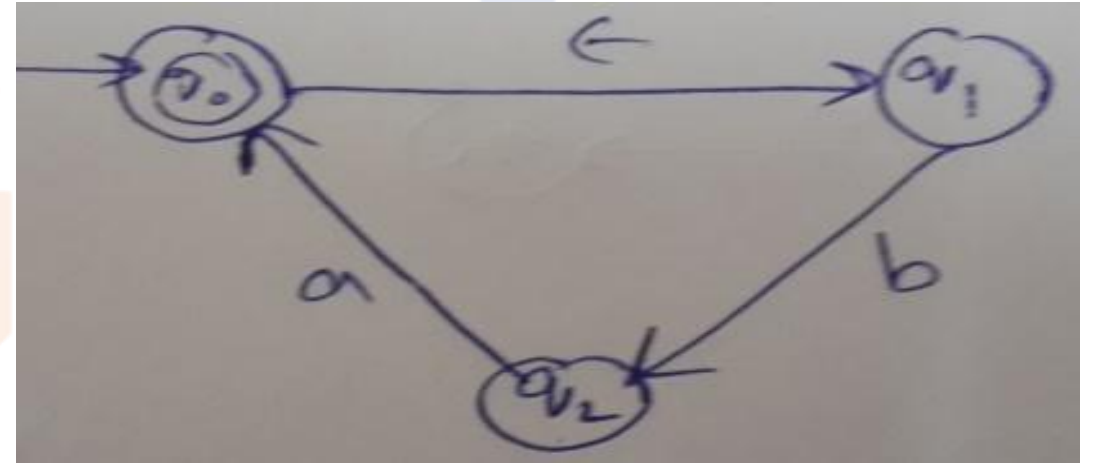
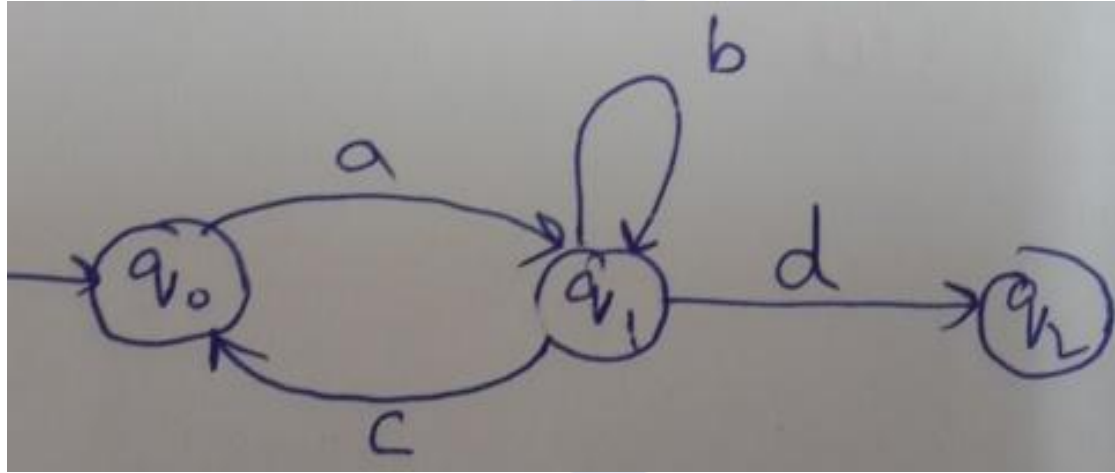
KnowledgeGate.AI



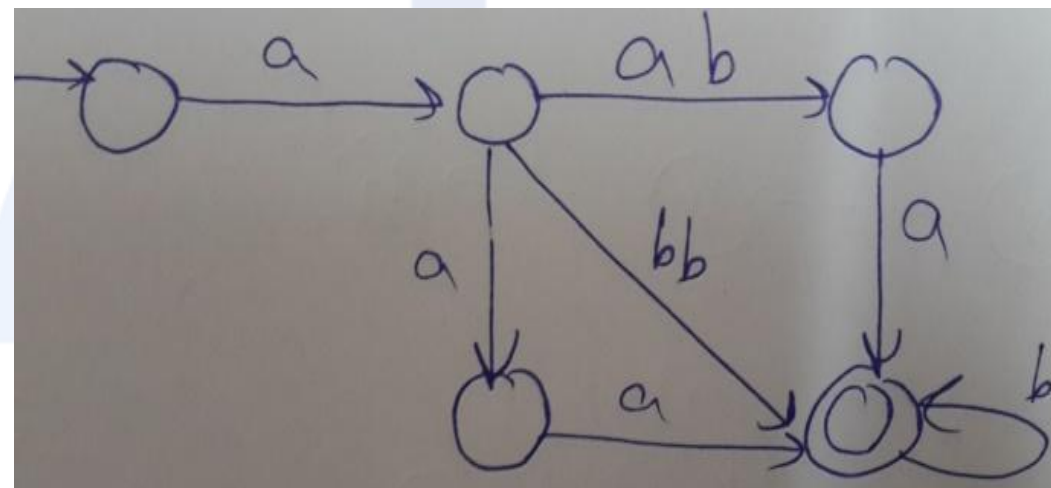
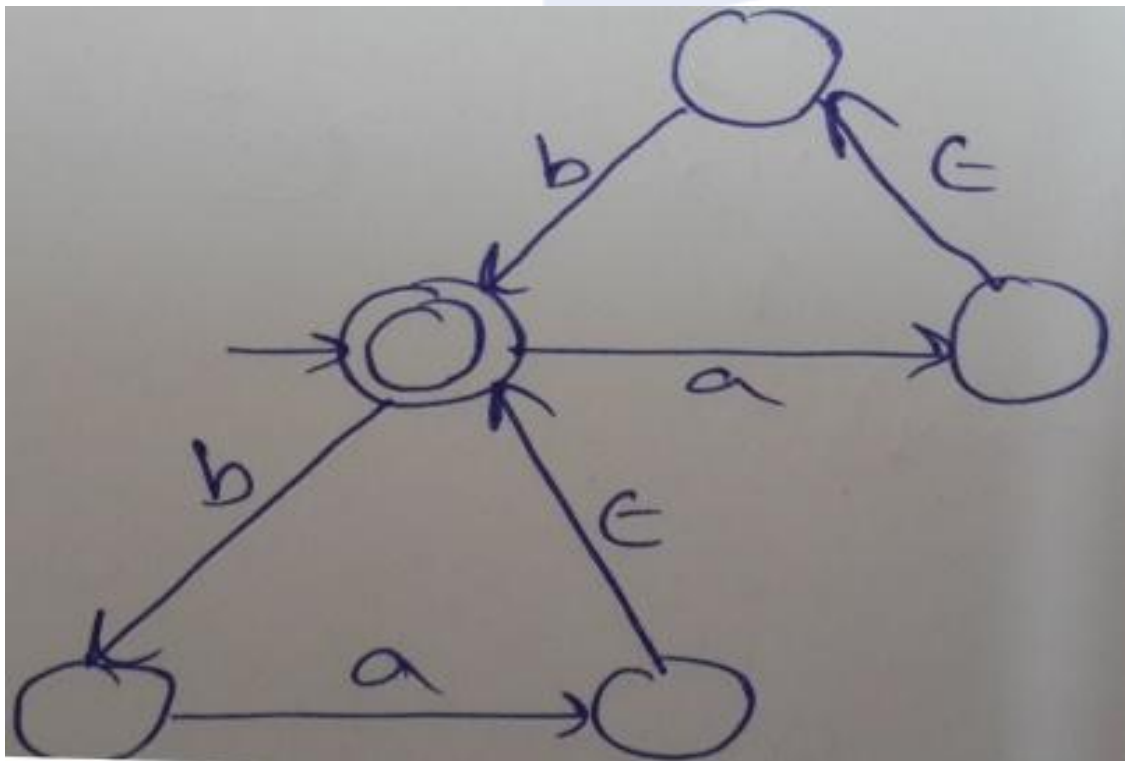
KnowledgeGate.AI

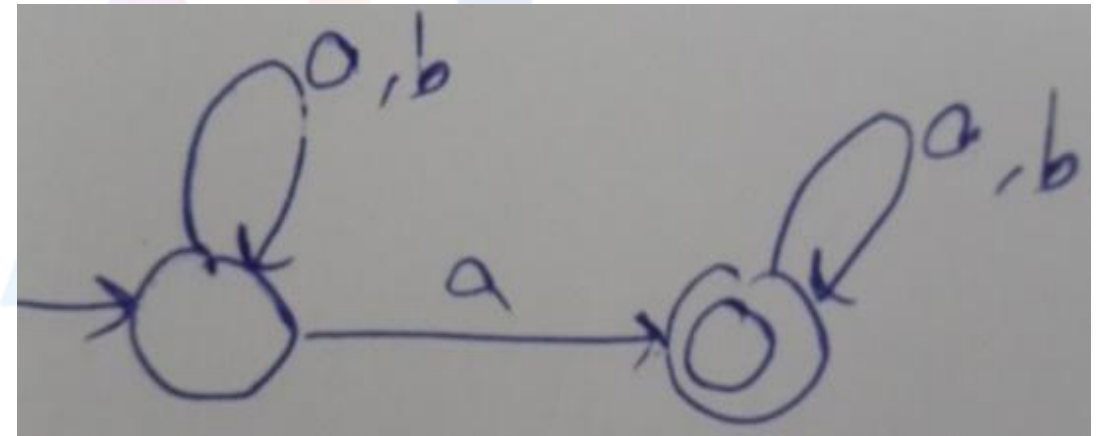
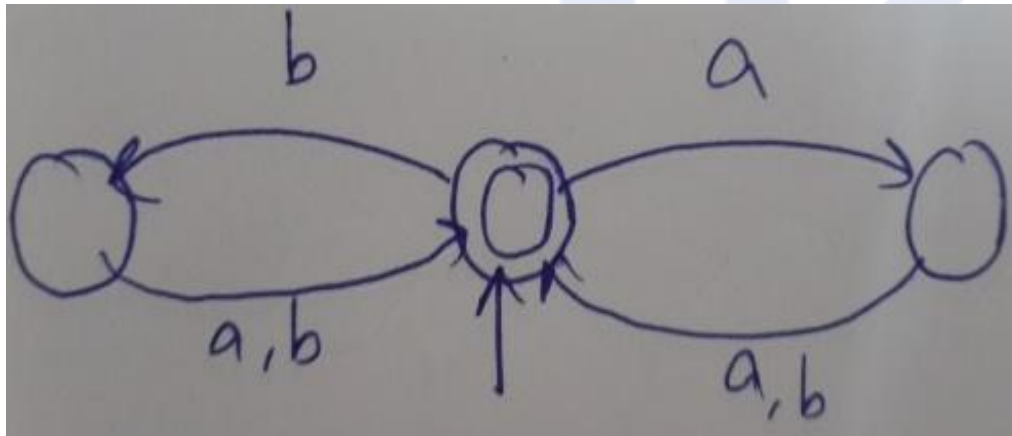


KnowledgeGate.AI

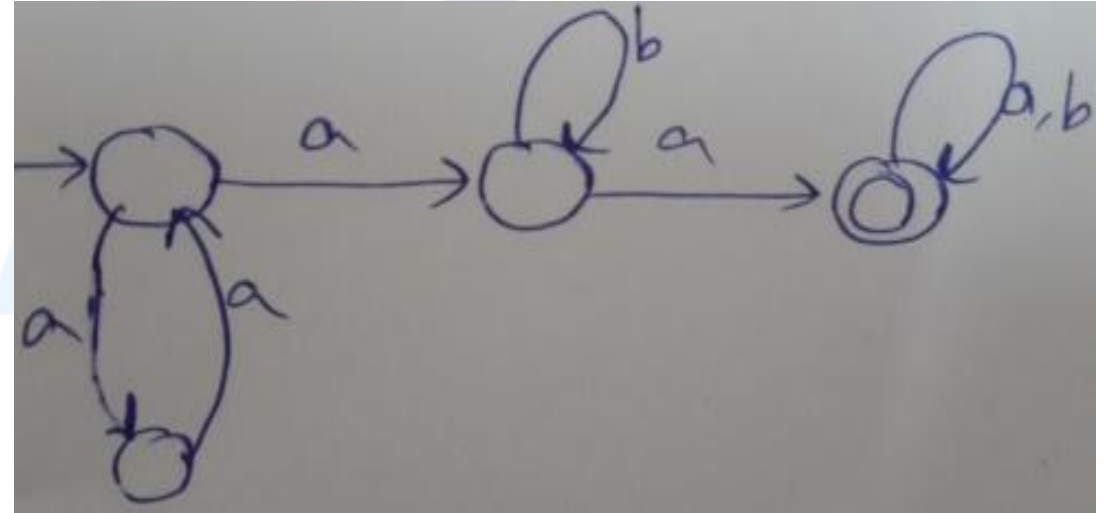
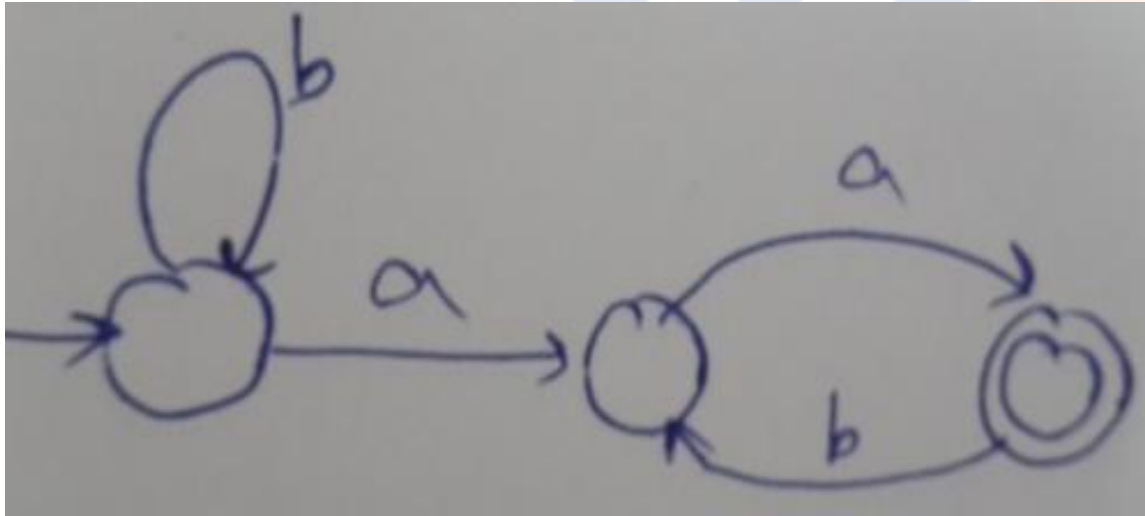


KnowledgeGate.AI

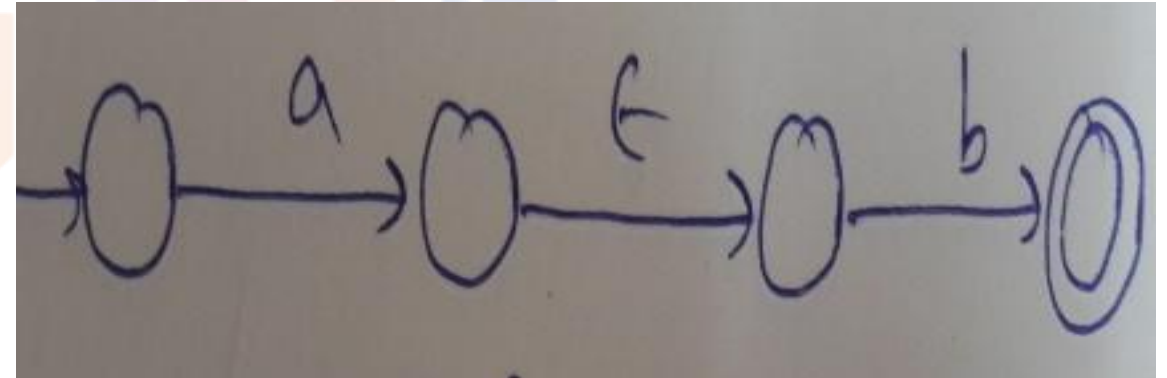
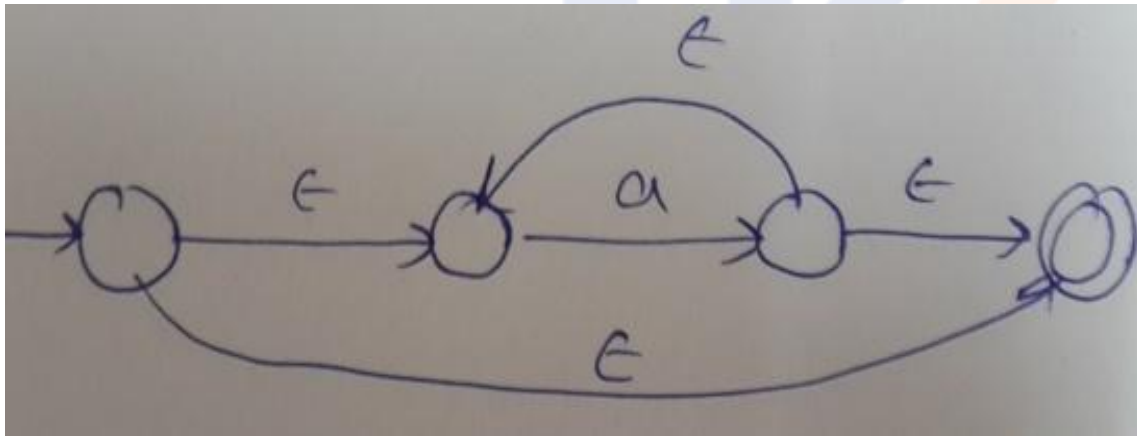




KnowledgeGate.AI



KnowledgeGate.AI



KnowledgeGate.AI

Module

Paper-2 | Theory of Computation

Topic

Regular Expressions

Subtopic

Regex & FA Equivalence

Lesson

Regex & FA Equivalence

Regular Expressions

Regex & FA Equivalence

KnowledgeGate.AI

Conversion from regular expression in Finite Automata

i) R^*

ii) $(R_1.R_2)^*$

iii) $(R_1+R_2)^*$

iv) $(R_1^*R_2.R_3^*)$

v) $R = a^*b(ab)^*$

vi) $R = (a + ba)^*ab^*$

vii) $R = (aa + aaa)^*$

viii) $R = (a + aaaaa)^*$



1. $0^* 10^* 10^*$

1. $((ab)^* + c^*)^*$

1. $(\epsilon + a + aa + aaa)b^* + (a + b)^* ba(a + b)^*$

1. $0^*(10^* 1^*)^* 0^*$



1. $a^*b(b^* + aa^*b)^*$

1. $((11^*0+0)(0 + 1)^*0^*1^*)$

1. $(ab+ bc+ acc).(a+ bc)^*.(\epsilon+ bc^*a)^*$



Q the string 1101 does not belong to the set

a) $110^*(0+1)$

b) $1(0+1)^*101$

c) $(10)^*(10)^*(00+11)^*$

d) $(00 + (11)^*01)^*$



Q which of the following re, over the $\Sigma = \{0, 1\}$ denotes the set of all strings that not contain 100 as substring

a) $0^*(0+1)^*$

b) 0^*1010^*

c) 0^*1^*01

d) $0^*(10+1)^*$



Module

Paper-2 | Theory of Computation

Topic

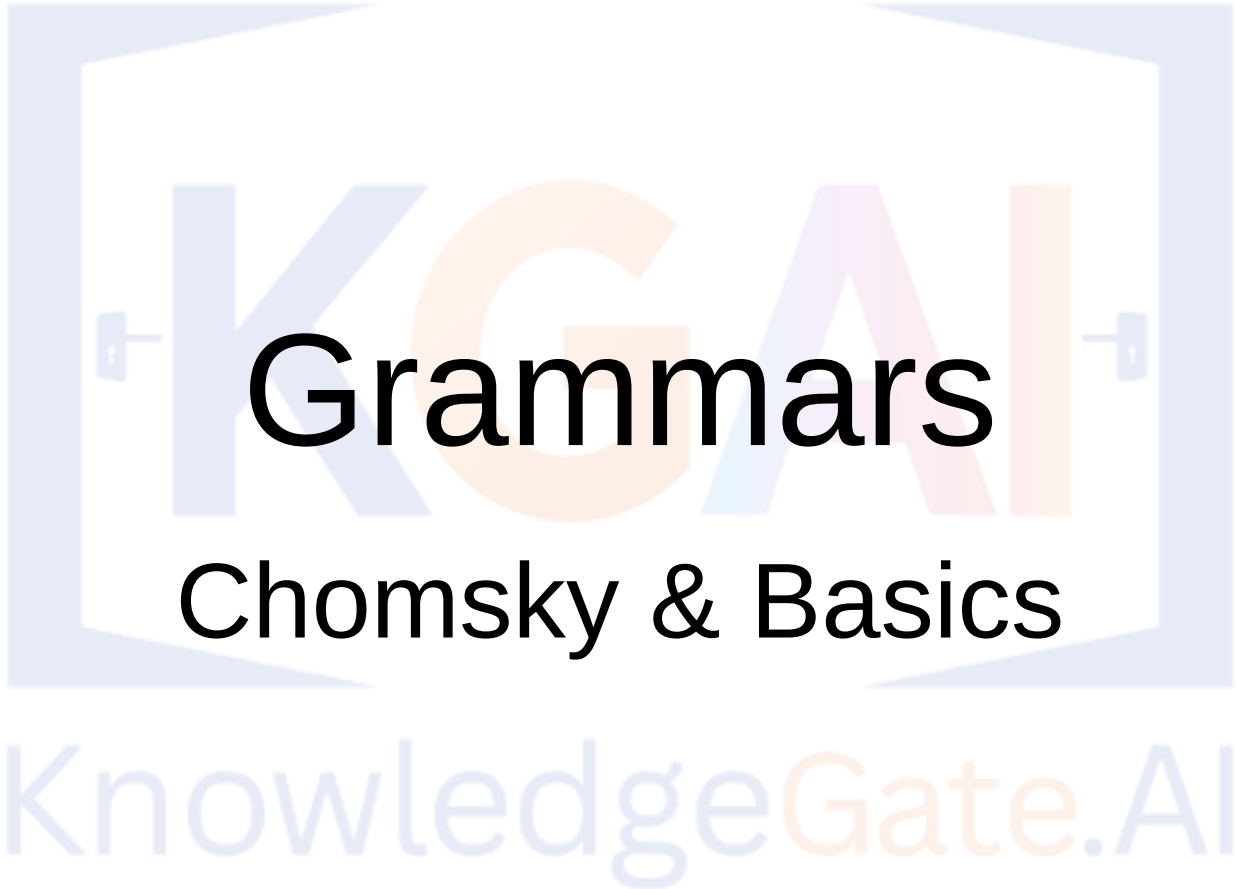
Grammar

Subtopic

Chomsky & Basics

Lesson

Chomsky & Basics

The background features a large, faint watermark of the KnowledgeGate.AI logo. It consists of a light blue square frame with stylized, multi-colored letters 'KGA' inside. Below the frame, the text 'KnowledgeGate.AI' is written in a light blue, sans-serif font.

Grammars

Chomsky & Basics

Formal Grammar

Language usually contains infinite number of strings. We cannot tabulate each and every string to represent the language, therefore like automata, grammar is also a mathematical model of representing a language, using which we can generate the entire language. Therefore, a grammar is usually thought of as a language generator. A phrase-structure grammar (or simply a grammar) is a 4-tuple (V_N, Σ, P, S) , where

- V_N is a finite nonempty set whose elements are called variables,
- Σ is a finite nonempty set whose elements are called terminals, $V_N \cap \Sigma = \Phi$.
- S is a special variable (i.e., an element of V_N ($S \in V_N$)) called the start symbol. Like every automaton has exactly one initial state, similarly every grammar has exactly one start symbol.
- P is a finite set whose elements are $\alpha \rightarrow \beta$. where α and β are strings on $V_N \cup \Sigma$. α has at least one symbol from V_N , the element of P are called productions or production rules or rewriting rules. $\{\Sigma \cup V_N\}^*$ some writer refers it as total alphabet.

For a formal valid production

$$\alpha \sqsubseteq \beta$$

$$\alpha \in \{\Sigma \cup V_n\}^* V_n \{\Sigma \cup V_n\}^*$$

$$\beta \in \{\Sigma \cup V_n\}^*$$



If $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1, S \rightarrow \Lambda\}, S)$, find $L(G)$.

Some points to note about productions

- Reverse substitution is not permitted. For example, if $S \rightarrow AB$ is a production, then we can replace S by AB but we cannot replace AB by S .
- No inversion operation is permitted. For example, if $S \rightarrow AB$ is a production, it is not necessary that $AB \rightarrow S$ is a production.
- To generate a string in the language, one begins with a string consisting of only a single *start symbol*. The *production rules* are then applied in any order, until a string that contains neither the start symbol nor designated *nonterminal symbols* is produced. A sequence of rule applications is called a *derivation*.
- A production rule is applied to a string by replacing one occurrence of the production rule's left-hand side in the string by that production rule's right-hand side.
- $L(G)$ is the set of all terminal strings derived from the start symbol S . G_1 and G_2 are equivalent if $L(G_1) = L(G_2)$.

Defining a language by grammar

- The concept of defining a language using grammar is, starting from a start symbol using the production rules of the grammar any time, deriving the string. Here every time during derivation a production is used as its LHS is replaced by its RHS, all the intermediate stages(strings) are called sentential forms. The language formed by the grammar consists of all distinct strings that can be generated in this manner.

$$L(G) = \{w \mid w \in \Sigma^*, S \xRightarrow{*} w\}$$

- $\xRightarrow{*}$ (reflexive, transitive closure) means from s we can derive w in zero or more steps

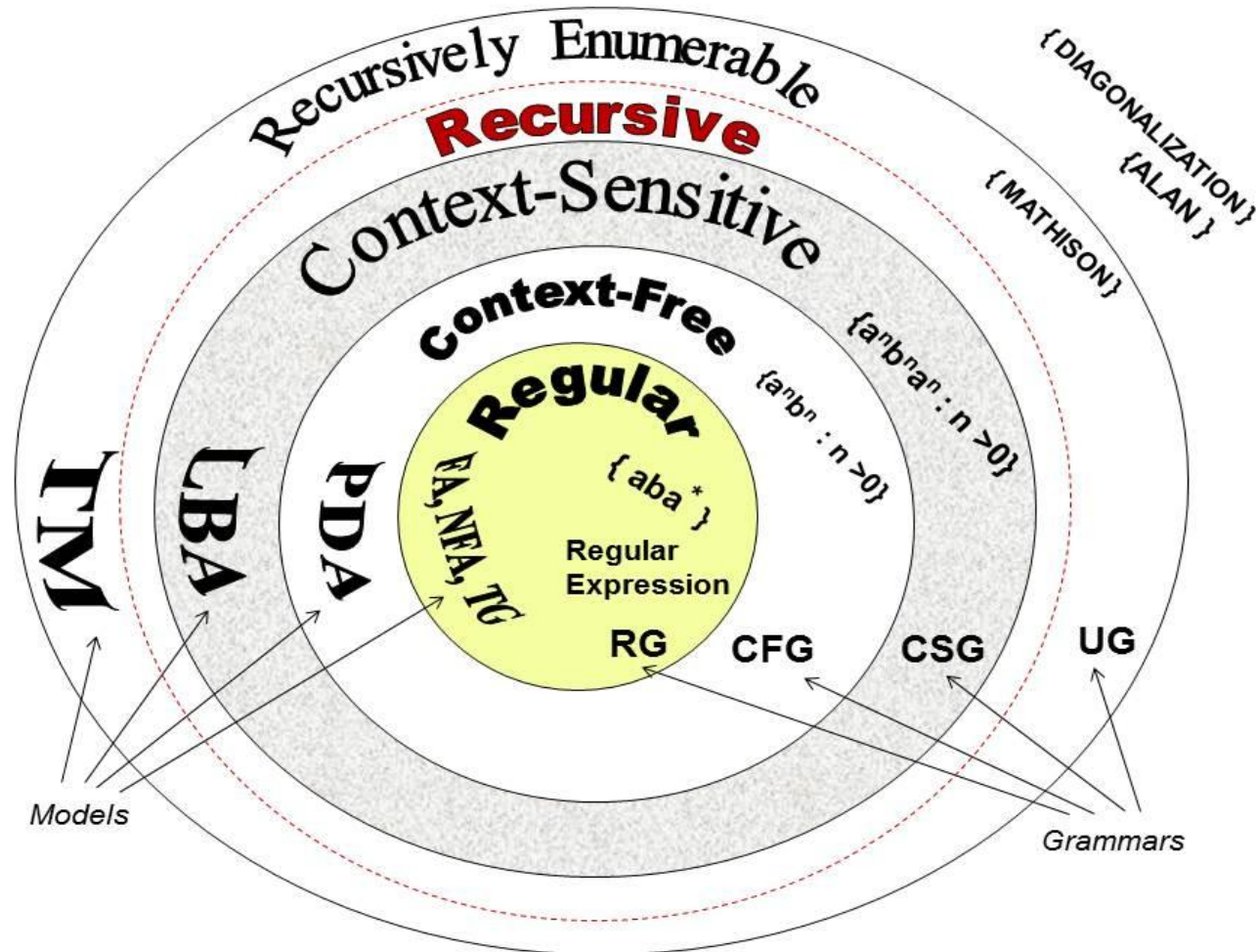
Chomsky Classification of Languages

- Chomsky classified the grammars into four types in terms of productions (types 0-3). This hierarchy of grammars was described by Noam Chomsky in 1956. From type 0 to type 3, we will be putting more and more restrictions. The more restrictive the grammar, the easier the language will be to process; the more liberal the grammar, the more complex the language will become.



LANGUAGES AND AUTOMATA

- Following are the machines that accepts the following grammars.



Type 0 Grammar

- Also known as Unrestricted Grammar, phase structured grammar, recursively enumerable grammar used to generate recursive enumerable language which is accepted by a Turing machine.
- A type 0 grammar is without any restrictions.
- No restriction on the production rule, that is if there is a production from

$$\alpha \rightarrow \beta$$

$$\alpha \in \{\Sigma \cup V_n\}^* V_n \{\Sigma \cup V_n\}^*$$

$$\beta \in \{\Sigma \cup V_n\}^*$$

Type 1 Grammar

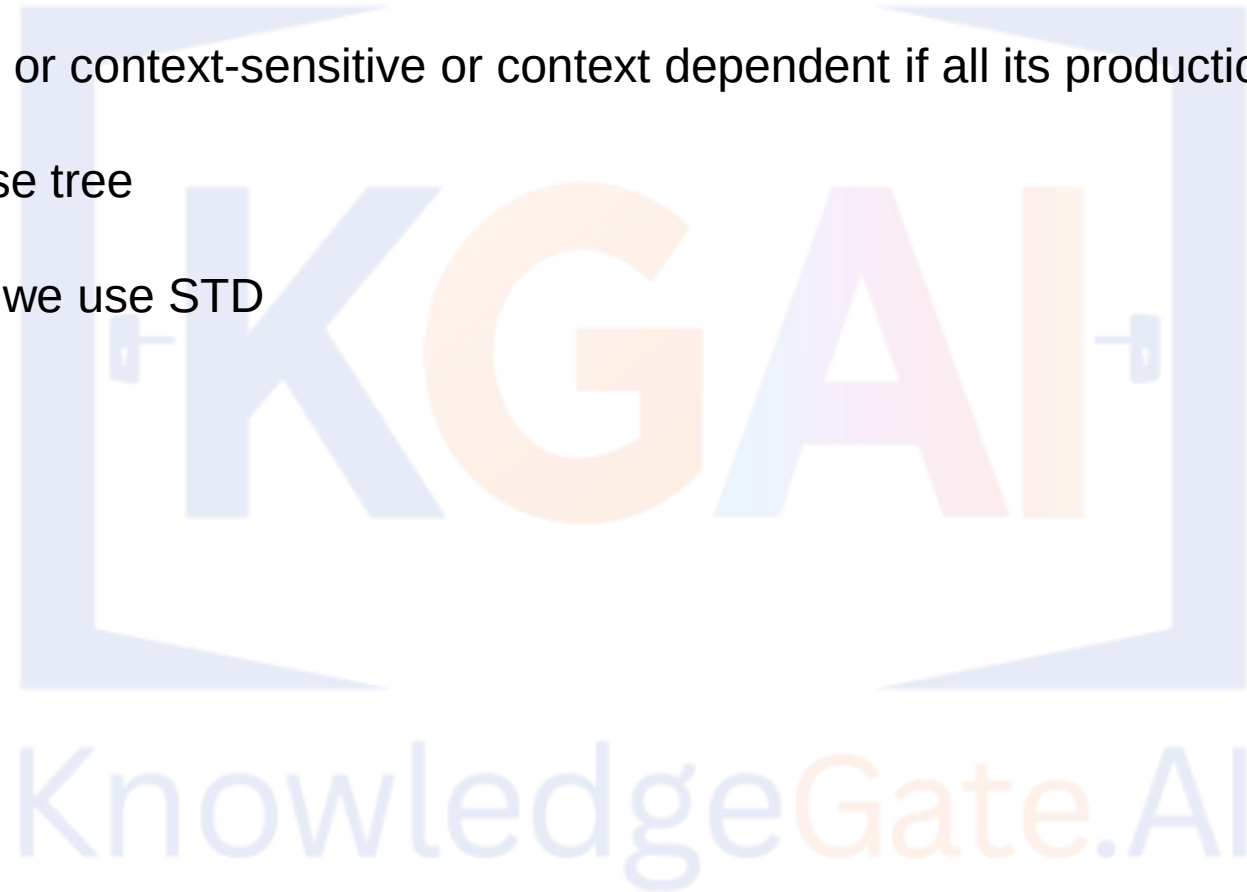
- Also known as case sensitive Grammar, length increasing grammar, non-contracting grammar, used to generate context sensitive language which is accepted by a linear bounded automaton.

$$\alpha A \beta \rightarrow \alpha \delta \beta$$
$$\alpha, \beta \in \{\Sigma \cup V_n\}^* \quad A \in V_n \quad \delta \in \{\Sigma \cup V_n\}^+$$

or

$$\alpha \rightarrow \beta$$
$$\alpha \in \{\Sigma \cup V_n\}^* V_n \{\Sigma \cup V_n\}^*$$
$$\beta \in \{\Sigma \cup V_n\}^+$$
$$|\alpha| \leq |\beta|$$

- As from the rule we can understand that we cannot have null production, in order to solve that problem, Production $S \rightarrow \epsilon$, is allowed if S do not appear on the right-hand side of the production.
- A grammar is called type 1 or context-sensitive or context dependent if all its productions are type 1 productions.
- Very difficult to have a parse tree
- FORTRAN, PL1 with CGF we use STD



Type 2 Grammar

- Also known as Context Free Grammar, which will generate context free language that will be accepted by push down automata. (NPDA default case)
- if there is a production, from
$$\alpha \rightarrow \beta$$
$$\alpha \in V_n \quad |\alpha| = 1$$
$$\beta \in \{\Sigma \cup V_n\}^*$$
- In other words, the L.H.S. has no left context or right context.
- A grammar is called a type 2 grammar if it contains only type 2 productions.
- Eg ALGOL 60, PASCAL

Type 3 Grammar

- Used to generate Regular Grammar which will generate regular language which will be accepted by finite machine.
- Regular grammar can be of two types either left linear or right linear.
- Left regular grammar, support two types of production
$$A \rightarrow a / Ba$$
$$A, B \in V_n \quad |A| = |B| = 1$$
$$a \in \Sigma^*$$
- Right regular grammar
$$A \rightarrow a / aB$$
$$A, B \in V_n \quad |A| = |B| = 1$$
$$a \in \Sigma^*$$
- however, if left-linear rules and right-linear rules are combined, the language need no longer be regular.

Module

Paper-2 | Theory of Computation

Topic

Grammar

Subtopic

Grammar Design via Regex

Lesson

Grammar Design via Regex

The background features a large, faint watermark of the KnowledgeGate.AI logo. It consists of a blue square frame containing the letters 'KGA' in a stylized font. The 'K' is blue, the 'G' is orange, and the 'A' is pink. Below the frame, the text 'KnowledgeGate.AI' is written in a light blue font, with 'Gate' in orange.

Grammars

Grammar Design via Regex

Q Consider the grammar

$S \rightarrow aaaS \mid a \mid aa$

$L(G) = ?$

a) $L(G) = \{w: |w| \bmod 3 = 0\}$

b) $L(G) = \{w: |w| \bmod 3 = 1 \text{ or } 2\}$

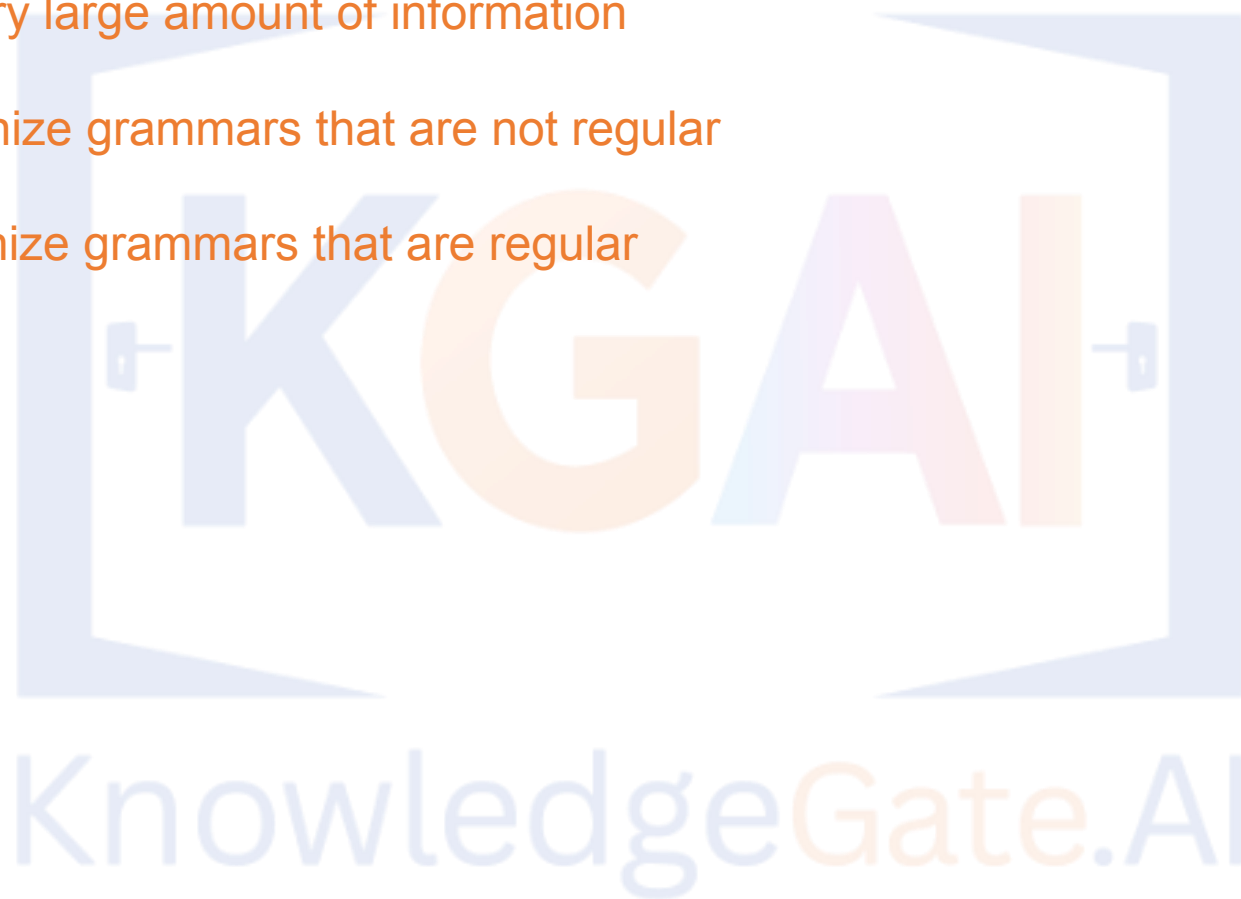
c) $L(G) = L(a^*)$

d) $L(G) = L(a^*) - (\lambda)$



Q The basic limitation of FSM is that

- a)** It cannot remember arbitrary large amount of information
- b)** It sometimes fails to recognize grammars that are not regular
- c)** It sometimes fails to recognize grammars that are regular
- d)** All of these



Q Consider the following grammar and identify its language?

$S \rightarrow aAb$

$A \rightarrow aB / b$

$B \rightarrow c$



Q Consider the following grammar and identify it's language?

$S \rightarrow AB / Bb$

$A \rightarrow b / c$

$B \rightarrow d$



Q Consider the following grammar and identify it's language?

$S \rightarrow aSb \mid \epsilon$



Q Consider the following grammar and identify its language?

$S \rightarrow aA / abS$

$A \rightarrow bS / b$



Q Consider the following grammar and identify it's language?

$S \rightarrow aAB$

$A \rightarrow aA / \epsilon$

$B \rightarrow b$



Q Consider the following grammar and identify it's language?

$S \rightarrow AB$

$A \rightarrow aA / \epsilon$

$B \rightarrow bB / b$



Q Consider the following grammar and identify it's language?

$S \rightarrow aSa / bSb / \epsilon$



Q What is the regular expression for the language generated by

$S \rightarrow aS \mid bA$

$A \rightarrow d \mid ccA$

a) a^*bd

b) $a^* (bd)(bcc)^* d$

c) $a^* b(cc)^* d$

d) None of these



Q Write the grammar for the regular expression a^*b^*

a) $S \rightarrow AB, A \rightarrow aA \mid bB \mid \epsilon, B \rightarrow bB \mid aA \mid \epsilon$

b) $S \rightarrow AB, A \rightarrow aA \mid \epsilon, B \rightarrow bB \mid \epsilon$

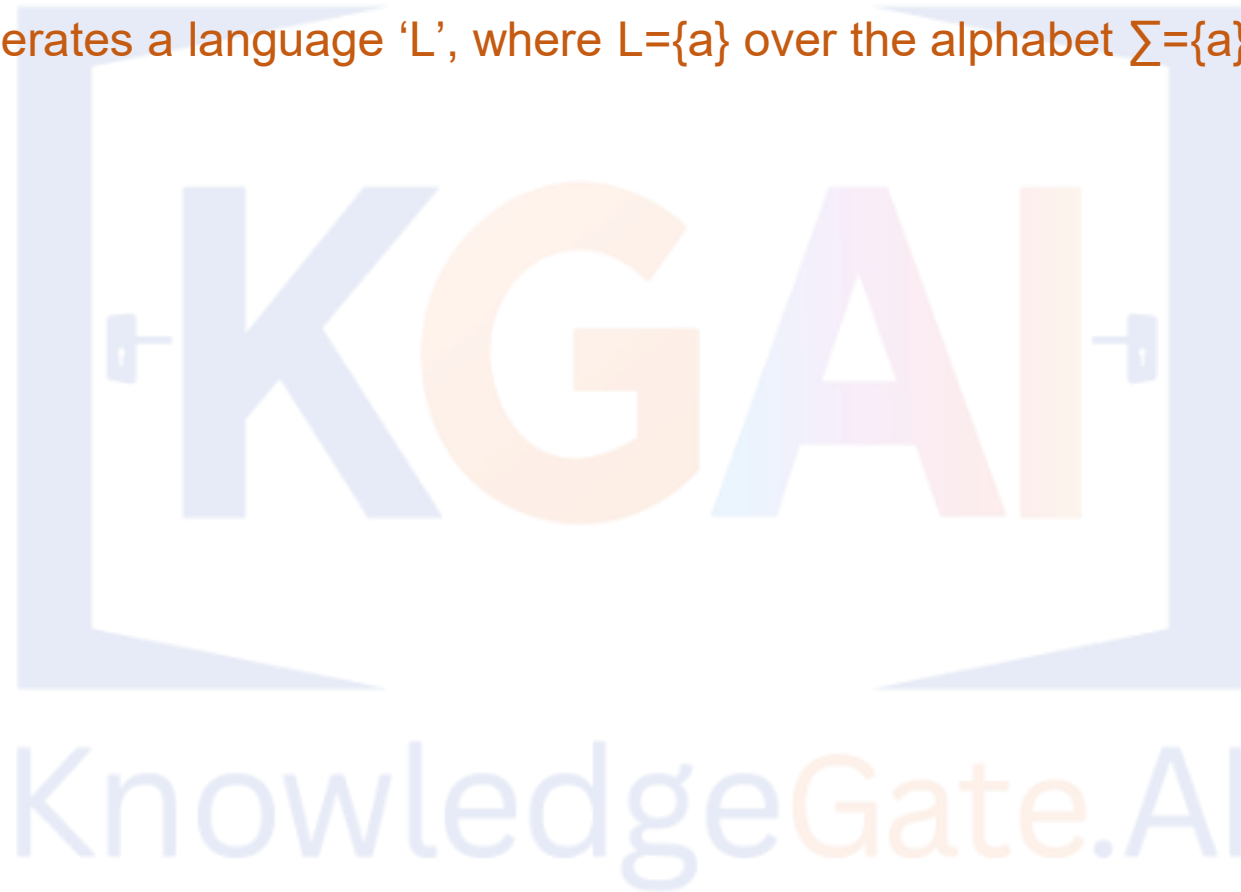
c) $S \rightarrow ab \mid \epsilon, A \rightarrow aA \mid \epsilon, B \rightarrow bB \mid \epsilon$

d) None of these



Designing Grammar

Q Design a grammar that generates a language 'L', where $L=\{a\}$ over the alphabet $\Sigma=\{a\}$.



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$, where every accepted string 'w' starts with substring s

i) s = b

ii) s = ab

iii) s = abb



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$. where every accepted string 'w' ends with substring 's'.

i) s = ab

ii) s = aa

iii) s = bab



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$. where every accepted string 'w' contains substring s.

i) abb

ii) aba



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with a.



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with same symbol.



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with different symbol.



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=SX$.

i) $s= aa/bb$

ii) $s=aaa/bbb$



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=XS$.

i) $s = aa/bb$

ii) $s = aaa/bbb$



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w , is like $w=XSX$.

i) $s = aa/bb$

ii) $s = aaa/bbb$



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' accepted must be like

i) $|w| = 3$

ii) $|w| \leq 3$

iii) $|w| \geq 3$



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain exactly two a's.



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain at least two a's.



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$, such that every string accepted must contain at most two a's.



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$ such that for every accepted string 2nd from left end is always b.



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' where $|W| = 0 \pmod{3}$?



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' where $|W| = 3(\text{mod } 4)$?



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' where $|W|_a = 0 \pmod{3}$?



Q Design a grammar that generates all strings over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' where $|W|_a = 2(\text{mod } 3)$?



- We actually did Regular Expression to Grammar
- Regular Grammar to Regular Expression

Q Consider the following Left recursive grammar and convert them into Regular Expression?

$A \rightarrow A\alpha / \beta$

$A \rightarrow A\alpha / \beta_1 / \beta_2$

$A \rightarrow A\alpha / \beta_1 / \beta_2 \text{ -----} / \beta_n$



$A \rightarrow A\alpha_1 / A\alpha_2 / \beta$

$A \rightarrow A\alpha_1 / A\alpha_2 / \text{-----}A\alpha_n / \beta$



$A \rightarrow A\alpha_1 / A\alpha_2 / \dots A\alpha_n / \beta_1 / \beta_2 \dots / \beta_m$



www.knowledgetate.ai

Module

Paper-2 | Theory of Computation

Topic

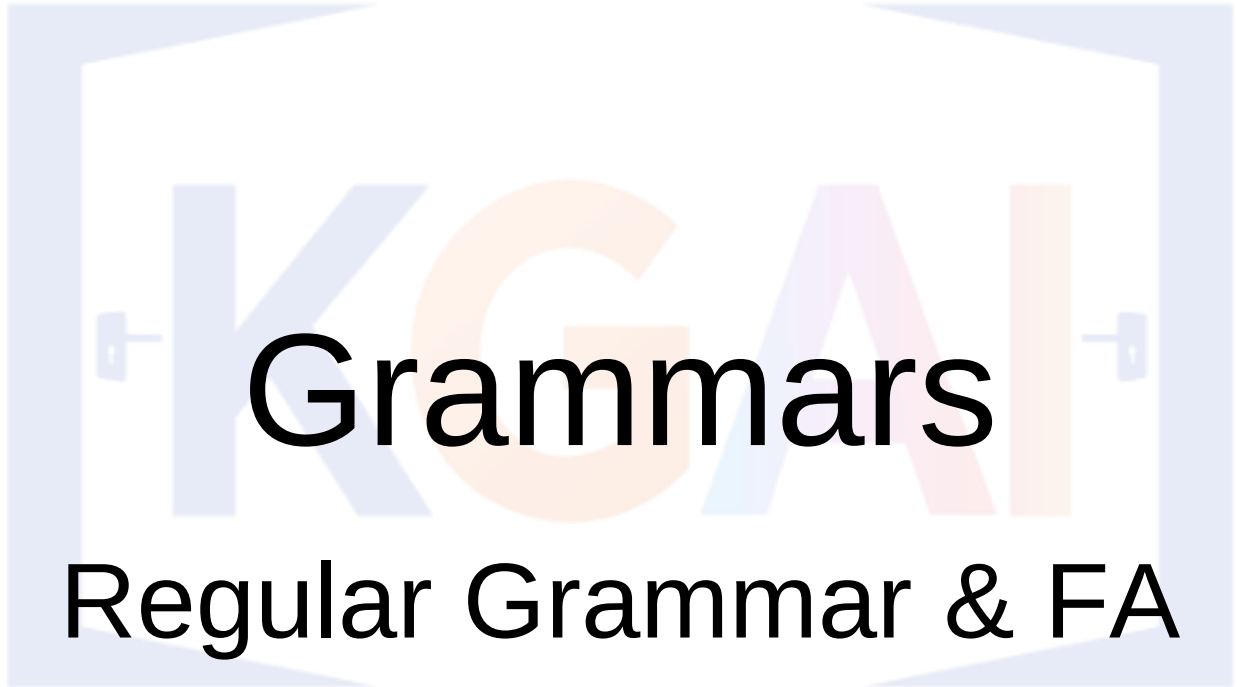
Grammar

Subtopic

Regular Grammar & FA

Lesson

Regular Grammar & FA

The logo for KnowledgeGate.AI is centered in the background. It features the letters 'KGA' in a large, stylized font. The 'K' is blue, the 'G' is orange, and the 'A' is purple. These letters are enclosed within a light blue square frame that has a 3D effect, with the top and bottom edges appearing to be slightly offset. The word 'KnowledgeGate.AI' is written in a smaller, sans-serif font below the 'KGA' letters, with 'Knowledge' in blue, 'Gate' in orange, and '.AI' in blue.

Grammars

Regular Grammar & FA

KnowledgeGate.AI

S -> 01S / 01



www.knowledgegate.ai

S -> 0011S / 01 / 10



www.knowledgegate.ai

S -> 01A / B11
A -> 011A / 01
B -> 101B / 11



www.knowledgegate.ai

S -> 011A / 101B

A -> 110A / 00

B -> 11B / S



Regular Grammar to (Finite Automata)

www.knowledgegate.ai

S -> 01S / 1



www.knowledgegate.ai

S -> 011S / 01



www.knowledgetate.ai

S -> 001S / 10A
A -> 101A / 0 / 1



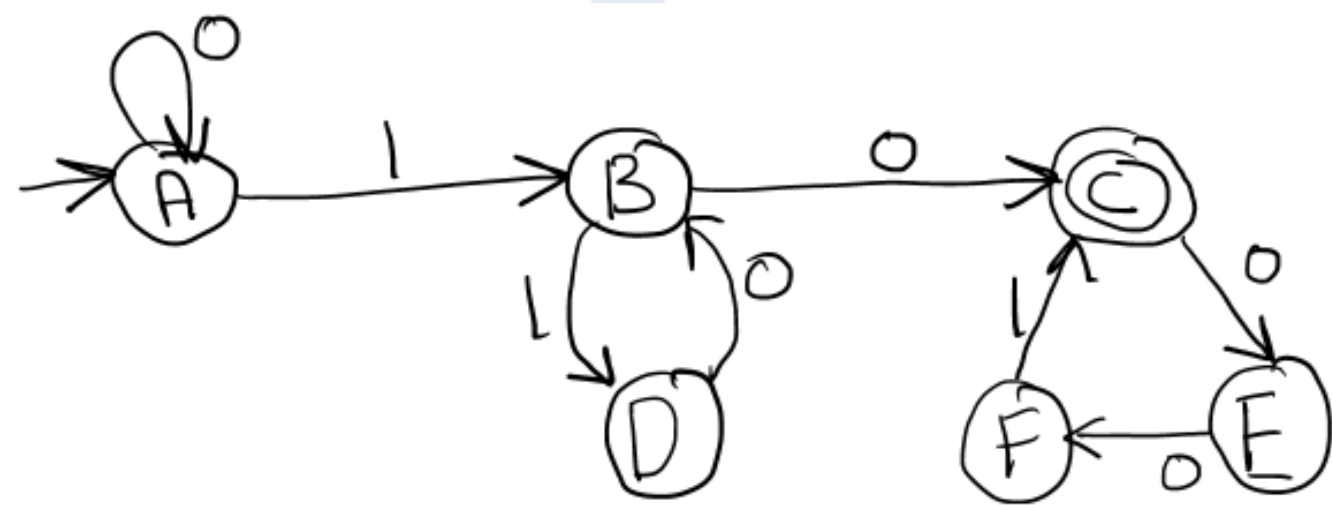
- We actually did Right Regular Grammar to (Finite Automata)
- Left Regular Grammar to (Finite Automata)

- Reverse the right hand side of every production.
- Construct the NFA.
- Interchange initial and final state.
- Change the direction of edges.

S -> S10 / 01



- Finite Automata to Regular Grammar



- We actually did (Finite Automata) To Right Regular Grammar
- Finite Automata to Left Regular Grammar

- Obtain the regular Expression
- Reverse the regular Expression
- Construct the finite automata
- Construct the right regular grammar
- Reverse right hand side of every production



- Every Left regular grammar can be converted into right regular and every right regular grammar can be converted into left regular grammar.
- **Procedure to convert Right linear grammar to Left linear grammar**
 - Obtain the RE from the grammar
 - Reverse the RE
 - Construct the finite automata for the regular expression
 - Now construct the right linear grammar
 - Reverse the left hand side of every production to get a left linear grammar.

KnowledgeGate.AI

Module

Paper-2 | Theory of Computation

Topic

Regular Language Properties

Subtopic

Decidability & Basics

Lesson

Decidability & Basics

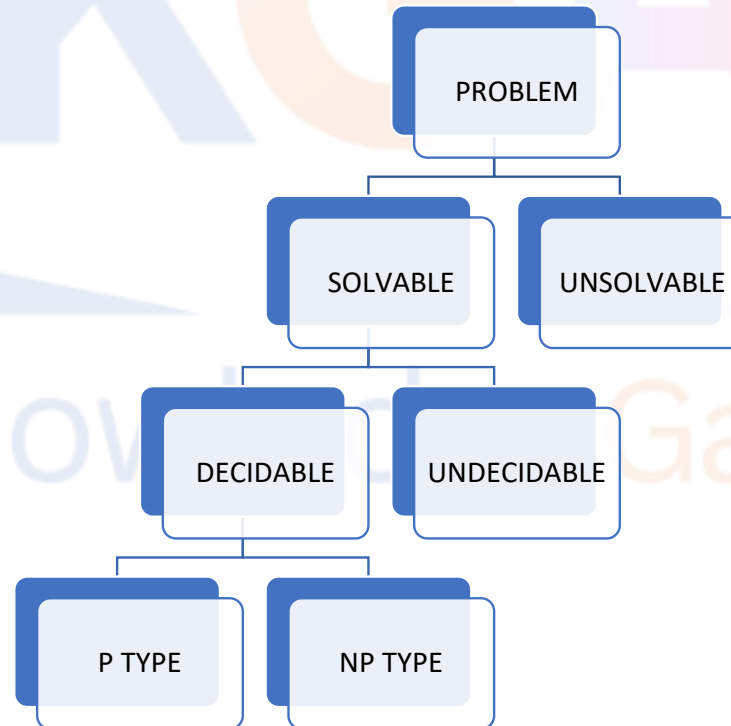
Regular Language Properties

Decidability & Basics

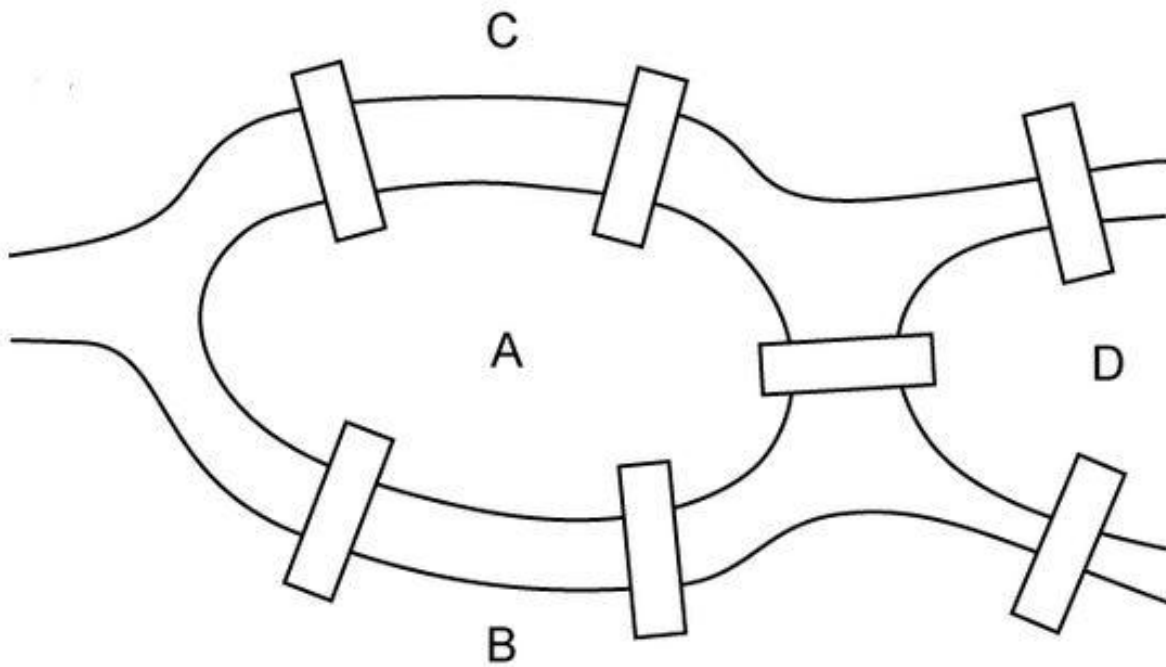
KnowledgeGate.AI

What computer science deals with?

- Don't
 - So we do not study how to design a computer
 - We do not study how to run a computer
- Do
 - We deal with problem solving, according to computer science a problem can be divided as follows



Konigsberg Bridge Problem



KnowledgeGate.AI

- **SOLVABLE** - A problem is said to be solvable if either we can solve it or if we can prove that the problem cannot be solved.
- **UNSOLVABLE** - A problem is said to be unsolvable if neither we can solve it, nor we can prove that the problem can not be solved.
- **P**- if there exist a polynomial time algorithm to solve a problem then problem is said to be decidable.
- **NP**- if there exist a non- polynomial time algo to solve a problem.

KnowledgeGate.AI

Membership

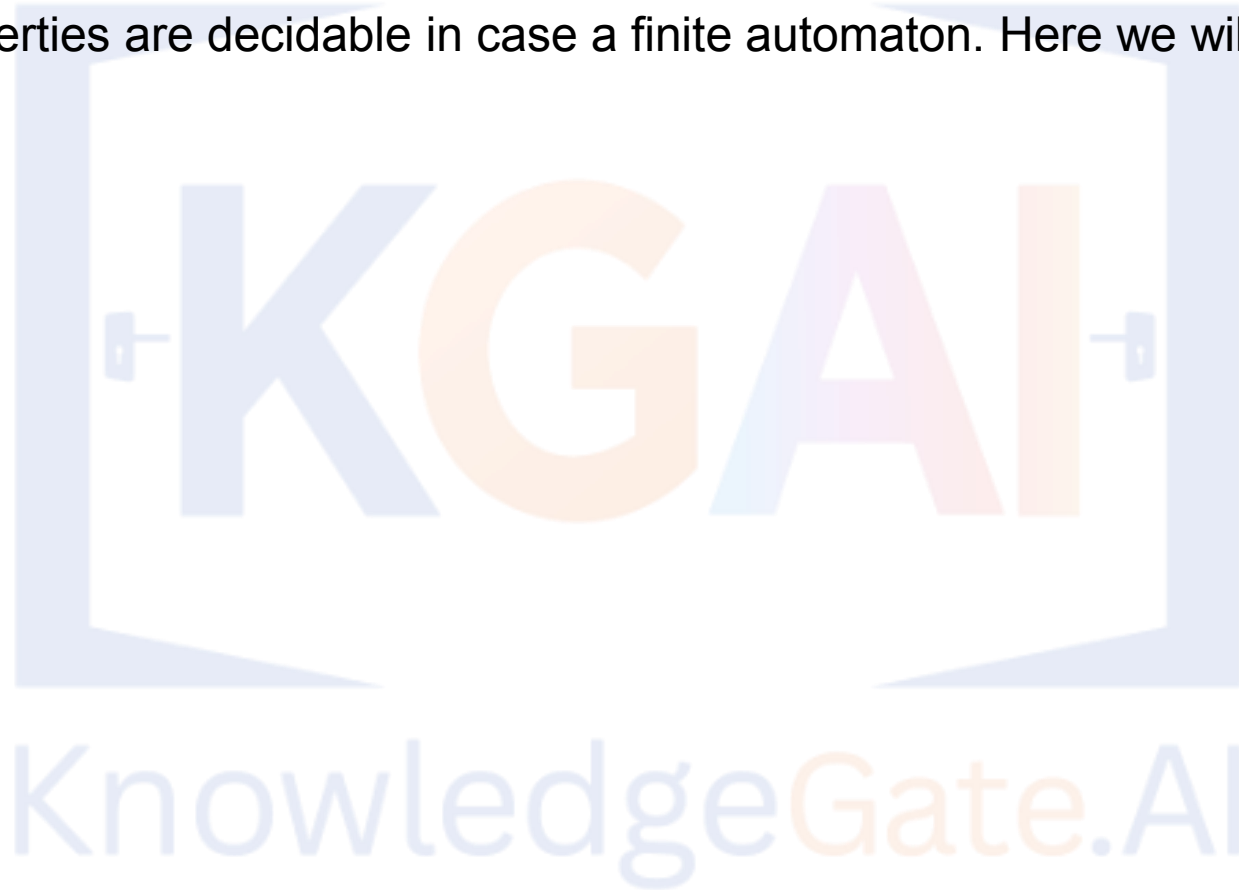
- Membership is a property to verify an arbitrary string is accepted by a finite automaton or not i.e. it is a member of the language or not.
- Let M is a finite automata that accepts some strings over an alphabet, and let 'w' be any string defined over the alphabet, if there exist a transition path in M , which starts at initial state & ends in anyone of the final state, then string 'w' is a member of M , otherwise 'w' is not a member of M .

KnowledgeGate.AI

Decision properties

- Approximately all the properties are decidable in case a finite automaton. Here we will use machine model to prove decision properties.

- i) Emptiness
- ii) Non-emptiness
- iii) Finiteness
- iv) Infiniteness
- v) Membership
- vi) Equality



Emptiness & Non-emptiness

- Step 1: - select the state that cannot be reached from the initial states & delete them (remove unreachable states)
- Step 2: - if the resulting machine contains at least one final states, so then the finite automata accepts the non-empty language.
- Step 3: - if the resulting machine is free from final state, then finite automata accepts empty language.

KnowledgeGate.AI

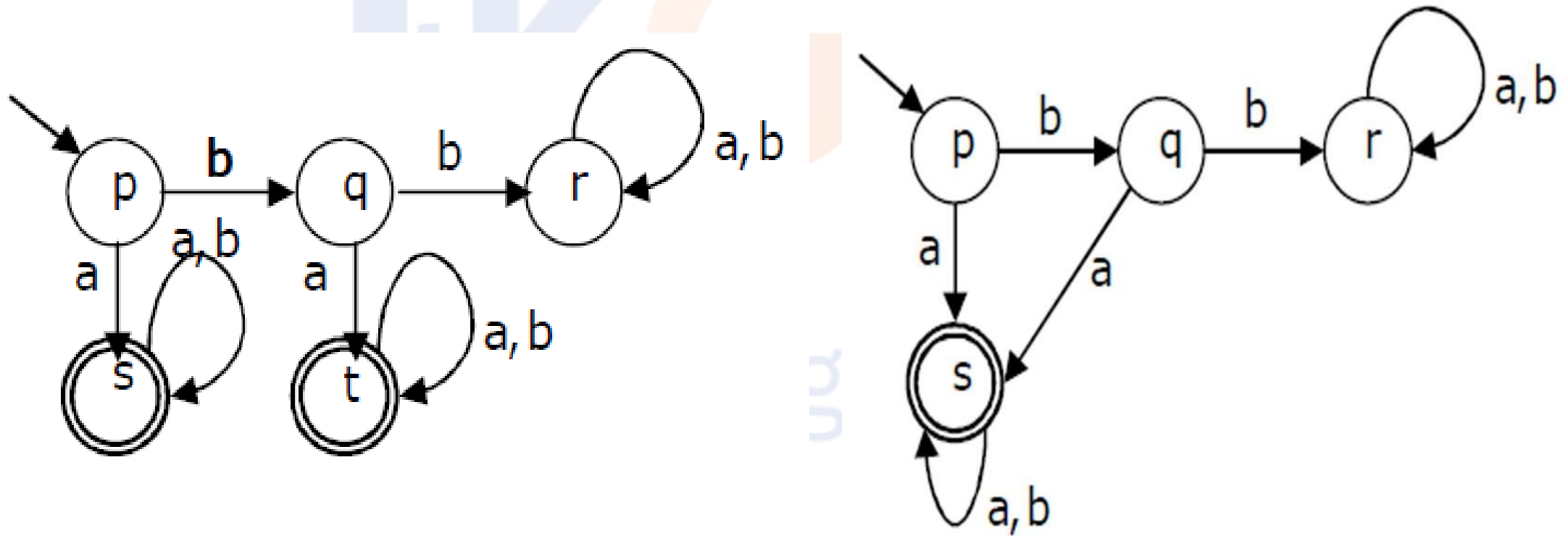
Finiteness & Infiniteness

- Step 1: - Select the state that cannot be reached from the initial state & delete them (remove unreachable states)
- Step 2: - Select the state from which we cannot reach the final state & delete them (remove dead states)
- Step 3: - If the resulting machine contains loops or cycles then the finite automata accepts infinite language
- Step 4: - If the resulting machine do not contain loops or cycles then the finite automata accepts finite language.

KnowledgeGate.AI

Equality

- Two finite state automata M_1 & M_2 is said to be equal if and only if, they accept the same language.
- Minimise the finite state automata and the minimal DFA will be unique.



	RL	DCFL	CFL	CSL	RS	RES
Emptiness	Y	Y	Y	X	N	N
Non-Emptiness	Y	Y	Y	X	N	N
Finiteness	Y	Y	Y	X	N	N
Infiniteness	Y	Y	Y	X	N	N
Membership	Y	Y	Y	X	Y	N
Equality	Y	N	N	X	N	N
Ambiguity	Y	N	N	X	N	N
Σ^*	Y	N	N	X	N	N
Halting	Y	Y	Y	X	Y	N

Module

Paper-2 | Theory of Computation

Topic

Regular Language Properties

Subtopic

Closure Properties

Lesson

Closure Properties

Regular Language Properties

Closure Properties

KnowledgeGate.AI

Closure Properties of Regular Languages

- Regular languages are closed under following operations
 - Kleene Closure
 - Positive closure
 - Complement
 - Reverse Operator
 - Prefix Operator
 - Suffix operator
 - Concatenation
 - Union
 - Intersection
 - Set Difference operator
 - Symmetric Difference



	RL	DCFL	CFL	CSL	RS	RES
Union	Y	N	Y	Y	Y	Y
Intersection	Y	N	N	Y	Y	Y
Complement	Y	Y	N	Y	Y	N
Set Difference	Y	N	N	Y	Y	N
Kleene Closure	Y	N	Y	Y	Y	Y
Positive Closure	Y	N	Y	Y	Y	Y
Concatenation	Y	N	Y	Y	Y	Y
Intersection with regular set	Y	Y	Y	Y	Y	Y
Reverse	Y	Y	Y	Y	Y	Y
Subset	N	N	N	N	N	N

	RL	DCFL	CFL	CSL	RS	RES
Homomorphism	Y	N	Y	N	N	Y
∈ Free Homomorphism	Y	N	Y	Y	Y	Y
Inverse Homomorphism	Y	Y	Y	Y	Y	Y
Substitution	Y	N	Y	N	N	Y
∈ Free Substitution	Y	N	Y	Y	Y	Y
Quotient with regular set	Y	Y	Y	N	Y	Y

Module

Paper-2 | Theory of Computation

Topic

Regular Language Properties

Subtopic

Pumping Lemma

Lesson

Pumping Lemma

Regular Language Properties

Pumping Lemma

KnowledgeGate.AI

Pumping Lemma For Regular Languages

- Pumping Lemma is used as a proof for irregularity of a language. Thus, if a language is regular, it always satisfies pumping lemma. If there exists at least one string made from pumping which is not in L , then L is surely not regular.
- The opposite of this may not always be true. That is, if Pumping Lemma holds, it does not mean that the language is regular.
- Pumping Lemma is used to prove that some of the language is non-regular.
- For the pumping lemma, i/p is NRL & o/p is also NRL.



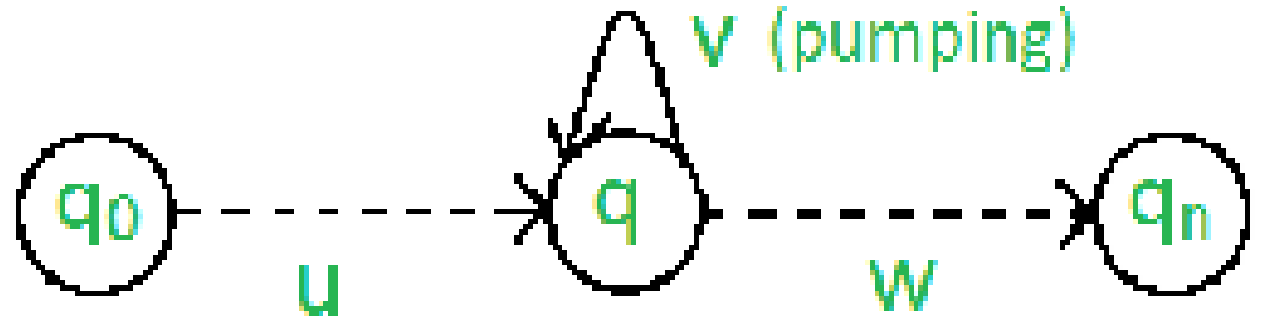
Pumping Lemma

www.knowledgegate.ai

Pumping Lemma For Regular Languages

For any regular language L , there exists an integer n , such that for all $z \in L$ with $|z| \geq n$, there exists $u, v, w \in \Sigma^*$, such that $z = uvw$, and

- o $|uv| \leq n$
- o $|v| \geq 1$
- o for all $i \geq 0$: $uv^i w \in L$



In simple terms, this means that if a string v is 'pumped', i.e., if v is inserted any number of times, the resultant string still remains in L .

Process of pumping Lemma

- Suppose that we need to prove that language L is a non-regular.
- Assume that L is a regular language, then L must satisfy pumping lemma property.
- Choose $z \in L$ such that $|z| \leq n$, split z into 3 parts.
 - o $|uv| \leq n$
 - o $|v| \geq 1$
- o If there exist at least one variable for i such that $uv^i w \notin L$, then L does not satisfy pumping lemma property so it is a contradiction, therefore language L is non-regular.

Q Proof that language $L = \{a^m b^n \mid m=n\}$ is non-regular?

$L = \{ab, aabb, aaabbb, aaaabbbb, \dots\}$

$z \in L$

$Z = a^k b^k$

For $i=1 \quad \square \quad uv^i w \quad \square \quad a^k b^k$

For $i=2 \quad \square \quad uv^i w \quad \square \quad a^{k+1} b^k \notin L$

There for L is not regular



Q Proof that language $L = \{a^m b^n \mid m < n\}$ is non-regular?



Q Proof that language $L = \{a^n b^n \mid n \text{ is a prime number}\}$?



Q Proof that language $L = \{a^{n^2} \mid n \geq 0\}$?



Module

Paper-2 | Theory of Computation

Topic

Moore & Mealy Machines

Subtopic

Basics & Moore Machine

Lesson

Basics & Moore Machine

Moore & Mealy Machines

Basics & Moore Machine

KnowledgeGate.AI

Moore and Mealy Machine

- Both moore and mealy machine are special case of DFA
- Both acts like o/p producers rather than language acceptors
- In moore and mealy machine no need to define the final states
- No concepts of dead states and no concepts of final states
- Mealy and Moore Machines are equivalent in power.



George H Mealy



Edward F Moore

Moore Machine

A Moore machine is a six-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where

- Q is a finite set of states:
- Σ is the input alphabet:
- Δ is the output alphabet.
- δ is the transition function $Q \times \Sigma$ into Q
- λ is the output function mapping Q into Δ and
- q_0 is the initial state.

KnowledgeGate.AI

Examples: The below table shows the transition table of a Moore Machine.

Present state	Next state δ		Output λ
	$a = 0$	$a = 1$	
$\rightarrow q_0$	q_3	q_1	0
q_1	q_1	q_2	1
q_2	q_2	q_3	0
q_3	q_3	q_0	0

- In moore machine for every state output is associated. If the length of i/p string is n , then length of o/p string will be $n+1$. Moore machine response for empty string ϵ .

Q construct a Moore machine take all the string of a's and b's as i/p and counts the no of a's in the i/p string in terms of 1, $\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$?



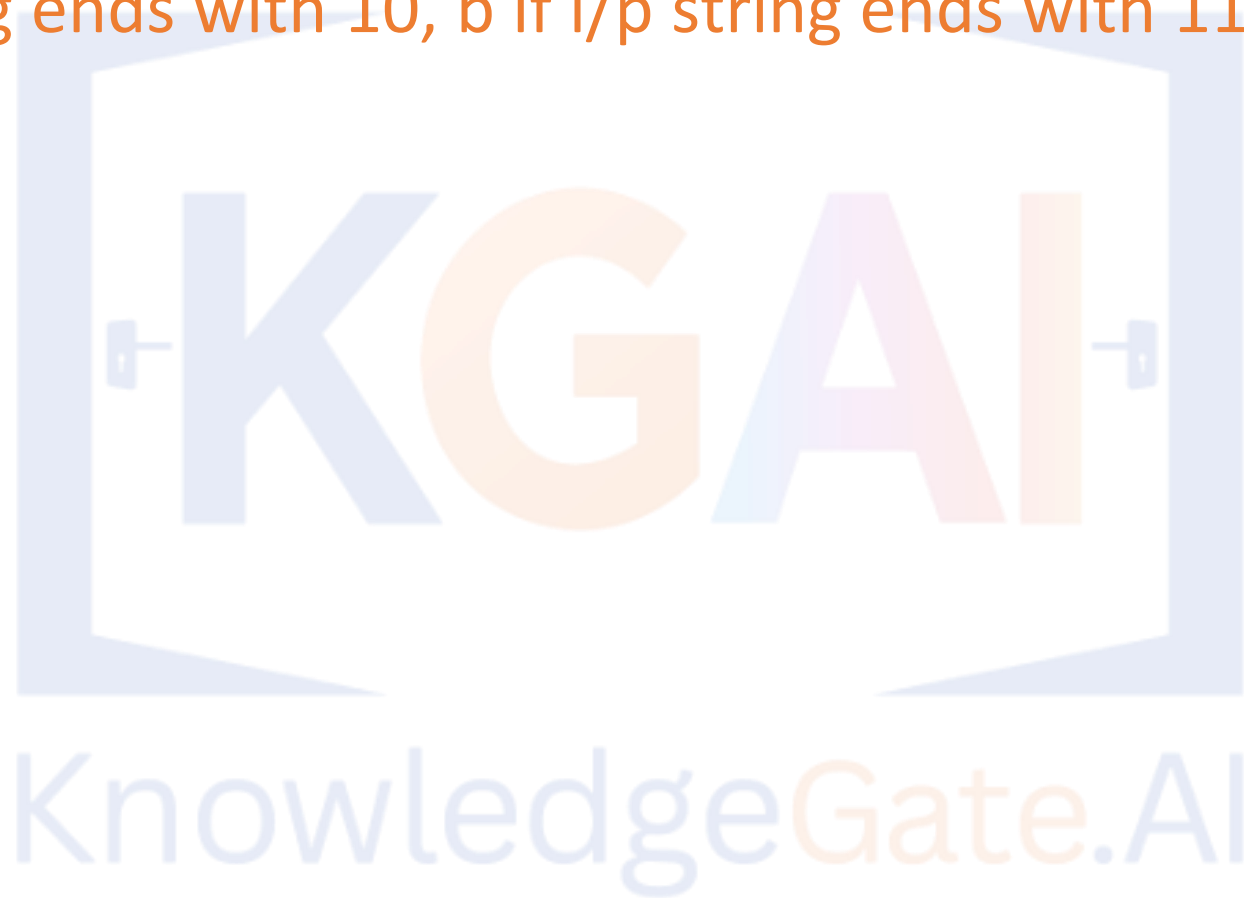
Q construct a Moore machine take all the string of a's and b's as i/p and counts the no of occurrence of sub-string 'ab' in terms of 1, $\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$?



Q construct a Moore machine take all the string of a's and b's as i/p and counts the no of occurrence of sub-string 'aa' in terms of 1, $\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$?



Q construct a Moore machine where $\Sigma = \{0, 1\}$, $\Delta = \{a, b, c\}$, machine should give o/p a, if the i/p string ends with 10, b if i/p string ends with 11, c otherwise?



Module

Paper-2 | Theory of Computation

Topic

Moore & Mealy Machines

Subtopic

Mealy Machine

Lesson

Mealy Machine

Moore & Mealy Machines

Mealy Machine

KnowledgeGate.AI

Mealy Machine

- Mealy machine is a six-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where all the symbols except λ have the same meaning as in the Moore machine. λ is the output function mapping $Q \times \Sigma$ into Δ .
- In case of mealy machine, the output symbol depends on the transition.

KnowledgeGate.AI

Example: The below table shows the transition table of a Mealy Machine.

Present state	Next state			
	a = 0		a = 1	
	state	output	state	output
→ q ₁	q ₃	0	q ₂	0
q ₂	q ₁	1	q ₄	0
q ₃	q ₂	1	q ₁	1
q ₄	q ₄	1	q ₃	0

KnowledgeGate.AI

- If the length of i/p string is n, then length of o/p string will be n. Mealy machine do not response for empty string ϵ .

Q construct a Mealy machine take all the string of a's and b's as i/p and counts the no of a's in the i/p string in terms of 1, $\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$?



Q construct a Mealy machine take all the string of a's and b's as i/p and counts the no of occurrence of sub-string 'ab' in terms of 1, $\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$?



Q construct a Mealy machine take all the string of a's and b's as i/p and counts the no of occurrence of sub-string 'aa' in terms of 1, $\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$?



Q construct a Mealy machine where $\Sigma = \{0, 1\}$, $\Delta = \{a, b, c\}$, machine should give o/p a, if the i/p string ends with 10, b if i/p string ends with 11, c otherwise?



Module

Paper-2 | Theory of Computation

Topic

Moore & Mealy Machines

Subtopic

Conversion & Equivalence

Lesson

Conversion & Equivalence

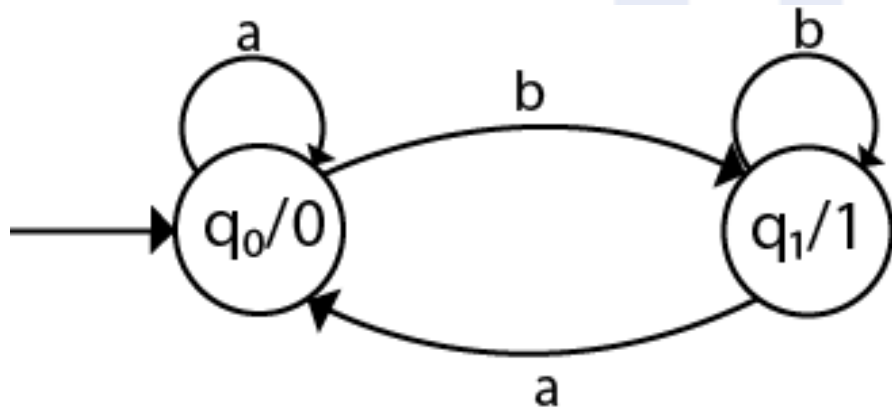
Moore & Mealy Machines

Conversion & Equivalence

KnowledgeGate.AI

CONVERSION OF MOORE TO MEALY MACHINE

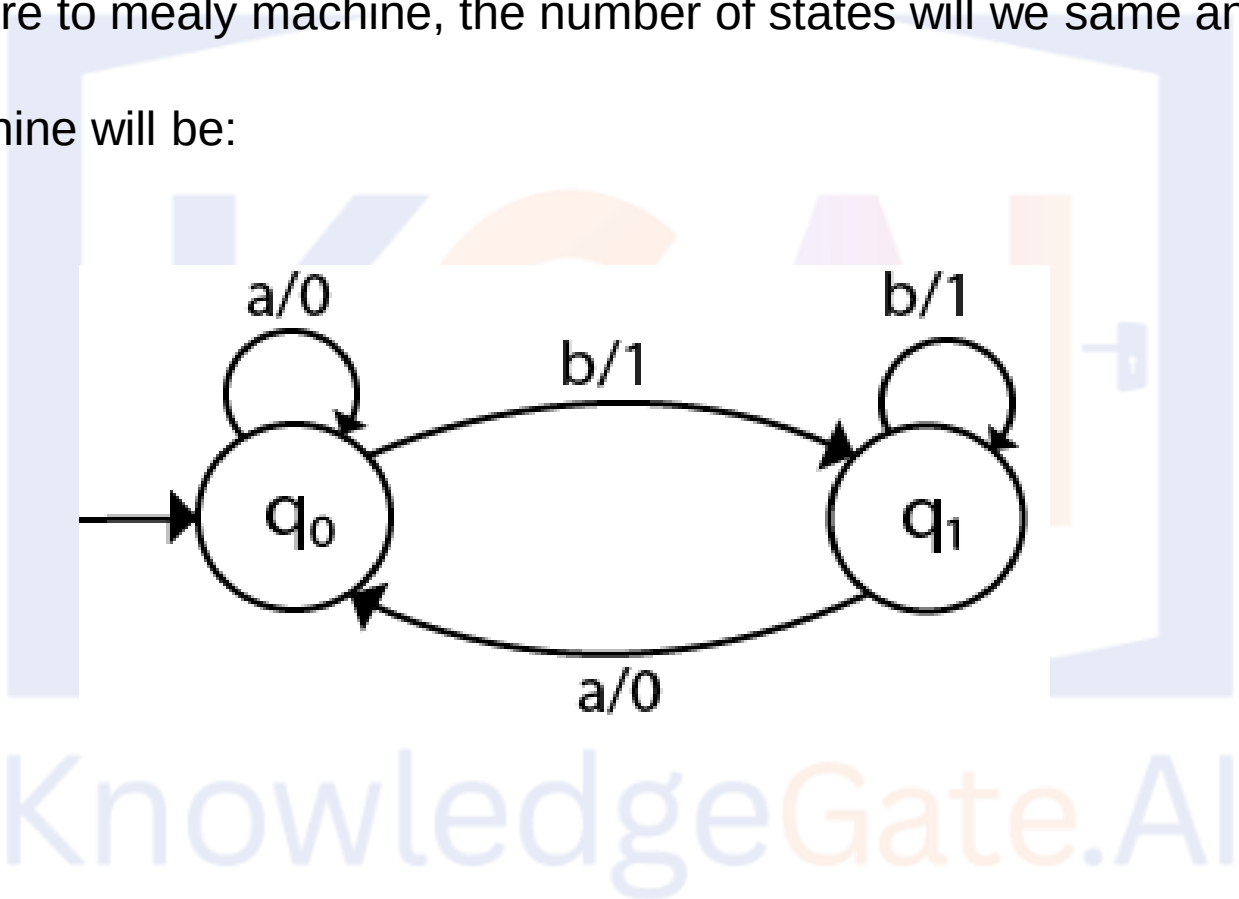
- Let us take an example to understand the conversion:
- Convert the following Moore machine into its equivalent Mealy machine.



Q	a	b	Output(λ)
q0	q0	q1	0
q1	q0	q1	1

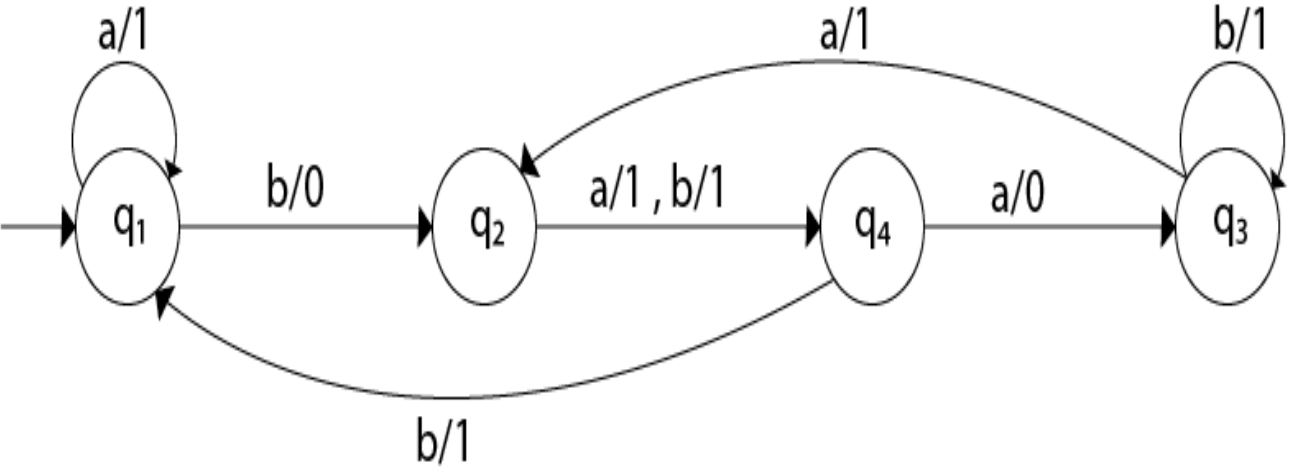
KnowledgeGate.AI

- To convert a mealy machine to moore machine all you need to do is just push out the outputs of states onto to the incoming transitions.
- While conversion from moore to mealy machine, the number of states will be same and there will be no extra states.
- The equivalent Moore Machine will be:



PROCEDURE FOR TRANSFORMING A MEALY MACHINE INTO A MOORE MACHINE

Consider the Mealy Machine:



Present State	Next State			
	a		b	
	State	O/P	State	O/P
q ₁	q ₁	1	q ₂	0
q ₂	q ₄	1	q ₄	1
q ₃	q ₂	1	q ₃	1
q ₄	q ₃	0	q ₁	1

Module

Paper-2 | Theory of Computation

Topic

Pushdown Automata & CFG

Subtopic

PDA Design & Basics

Lesson

PDA Design & Basics

Pushdown Automata & CFG

PDA Design & Basics

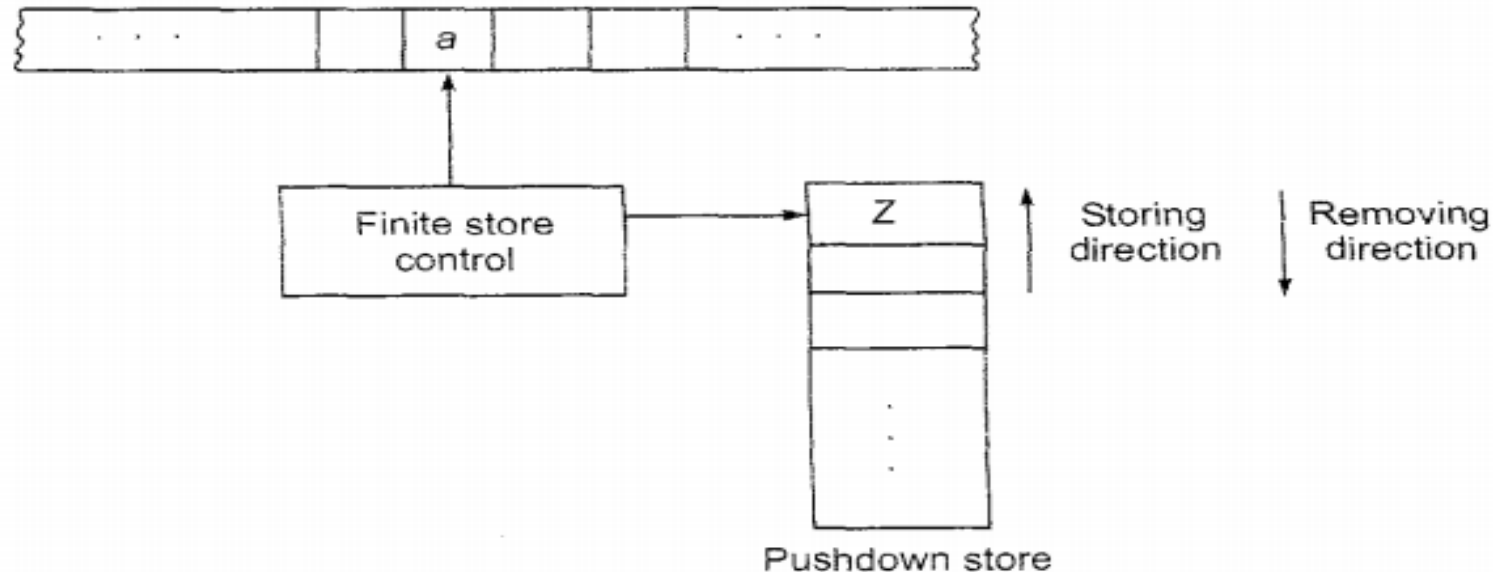
KnowledgeGate.AI

CONTEXT-FREE LANGUAGES AND PUSH DOWN AUTOMATA

- We Already understand the limitation of finite automata, that it cannot do the infinite comparison between the symbols.
- Let us consider $L = \{a^n b^n \mid n \geq 1\}$. This is not regular, as it has to remember the number of a's in a string and so it will require an infinite number of states, which is logically not possible.
- This difficulty can be avoided by adding an auxiliary memory in the form of a 'stack'. The reason we choose stack because it is the simplest memory possible.
- This type of arrangement where ***a finite automaton has a stack leads to the generation of a pushdown automaton.***

BLOCK DIAGRAM OF PDA

- Finite control unit is also called as memory unit it is static and limited. So to process the i/p string if the static memory is not sufficient then we can use the stack.
- i/p tape is divided into cells where is cell is capable of holding one symbol at a time. At stack of infinite size, which support three operations push, pop and skip.
- The accepting power of a pda is more than that of finite automata and less than that of linear bounded automata
- The power of non-deterministic pda is more than the power of deterministic pda.



Formal Definition of DPDA

A DPDA is a 7-tuple, namely $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where

- (i) Q – is a finite nonempty set of states,
- (ii) Σ – is a finite nonempty set of input symbols,
- (iii) Γ – is a finite nonempty set of pushdown symbols,
- (iv) q_0 – is a special state called the initial state,
- (v) Z_0 – is a special pushdown symbol called the initial symbol on the pushdown store.
- (vi) F – is a set of final states, a subset of Q and
- (vii) δ – is a transition function from $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma$ to the set of finite subsets of $Q \times \Gamma^*$.

Representation of States

(1) **PUSH** – one symbol can be inserted into the stack at one time.

$$\delta(q_i, a, z_0) = (q_j, az_0)$$



Representation of States

(2) **POP** – one symbol can be deleted from the stack at one time.

$$\delta(q_i, a, z_0) = (q_j, \varepsilon)$$



Representation of States

(3) SKIP – IT means no stack operation, status of the stack will remain same, before a after the operation

$$\delta(q_i, a, z_0) = (q_j, z_0)$$

note- if pda perform a push or a pop operation at least one's during processing of string than we say that pda is using the stack.

Instantaneous Description (ID)

- An **instantaneous description (ID)** is (q, x, α) , where $q \in Q$, $x \in \Sigma^*$ and $\alpha \in \Gamma^*$.
- An initial ID is (q_0, x, Z_0) , this means that initially the pda is in the initial state q_0 , the input string to be processed is x and the PDS has only one symbol, namely Z_0 .
- **ACCEPTANCE BY PDA** There is no change in the language acceptance capability of the pda either we accept by final state or empty stack.

Q Design a Deterministic Push Down Automata for $L = \{a, ab\}$?



Q Design a Deterministic Push Down Automata for $\{a^n b^n \mid n \geq 1\}$?



Q Design a Deterministic Push Down Automata for $\{a^n b^{2n} \mid n \geq 1\}$?



Q Design a PDA for $\{w c w^r \mid w \in (a, b)^*\}$?



Q Construct PDA that accepts $L = \{w|_{a=b} \mid w \in (a, b)^*\}$?



Q consider the following mapping and find the correct language?

$$\delta(q_0, 1, z_0) = (q_0, xz_0)$$

$$\delta(q_0, 1, x) = (q_0, xx)$$

$$\delta(q_0, 0, x) = (q_1, \epsilon)$$

$$\delta(q_1, 0, x) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, x) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_F, z_0)$$

a) $L = \{a^m b^n \mid m = n\}$

b) $L = \{a^m b^n \mid m \neq n\}$

c) $L = \{a^m b^n \mid m \geq n\}$

d) $L = \{a^m b^n \mid m \leq n\}$



Q consider the following mapping and find the correct language?

$$\delta(q_0, a, z_0) = (q_1, z_0)$$

$$\delta(q_0, b, z_0) = (q_2, z_0)$$

$$\delta(q_1, a, z_0) = (q_1, z_0)$$

$$\delta(q_1, b, z_0) = (q_2, z_0)$$

$$\delta(q_2, a, z_0) = (q_2, z_0)$$

$$\delta(q_2, b, z_0) = (q_0, z_0)$$

$$\delta(q_1, \epsilon, z_0) = (q_f, z_0)$$

a) ending with a

b) ending with a, contain even number of a

c) ending with a, contain odd number of a

d) ending with a, contain even number of b



Non- Deterministic PDA

- Non- Deterministic PDA can also be defined using 7 tuples.
- $\delta: Q \times \{\Sigma \cup \epsilon\} \times \Gamma \rightarrow 2^{(Q \times \Gamma^*)}$
- i.e. on a given input symbol and stack symbol a NPDA can move to more than one state.
- Rest all other tuples are same as DPDA.

KnowledgeGate.AI

Q Construct pda that accepts a language $L = \{w w^r \mid w \in (a, b)^*\}$?



Module

Paper-2 | Theory of Computation

Topic

Pushdown Automata & CFG

Subtopic

Context-Free Grammars

Lesson

Context-Free Grammars

Pushdown Automata & CFG

Context-Free Grammars

KnowledgeGate.AI

Linear Grammar in Computer Science

- A **linear grammar** is a type of context-free grammar where each production's right-hand side contains at most one nonterminal symbol.
- Example of a simple linear grammar:
- Let G be a grammar with nonterminal set $N=\{S\}$, terminal set $\Sigma=\{a,b\}$, and the start symbol S .
- Production rules:
 - $S \rightarrow aSb$
 - $S \rightarrow \epsilon$
 - This grammar generates the language $L=\{a^n b^n | n \geq 0\}$, which is a **linear language**.
- **Types of Linear Grammars:**
 - **Left-Linear Grammars:** Nonterminal symbols appear only at the left ends of the right-hand sides.
 - **Right-Linear Grammars:** Nonterminal symbols appear only at the right ends of the right-hand sides.
 - Both left-linear and right-linear grammars generate **regular languages**.

- **General Linear Grammars:**
 - Nonterminal symbols can appear at either end of the production, but not necessarily always at the same end.
 - Any linear grammar can be transformed into this form without changing the language generated.
- For example, the grammar:
 - $S \rightarrow aAS$
 - $A \rightarrow SbA$
 - $S \rightarrow \epsilon$
 - generates the same language as the example above.

KnowledgeGate.AI

- **Relation to Other Languages:**

- All **regular languages** are linear, but not all linear languages are regular. For example, $\{a^n b^n \mid n \geq 0\}$ is linear but non-regular.
- All linear languages are **context-free**, but not all context-free languages are linear. For example, the well-balanced bracket language is context-free but not linear.

- **Determinism and Complexity:**

- Regular linear languages are **deterministic**, but some linear languages are **nondeterministic**.
- For example, the language of even-length **palindromes** (generated by the grammar $S \rightarrow 0S0 \mid 1S1 \mid \epsilon$) is nondeterministic.
- **Nondeterministic** linear languages cannot always be recognized in **linear time**.
- Moreover, it is **undecidable** whether a given context-free language is linear.

KnowledgeGate.AI

Module

Paper-2 | Theory of Computation

Topic

Pushdown Automata & CFG

Subtopic

Decision Properties

Lesson

Decision Properties

Pushdown Automata & CFG

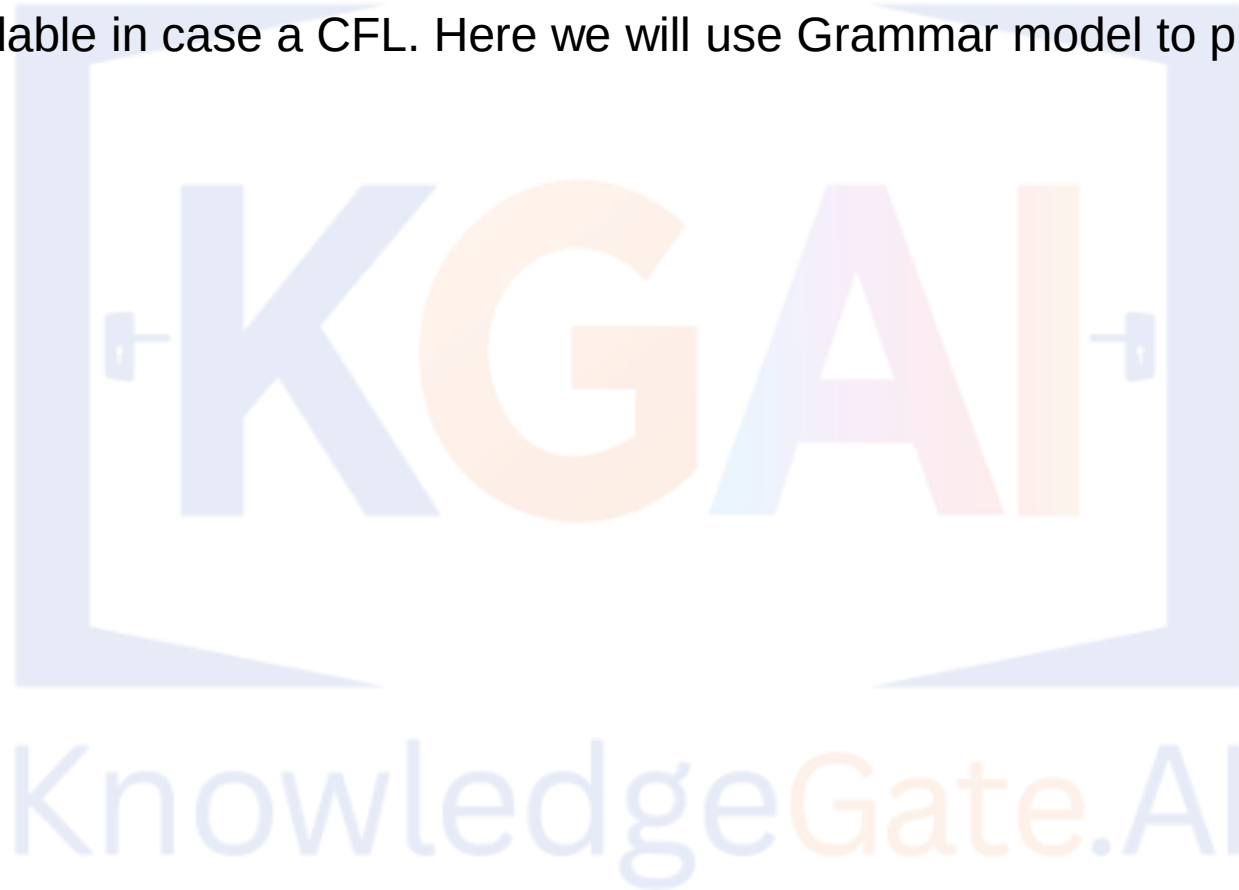
Decision Properties

KnowledgeGate.AI

Decision properties

Following properties are decidable in case a CFL. Here we will use Grammar model to proof decision properties.

- i)** Emptiness
- ii)** Non-emptiness
- iii)** Finiteness
- iv)** Infiniteness
- v)** Membership



Q consider the following CFG and identify which of the following CFG generate Empty language?

$S \rightarrow aAB / Aa$

$A \rightarrow a$

$S \rightarrow aAB$

$A \rightarrow a / b$

$S \rightarrow aAB / aB$

$A \rightarrow aBb$

$B \rightarrow aA$



Q consider the following CFG and identify which of the following CFG generate Finite language?

$S \rightarrow SS \mid AB$

$A \rightarrow BC \mid a$

$B \rightarrow CC \mid b$

$S \rightarrow AB$

$A \rightarrow B \mid a$

$S \rightarrow AB$

$A \rightarrow BC \mid a$

$B \rightarrow CC \mid b$

$C \rightarrow AB$



Q consider the following CFG and check out the membership properties?

$S \rightarrow AB \mid BB$

$A \rightarrow BA \mid AS \mid b$

$B \rightarrow AA \mid SB \mid a$

$w_1 = aba$

$w_2 = abaab$

$w_3 = abababba$



CYK algorithm

- In computer science, the **Cocke–Younger–Kasami algorithm** (alternatively called **CYK**, or **CKY**) is a parsing algorithm for context-free grammars, named after its inventors, John Cocke, Daniel Younger and [Tadao Kasami](#). It employs bottom-up parsing and dynamic programming.
- The standard version of CYK operates only on context-free grammars given in Chomsky normal form (CNF). However any context-free grammar may be transformed to a CNF grammar expressing the same language (Sipser 1997).
- The importance of the CYK algorithm stems from its high efficiency in certain situations. Using Big O notation, the worst case running time of CYK is $O(n^3 \cdot |G|)$, Where n is the length of the parsed string and $|G|$ is the size of the CNF grammar G .
- This makes it one of the most efficient parsing algorithms in terms of worst-case [asymptotic complexity](#), although other algorithms exist with better average running time in many practical scenarios.

Following properties are Undecidable in case a CFL

i) Equality

ii) Ambiguity



	RL	DCFL	CFL	CSL	RS	RES
Emptiness	Y	Y	Y	X	N	N
Non-Emptiness	Y	Y	Y	X	N	N
Finiteness	Y	Y	Y	X	N	N
Infiniteness	Y	Y	Y	X	N	N
Membership	Y	Y	Y	X	Y	N
Equality	Y	N	N	X	N	N
Ambiguity	Y	N	N	X	N	N
Σ^*	Y	N	N	X	N	N
Halting	Y	Y	Y	X	Y	N

Module

Paper-2 | Theory of Computation

Topic

Pushdown Automata & CFG

Subtopic

Closure Properties

Lesson

Closure Properties

Pushdown Automata & CFG

Closure Properties

KnowledgeGate.AI

Closure Properties of Deterministic Context Free Languages

- **Deterministic Context Free Languages** are closed under following operations
 - Complement
 - Intersection with regular set
 - Inverse Homeomorphism
- **Deterministic Context Free Languages** are not closed under following operations
 - Union
 - Concatenation
 - Kleen closure
 - homomorphism
 - Substitution
 - Reverse operator
 - Intersection

Closure Properties of Context Free Languages

- **Context Free Languages** are closed under following operations
 - Union
 - Concatenation
 - Kleen Closure
 - Substitution
 - Homomorphism
 - Inverse Homomorphism
 - Reverse Operator
 - Intersection with regular set
- **Context Free Languages** are not closed under following operations
 - Intersection
 - Complement
 - Symmetric Difference

	RL	DCFL	CFL	CSL	RS	RES
Union	Y	N	Y	Y	Y	Y
Intersection	Y	N	N	Y	Y	Y
Complement	Y	Y	N	Y	Y	N
Set Difference	Y	N	N	Y	Y	N
Kleene Closure	Y	N	Y	Y	Y	Y
Positive Closure	Y	N	Y	Y	Y	Y
Concatenation	Y	N	Y	Y	Y	Y
Intersection with regular set	Y	Y	Y	Y	Y	Y
Reverse	Y	Y	Y	Y	Y	Y
Subset	N	N	N	N	N	N

	RL	DCFL	CFL	CSL	RS	RES
Homomorphism	Y	N	Y	N	N	Y
∈ Free Homomorphism	Y	N	Y	Y	Y	Y
Inverse Homomorphism	Y	Y	Y	Y	Y	Y
Substitution	Y	N	Y	N	N	Y
∈ Free Substitution	Y	N	Y	Y	Y	Y
Quotient with regular set	Y	Y	Y	N	Y	Y

Module

Paper-2 | Theory of Computation

Topic

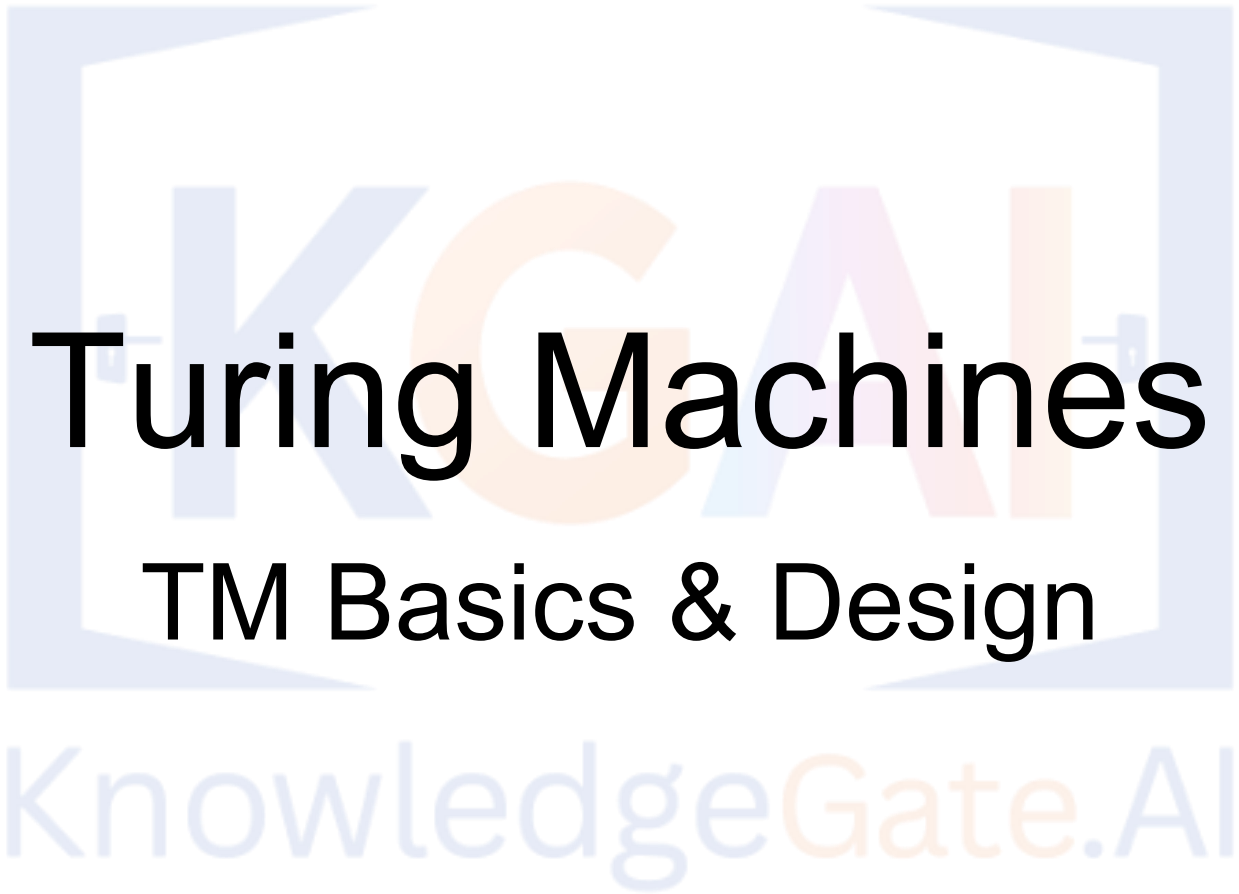
Turing Machines

Subtopic

TM Basics & Design

Lesson

TM Basics & Design

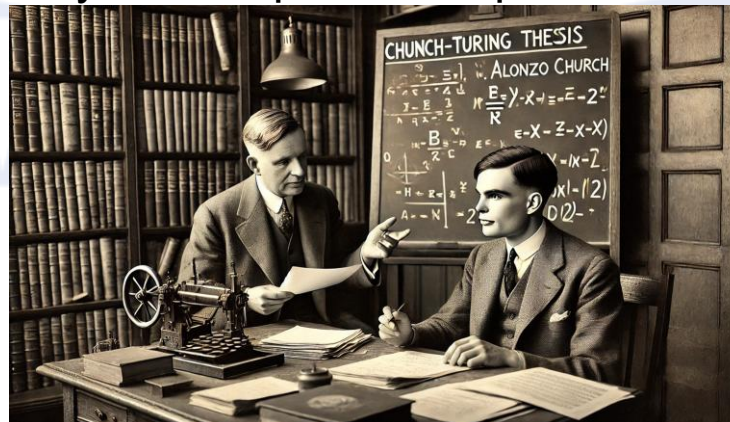
The background features a large, faint watermark of the KnowledgeGate.AI logo. It consists of a blue square frame containing the letters 'KGA' in a stylized font. The 'K' is blue, the 'G' is orange, and the 'A' is pink. Below the frame, the text 'KnowledgeGate.AI' is written in a light blue font, with 'Gate' in orange.

Turing Machines

TM Basics & Design

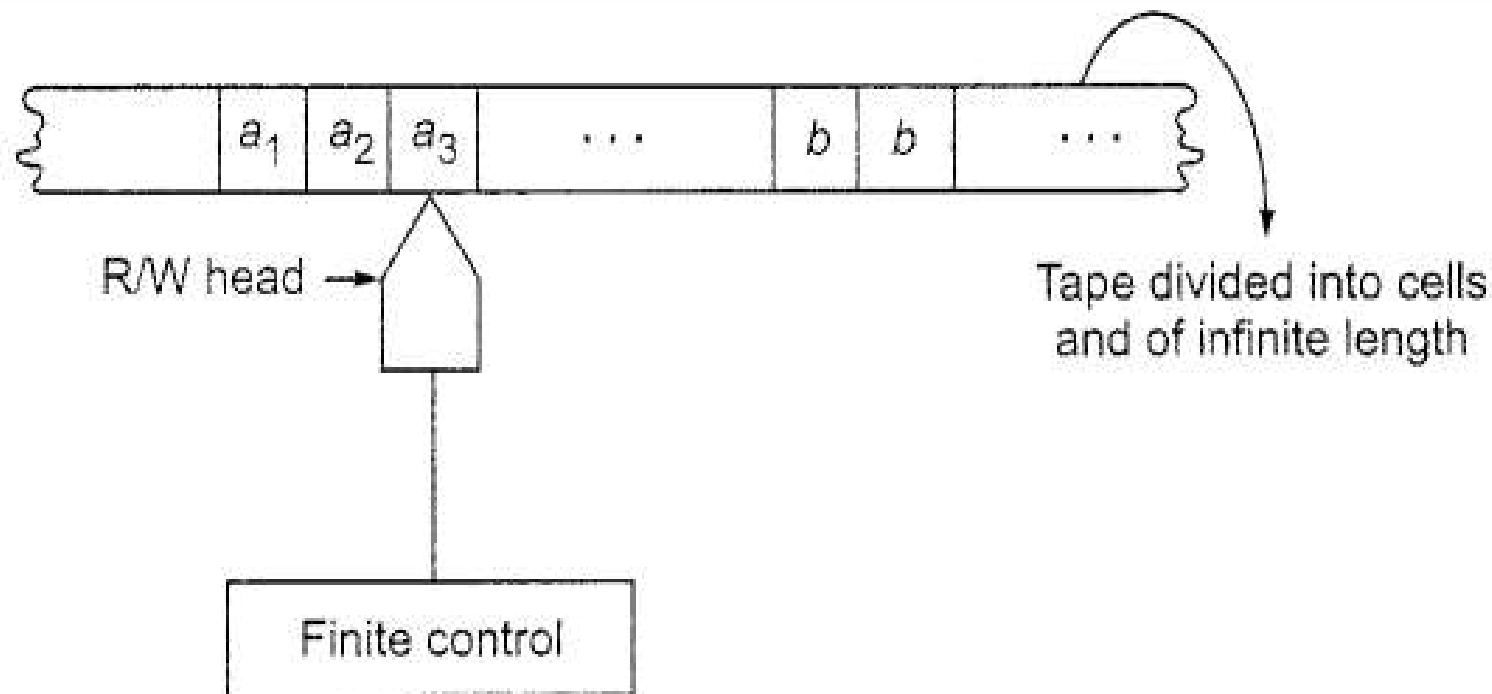
Turing Machine

- The **Church-Turing thesis** posits that any computational task performable by a human or machine can be executed by a Turing machine. It is widely regarded as a fundamental theoretical model of computation in computer science.
- Turing machines serve several key purposes:
 - **General Automaton:** Turing machines are the most general form of automaton, capable of simulating any algorithmic process.
 - **Type-0 Language Acceptance:** They can recognize type-0 languages, which are the most complex in the Chomsky hierarchy.
 - **Function Computation:** Turing machines can compute any computable function.
 - **Undecidability & Complexity:** They help determine the undecidability of certain languages and are used to measure both time and space complexity for computational problems.



- **Components of a Turing Machine:**

- **Infinite Tape**: The tape acts as both input storage and memory. It is infinitely long and divided into cells, each capable of holding a single symbol. The tape can be accessed randomly in either direction, referred to as a two-way infinite tape.
- **Read-Write Head**: The head reads symbols from the tape and can also write over them. After reading or writing, it moves one cell either to the left or right.
- **Finite Control Unit**: The control unit governs transitions based on the current state and symbol under the read-write head. It determines the next action, ultimately leading to the machine's output.



FORMAL DEFINITION

A Turing machine M is a 7-tuple, namely $(Q, \Sigma, \Gamma, \delta, q_0, b, F)$, where

- Q is a finite nonempty set of states.
- Γ is a finite nonempty set of tape symbols,
- $b \in \Gamma$ is the blank.
- Σ is a nonempty set of input symbols and is a subset of Γ and $b \notin \Sigma$.
- δ is the transition function mapping $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times (L/R)$. It says that, on providing a tape symbol, from a particular state there will be a transition to another state, with a different or same tape symbol with defining whether next the machine needs to move in left/ right.
- $q_0 \in Q$ is the initial state, and
- $F \subseteq Q$ is the set of final states.

Q Design a Turing machine for $L = \{ab\}$?



Q Design a Turing machine for $L = \{a, ab\}$?



Q Consider the following machines and corresponding transition function

Machine

Transition function

1. DFA

$$A. Q \times \Sigma \cup \{\epsilon\} \times \Gamma \rightarrow Q \times \Gamma^*$$

2. NFA

$$B. Q \times \Sigma \rightarrow 2^Q$$

3. PDA

$$C. Q \times \Sigma \rightarrow Q$$

4. TM

$$D. Q \times \Gamma \rightarrow \underline{Q} \times \Gamma \times \{L, R\}$$

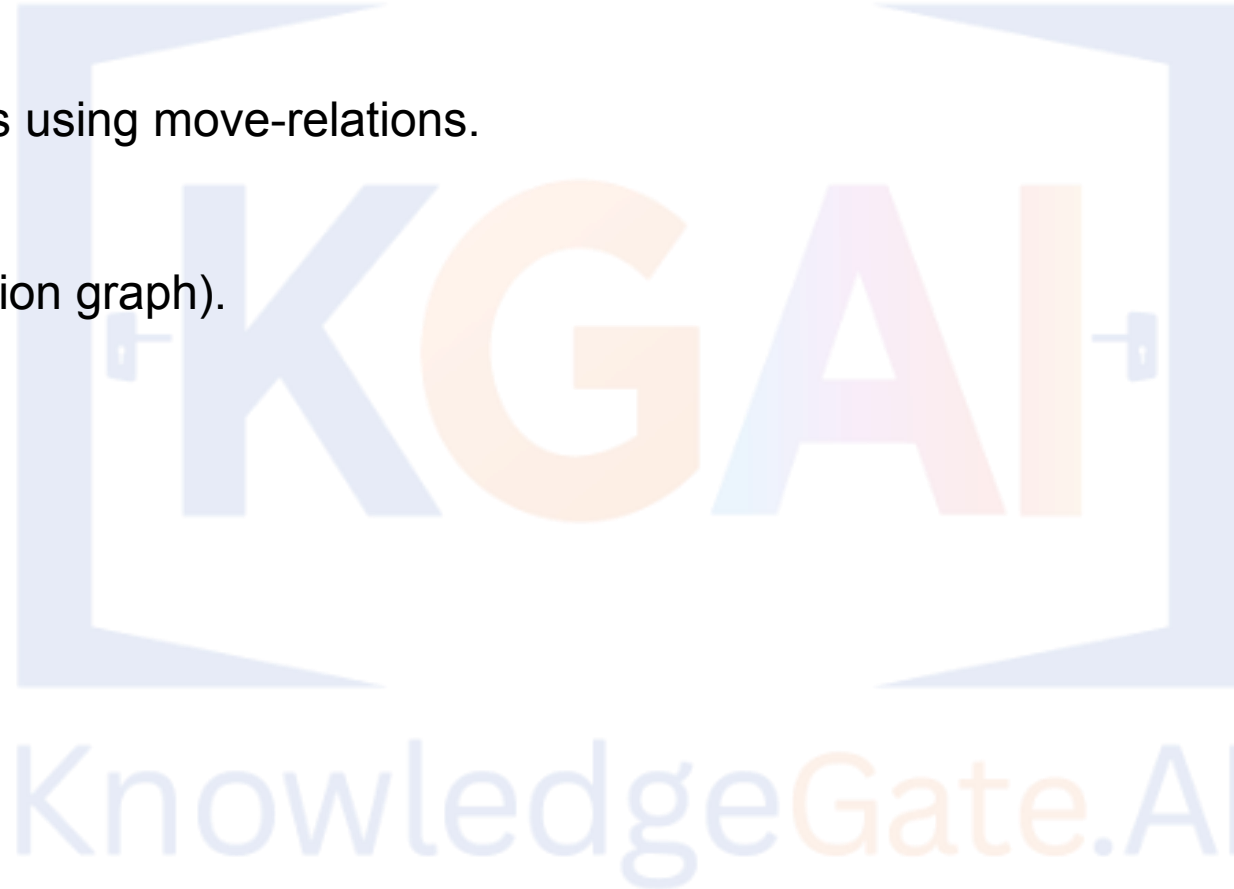
$$E. Q \times \Sigma \rightarrow \Delta$$

$$F. Q \rightarrow \Delta$$

REPRESENTATION OF TURING MACHINES

Three representations:

- Instantaneous descriptions using move-relations.
- Transition table, and
- Transition diagram (transition graph).



LANGUAGE ACCEPTABILITY BY TURING MACHINES

- A string w in Σ^* is said to be accepted by a $TM(M)$ if after parsing the string w Turing machine must halts on final state.
- $q_0w \vdash^* \alpha_1p\alpha_2$ for some $p \in F$ and $\alpha_1, \alpha_2 \in \Gamma^*$.
- M does not accept w if the machine M either halts in a non-accepting state or does not halt and goes into a loop.

KnowledgeGate.AI

Q Design a Turing machine for $L = \{a^n b^n \mid n \geq 1\}$?



Q Design a Turing machine for $L = \{a^n b^n c^n \mid n \geq 0\}$?



Q Design a Turing machine for $L = \{w c w \mid w \in \{0, 1\}^*\}$?



Q Design a Turing machine for addition of two number in unary?



Q Design a Turing machine for converting unary number into binary number?



Q A single tape Turing Machine M has two states q_0 and q_1 , of which q_0 is the starting state. The tape alphabet of M is $\{0, 1, B\}$ and its input alphabet is $\{0, 1\}$. The symbol B is the blank symbol used to indicate end of an input string. The transition function of M is described in the following table. Which of the following statements is true about M ?

- (A)** M does not halt on any string in $(0 + 1)^+$
- (B)** M does not halt on any string in $(00 + 1)^*$
- (C)** M halts on all string ending in a 0
- (D)** M halts on all string ending in a 1

	0	1	B
q_0	$q_1, 1, R$	$q_1, 1, R$	Halt
q_1	$q_1, 1, R$	$q_0, 1, L$	q_0, B, L

Q Consider the transition table of a TM given below. Here “b” represents the blank symbol. Which of the following strings will not accepted?

a) 010101

b) 101010

c) 110011

d) Both a and b

δ	0	1	b
q_0	q_0 0R	q_1 0R	q_2 0R
q_1	q_1 0R	q_0 0R	-
q_2	-	-	-

Q The given Turing machine accepts

- a)** set of all even palindromes over $\{0, 1\}$
- b)** strings over $\{0, 1\}$ containing even number of 1's
- c)** strings over $\{0, 1\}$ containing even number of 1's and odd no. of 0's
- d)** string over $\{0, 1\}$ starting with zero

δ	0	1	b
q_0	q_0 0R	q_1 0R	q_2 0R
q_1	q_1 0R	q_0 0R	-
q_2	-	-	-

KnowledgeGate.AI

- **Deterministic and Non-Deterministic Turing Machines:**
 - **Deterministic:** Each state and symbol pair leads to a single possible move.
 - **Non-Deterministic:** Multiple moves may be possible for a given state and symbol. However, non-deterministic Turing machines do not add computational power and can always be converted into a deterministic Turing machine.



Halt in a Turing Machine

- When a Turing machine reaches a state where no further transitions are defined or needed, it is said to **halt**. There are two types of halts:
 - **Final Halt**: The machine halts in a final (accepting) state, indicating that the input string is accepted.
 - **Non-Final Halt**: The machine halts in a non-final state, meaning the input string is rejected.
- After processing an input string, a Turing machine can experience one of three outcomes:
 - **Final Halt**: The machine accepts the string.
 - **Non-Final Halt**: The machine rejects the string.
 - **Infinite Loop**: The machine enters an infinite loop, in which case it is undecidable whether the string is accepted or rejected.

KnowledgeGate.AI

Recursively Enumerable Languages

- **Recursive Set:**
 - A language L is a recursive set if it is accepted by a Turing machine in such a way that:
 - For all $w \in L$, the machine halts in a final state (accepts).
 - For all $w \notin L$, the machine halts in a non-final state (rejects).
 - The membership property is clearly defined here—every string is either accepted or rejected.
- **Recursively Enumerable Set (REL):**
 - A language L is recursively enumerable if it is accepted by a Turing machine, where:
 - For all $w \in L$, the machine halts in a final state (accepts).
 - For all $w \notin L$, the machine may either halt in a non-final state (reject) or enter an infinite loop.
 - The membership property is not defined because the machine might never halt for certain strings.
- **Decidability:**
 - A set is **decidable** if both the set and its complement are recognizable by a Turing machine.
 - Some sets are **not recognizable**, meaning that even the members of the set cannot be identified, as there are more languages than programs available to recognize them.

Module

Paper-2 | Theory of Computation

Topic

Turing Machines

Subtopic

TM Variations & UTM

Lesson

TM Variations & UTM

The logo for KnowledgeGate.AI is a large, light blue square with rounded corners. Inside the square, the letters 'K', 'G', and 'A' are prominently displayed in a large, stylized font. The 'K' is light blue, the 'G' is light orange, and the 'A' is light purple. The letters are slightly overlapping and have a soft, glowing effect.

Turing Machines

TM Variations & UTM

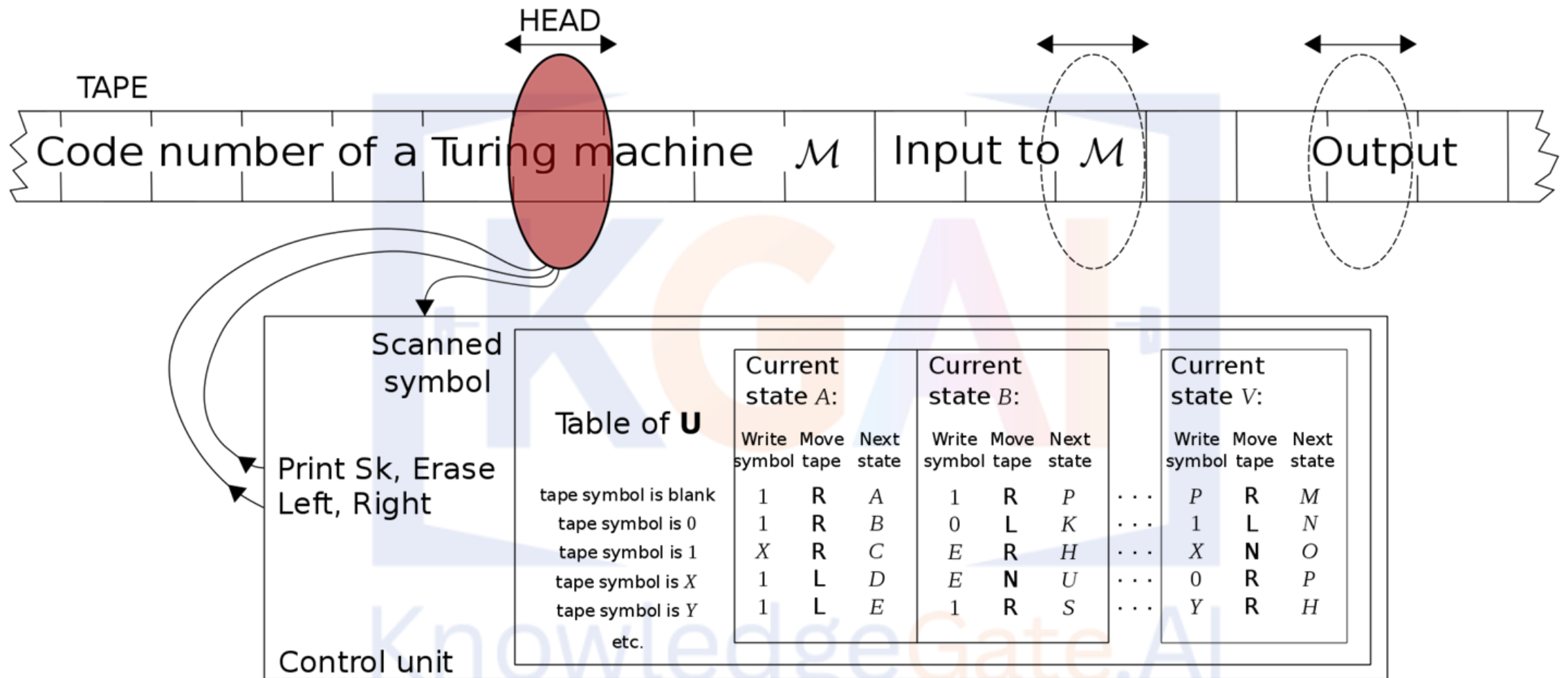
KnowledgeGate.AI

Versions of Turing Machines:

- **Multi-Tape Turing Machine:** Uses multiple tapes with corresponding read/write heads. However, every multi-tape machine can be simulated by a single-tape machine.
- **Multi-Head Turing Machine:** A machine with multiple read/write heads.
- **Multi-Dimensional Turing Machine:** Operates on tapes of multiple dimensions (e.g., 2D grid).
- **Turing Machine with Stay Option:** The head can remain in place instead of moving left or right.
- **One-Way Infinite Tape (Semi-Infinite Tape):** The tape is infinite in only one direction.
- **Offline Turing Machine:** The input tape is read-only and cannot be modified.
- **Jumping Turing Machine:** Can make multiple moves in one transition.
- **Non-Erasing Turing Machine:** Cannot erase or overwrite the input symbols with blanks.
- **Always Writing Turing Machine:** Must replace every symbol it reads with another symbol.
- **Finite Automaton with Queue:** Combines a finite automaton with an unbounded queue.
- **Turing Machine with 3 States:** A simplified Turing machine with only 3 states.
- **Multi-Tape TM with Stay Option and 2 States:** A variation with multiple tapes and stay options but limited to two states.
- **Non-Deterministic TM:** Similar to the non-deterministic Turing machine but with a stay option.
- **NPDA with Two Stacks:** A non-deterministic pushdown automaton with two independent stacks.

Universal Turing Machine (UTM)

- In computer science, a **Universal Turing Machine** (UTM) is a Turing machine capable of simulating any other Turing machine on any input. It does this by reading both the description of the machine to be simulated and the input for that machine from its own tape.
 - Every Turing machine computes a fixed **partial computable function** from input strings over its alphabet, behaving like a computer with a specific program.
 - The action table of any Turing machine can be encoded into a string. A UTM can then read this string, which describes the action table, followed by a string that describes the input tape, and simulate the encoded Turing machine's behavior.
- In his 1936 paper, Turing described this concept in detail, stating:
 - *"It is possible to invent a single machine which can be used to compute any computable sequence. If this machine UUU is supplied with a tape containing the 'standard description' (S.D.) of some machine MMM, UUU will compute the same sequence as MMM."*



Module

Paper-2 | Theory of Computation

Topic

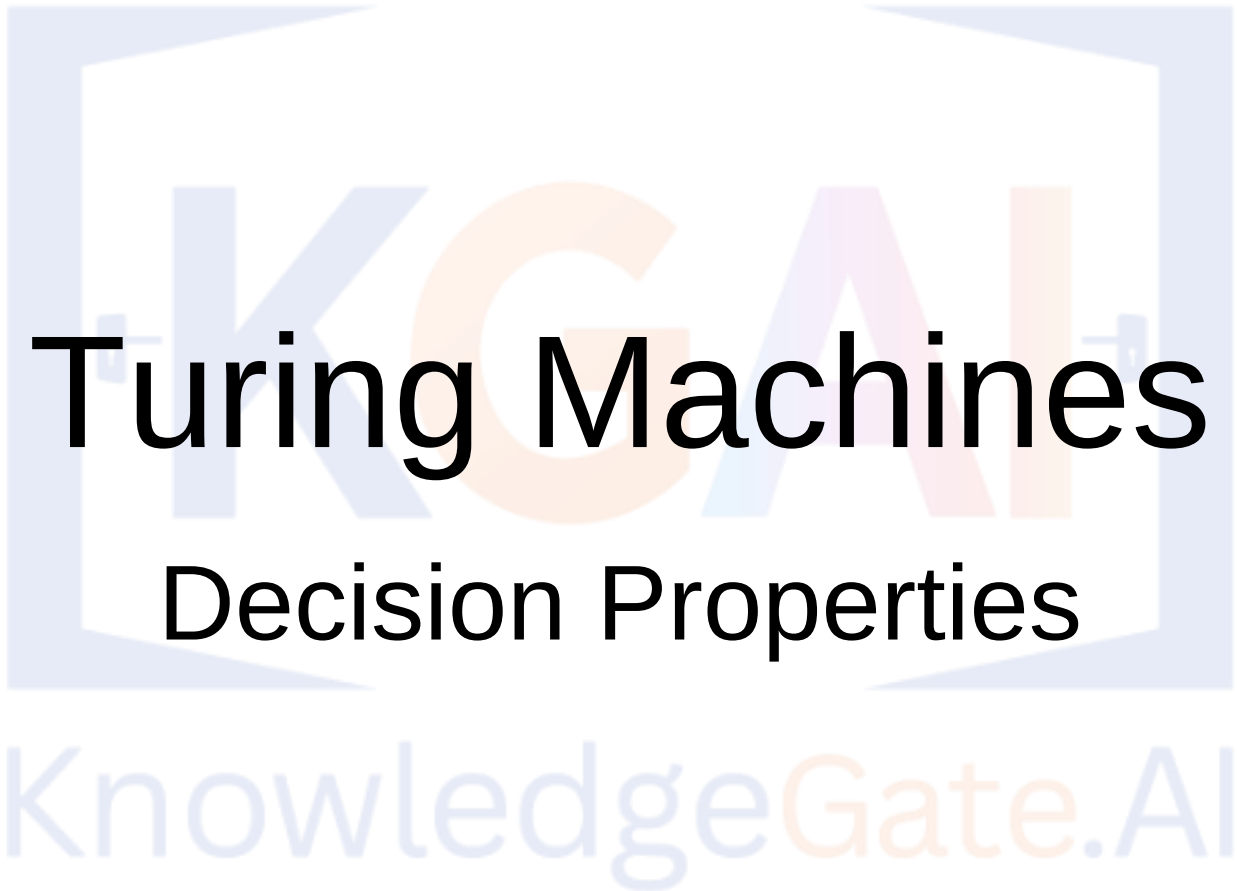
Turing Machines

Subtopic

Decision Properties

Lesson

Decision Properties

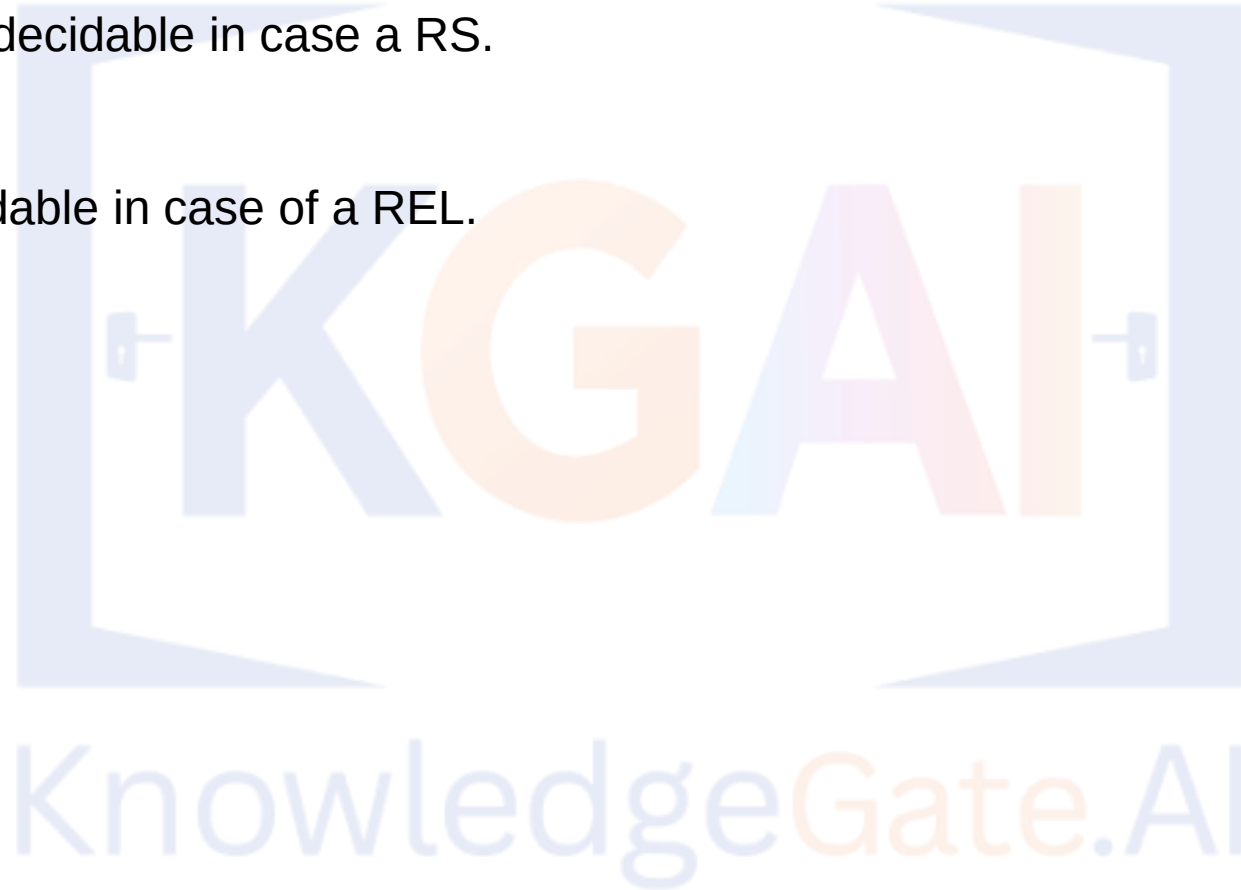
The background features a large, faint watermark of the KnowledgeGate.AI logo. It consists of a blue square frame containing the letters 'KGA' in a stylized font. The 'K' is blue, the 'G' is orange, and the 'A' is pink. Below the frame, the text 'KnowledgeGate.AI' is written in a light blue font, with 'Gate' in orange.

Turing Machines

Decision Properties

Decision properties

- Following properties are decidable in case a RS.
 - Membership
- All properties are undecidable in case of a REL.



Module

Paper-2 | Theory of Computation

Topic

Turing Machines

Subtopic

Closure Properties

Lesson

Closure Properties

The background features a large, faint watermark of the KnowledgeGate.AI logo. It consists of a blue square frame containing the letters 'KGA' in a stylized font. The 'K' is blue, the 'G' is orange, and the 'A' is purple. Below the frame, the text 'KnowledgeGate.AI' is written in a light blue font, with 'Gate' in orange.

Turing Machines

Closure Properties

Closure Properties of Recursive Set

- Recursive languages are closed under following operations
 - Union
 - Concatenation
 - Intersection
 - Reverse
 - Complement
 - Inverse homomorphism
 - Intersection with regular set
 - Set Difference
 - Kleen closure
- Recursive languages are not closed under following operations
 - Homomorphism
 - Substitution

Closure Properties of Recursive Enumerable Set

- Recursive Enumerable are closed under following operations
 - Union
 - Concatenation
 - Kleen Closure
 - Intersection
 - Substitution
 - Homomorphism
 - Inverse Homomorphism
 - Intersection with regular set
 - Reverse operation
- Recursive Enumerable are not closed under following operations
 - Compliment
 - Set Difference

	RL	DCFL	CFL	CSL	RS	RES
Union	Y	N	Y	Y	Y	Y
Intersection	Y	N	N	Y	Y	Y
Complement	Y	Y	N	Y	Y	N
Set Difference	Y	N	N	Y	Y	N
Kleene Closure	Y	N	Y	Y	Y	Y
Positive Closure	Y	N	Y	Y	Y	Y
Concatenation	Y	N	Y	Y	Y	Y
Intersection with regular set	Y	Y	Y	Y	Y	Y
Reverse	Y	Y	Y	Y	Y	Y
Subset	N	N	N	N	N	N

KnowledgeGate.AI

	RL	DCFL	CFL	CSL	RS	RES
Homomorphism	Y	N	Y	N	N	Y
∈ Free Homomorphism	Y	N	Y	Y	Y	Y
Inverse Homomorphism	Y	Y	Y	Y	Y	Y
Substitution	Y	N	Y	N	N	Y
∈ Free Substitution	Y	N	Y	Y	Y	Y
Quotient with regular set	Y	Y	Y	N	Y	Y

KnowledgeGate.AI

Module

Paper-2 | Theory of Computation

Topic

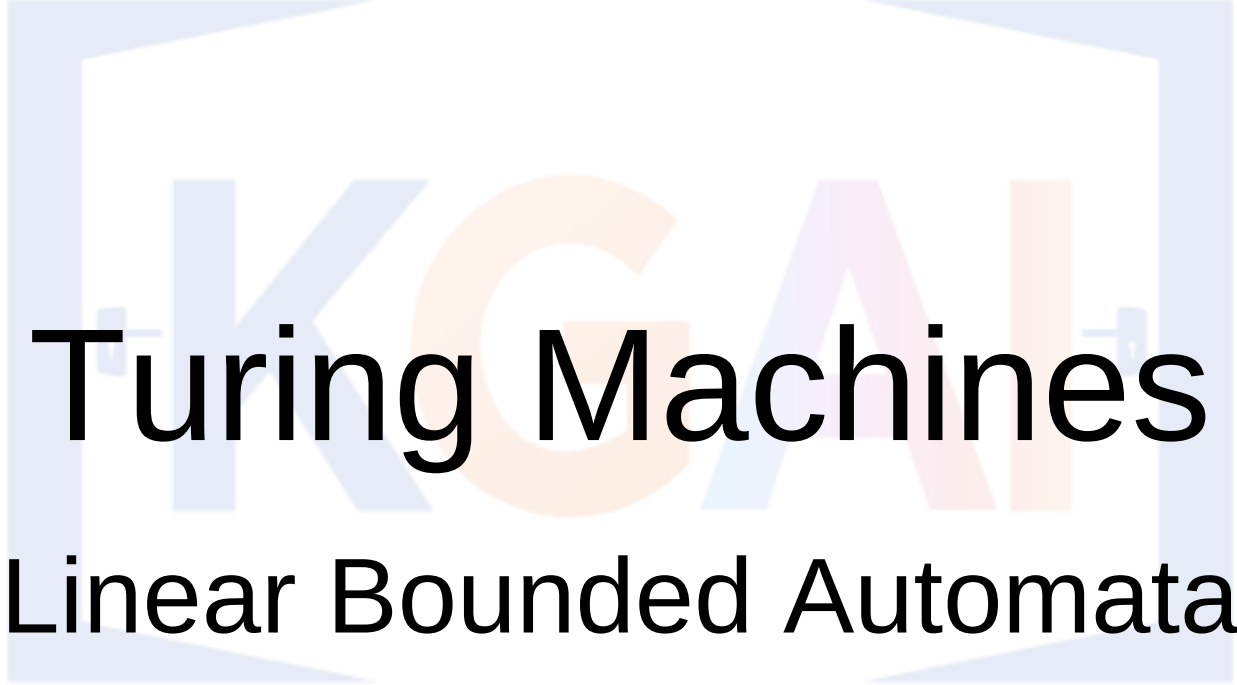
Turing Machines

Subtopic

Linear Bounded Automata

Lesson

Linear Bounded Automata

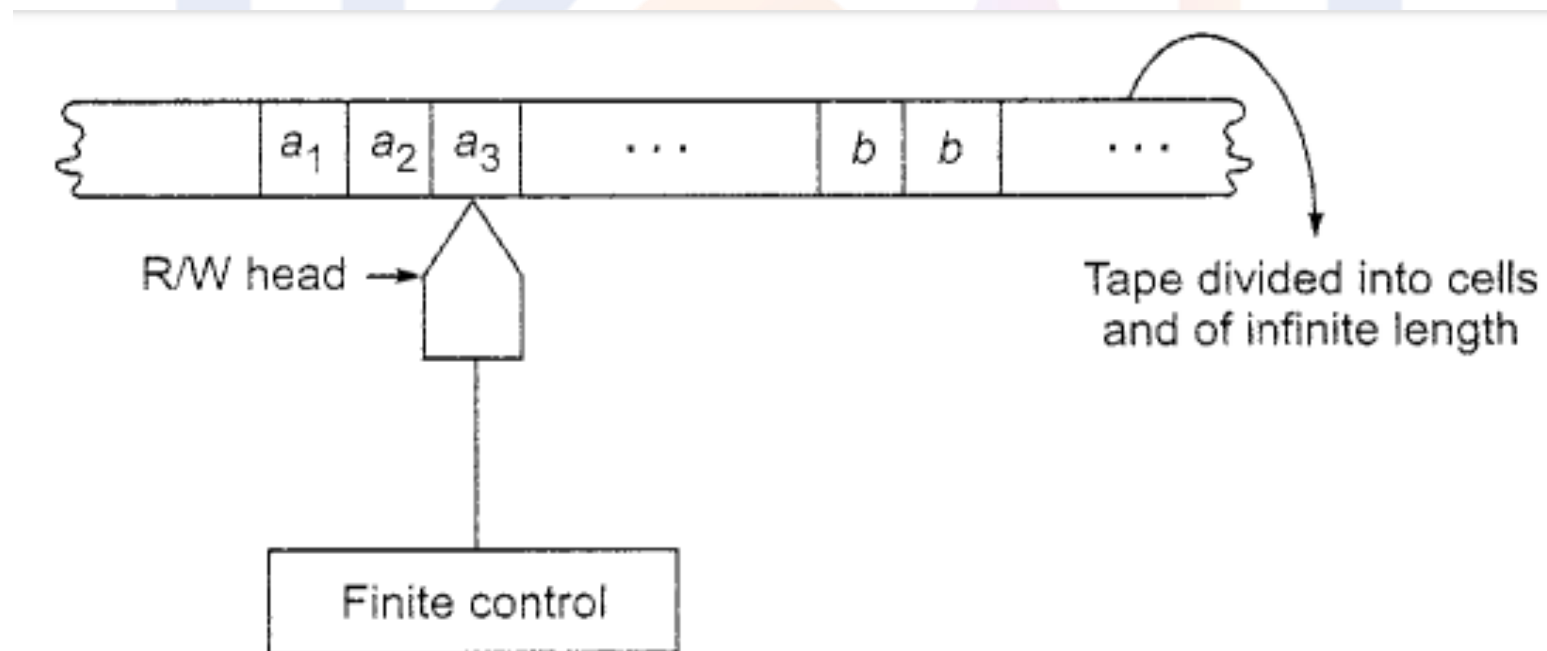
The logo for KnowledgeGate.AI is centered in the background. It features a large, stylized 'KGA' in blue, orange, and purple colors, enclosed within a blue square frame. Below the frame, the text 'KnowledgeGate.AI' is written in a light blue, sans-serif font.

Turing Machines

Linear Bounded Automata

Linear Bounded Automaton (LBA)

- An **LBA** is a variant of a Turing machine with the following property: The tape length is finite and bounded, unlike a traditional Turing machine that has an infinite tape.



	RL	DCFL	CFL	CSL	RS	RES
Emptiness	Y	Y	Y	X	N	N
Non-Emptiness	Y	Y	Y	X	N	N
Finiteness	Y	Y	Y	X	N	N
Infiniteness	Y	Y	Y	X	N	N
Membership	Y	Y	Y	X	Y	N
Equality	Y	N	N	X	N	N
Ambiguity	Y	N	N	X	N	N
Σ^*	Y	N	N	X	N	N
Halting	Y	Y	Y	X	Y	N